

Web-Based Learning System

Project Documentation

A complete ASP.NET Core MVC learning management system: course catalogue, lessons, quizzes and progress tracking for students, with a full administration area. Deployed on Railway with a Firebase Hosting presentation site.

Author	Khaytboy Khayrullaev (TP072305)
Module	CT050-3-2-WAPP Web Applications
Institution	Asia Pacific University of Technology & Innovation
Assessment	Group assignment, carried out individually
Live application	learnhub-production-7081.up.railway.app
Project website	learnhub-wapp.web.app
Repository	github.com/KhayitOff/LearnHub
Generated	14 September 2026

Contents

Proposal	2
Requirement Traceability	7
Requirements Checklist	19
Use Cases	24
Navigation Structure	30
Wireframes	33
Flowcharts	40
Architecture	51
Database Design	57
Entity Relationship Diagram	61
Security Review	64
Testing	68
Deployment	73
User Guide	80
Report Notes	90
Git Workflow	110
Team and Responsibilities	113
Viva Preparation	115

Proposal

Project title	LearnHub – Web-Based Learning Management System
Module	CT050-3-2-WAPP Web Applications (group assignment)
Institution	Asia Pacific University of Technology & Innovation (APU)
Team	Khaytboy Khayrullaev (TP072305) — carried out individually
Technology	ASP.NET Core MVC on .NET 10, C#, Razor, Bootstrap 5.3, Entity Framework Core 10 with SQLite, ASP.NET Core Identity

1. Background and problem

Computing students learn from many scattered sources: lecture slides, videos, blog posts and cheat sheets. Three problems follow:

1. **No structure.** Material has no clear order, so beginners do not know what to study next or how far they are from finishing a topic.
2. **No feedback.** Reading and watching give no evidence of understanding. Students need short quizzes with explanations, not only a score.
3. **No single place to manage content.** Lecturers and teaching assistants need one tool to publish courses, lessons, files and quizzes, see who is learning and remove outdated material, without editing web pages by hand.

2. Proposed solution

LearnHub is a web application in which:

- **guests** explore a catalogue of computing courses, read course details and try free preview lessons;
- **students** register, enrol in courses, work through lessons in order, mark them complete, take quizzes with explained answers and follow their progress on a personal dashboard;
- **administrators** manage courses, categories, lessons, uploads, quizzes, enrolments, users and contact messages in a protected admin area with live statistics.

The interface uses one idea throughout: LearnHub is an **interchange**. Each subject category is a coloured transit line, a course is a route, lessons are stops, and progress is how far along the route a student has travelled.

3. Mission statement

LearnHub gives every computing student a clear route through each subject, with honest feedback at every stop, and gives administrators one simple place to keep that route up to date.

4. Objectives

#	Objective (SMART)	Measure of success
O1	Deliver a public catalogue in which guests can search, filter and open published courses within the first four weeks.	Search, category and difficulty filters and course details pages work, and preview lessons open without an account.
O2	Let a student register, enrol, complete lessons and take a quiz in one continuous journey.	An automated end-to-end test covers register → enrol → lesson → quiz → result.
O3	Show each student meaningful progress data on a dashboard built from real database records.	Dashboard shows enrolled, completed and available courses, lessons completed, quiz results, recent activity and recommendations.
O4	Provide administrators with full create, read, update and delete management of all learning content, enrolments and users.	Every admin entity has working list, create, edit and delete pages, tested through the real forms.
O5	Protect the application against the common web risks in the OWASP Top 10.	Security review completed; automated tests for access control, CSRF, XSS, uploads and logout pass.
O6	Make every page usable on phones, tablets and desktops and accessible to keyboard and screen-reader users.	Browser tests pass at three screen sizes with no WCAG 2.2 A/AA violations reported by axe-core.
O7	Ship the application as a Docker image to Railway, publish the static presentation site on Firebase Hosting, and run automated tests before every delivery.	The three CI jobs (build-and-test , container , e2e) pass; the presentation site is live and links to the application; the Railway deployment files and commands are complete and ready to run.

5. Target audience

Persona 1 – Nurul, first-year computing student

Age and situation	19, Diploma in IT, lives in a hostel, studies between classes
Devices	Mid-range Android phone on campus Wi-Fi; shared laptop in the library

Goals	Understand programming and databases fundamentals before assessments; know what to study next
Frustrations	Long videos without structure; not knowing whether she really understood a topic
What LearnHub offers	Short lessons in a fixed order, a Resume button on the dashboard, quizzes that explain every answer, and a layout that works on a phone

Persona 2 – Daniel, career switcher

Age and situation	31, works full time in logistics, learning web development in the evenings
Devices	Personal laptop; phone during commutes
Goals	Build practical skills in HTML, CSS, JavaScript, ASP.NET Core and security; see steady progress
Frustrations	Limited time; losing track of where he stopped; content that assumes prior knowledge
What LearnHub offers	Difficulty levels and durations on every course, progress bars, "Continue learning" with the next lesson, and PDF cheat sheets for quick revision

Persona 3 – Ms. Priya, lecturer and course administrator

Age and situation	42, lecturer responsible for networking and cloud modules
Devices	Office desktop; tablet in meetings
Goals	Publish and update course material quickly; check enrolments and quiz performance; keep content accurate
Frustrations	Complex tools; accidentally deleting material; no overview of learner activity
What LearnHub offers	A focused admin area with a dashboard, publish and unpublish without deleting, confirmation pages that show the impact of a deletion, validated uploads and quiz result reports

6. Scope

In scope

Role	Features
Guest	Home, About, Contact form, Privacy; course catalogue with search, filters, sorting and pagination; course details; free preview lessons; registration and login
Student	Dashboard; profile and password change; enrol and leave courses; My Courses; lessons (article, video, PDF, image, link); mark complete; progress; quizzes with instant marking and explanations; result history; logout
Administrator	Dashboard with statistics; courses (with cover images and publishing); categories; learning resources (with uploads); quizzes, questions and answer options; enrolments; quiz results; users (roles, deactivation, deletion); contact messages
Platform	ASP.NET Core Identity with Admin and Student roles; a single SQLite database with EF Core code-first migrations and demo data; security controls; responsive and accessible design; automated tests; CI; the application packaged as a Docker image for Railway; a Firebase Hosting presentation site

Out of scope for version 1.0

AI tutor, live video classes, chat, payments, certificates, email delivery (email confirmation and password reset by email), an instructor self-service role, discussion forums, native mobile apps and multiple interface languages.

7. Requirements summary

The complete, numbered list is in [REQUIREMENTS_CHECKLIST.md](#).

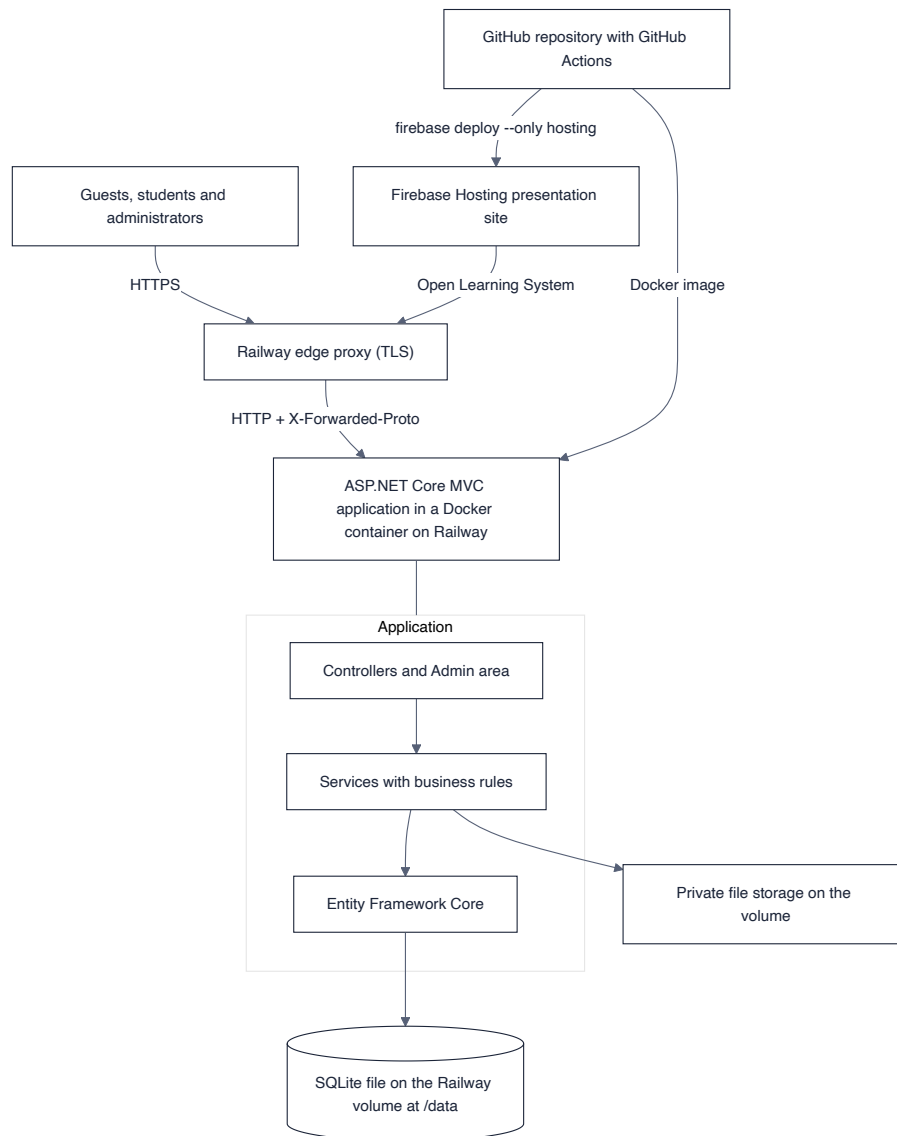
Group	IDs	Examples
Technology	TECH-01–11	.NET 10 MVC, EF Core, Identity, Razor, Bootstrap, JavaScript, multimedia
Guest	GUEST-01–10	Browse, search, filter, details, preview, register, login, contact
Student	STU-01–13	Dashboard, enrolment, lessons, progress, quizzes, results, profile
Administrator	ADM-01–11	Protected area, CRUD for all content, users, enrolments, results, messages
Database	DB-01–10	Keys, constraints, indexes, delete rules, migrations, seed data, ERD
Security and validation	AUTH, VAL, SEC	Password hashing, roles, CSRF, XSS, IDOR, overposting, upload safety
Quality	ERR, UI, TEST	Error pages, logging, responsive and accessible UI, automated and manual tests
Delivery	GIT, CI, DEP, HOST, DOC	Repository, workflows, Railway container deployment, Firebase Hosting site, documentation

Non-functional requirements: pages respond within about one second once the container is warm; the interface meets WCAG 2.2 A/AA contrast and keyboard access; no secret is stored in the repository — the administrator password is supplied through `Seed__AdminPassword` in the environment and every other credential is generated per CI run; the application can be rebuilt from an empty database with migrations and seed data.

8. Proposed technology

Choice	Reason
ASP.NET Core MVC on .NET 10 (LTS)	Required .NET technology; long-term support until 2028; built-in model binding, validation, antiforgery and Razor views
C#	Strong typing and the language of the module
Razor views with HTML5 semantic markup	Server-rendered pages that work without JavaScript and stay accessible to assistive technology
Entity Framework Core 10 with SQLite	LINQ queries are parameterised by default, code-first migrations keep the schema in version control, and the whole database is one file that needs no server
SQLite on a Railway volume	One provider in development, testing and production, so no behaviour can differ between environments; the database file and the Data Protection key ring live on a mounted volume at <code>/data</code> , so a redeploy keeps every account and upload; nothing external has to be provisioned, explained or demonstrated
ASP.NET Core Identity	Proven password hashing, logout, Admin and Student roles and cookie authentication instead of custom security code
Bootstrap 5.3 with a custom design system	Accessible, responsive components, restyled so the site has its own identity
Vanilla JavaScript and jQuery Validation Unobtrusive	Progressive enhancement; client validation generated from the same server rules
xUnit and Playwright	Fast automated tests of the real application, plus browser tests on several screen sizes
GitHub and GitHub Actions	Version control, pull requests and the three CI jobs that build, run the container and exercise the site in a browser
Railway (Docker image)	Runs the application itself plus its <code>/data</code> volume and its TLS edge proxy; a Dockerfile and <code>railway.json</code> describe the whole deployment
Firebase Hosting	Serves the static presentation site from <code>FirebaseLanding/</code> over HTTPS with its own caching rules, for free

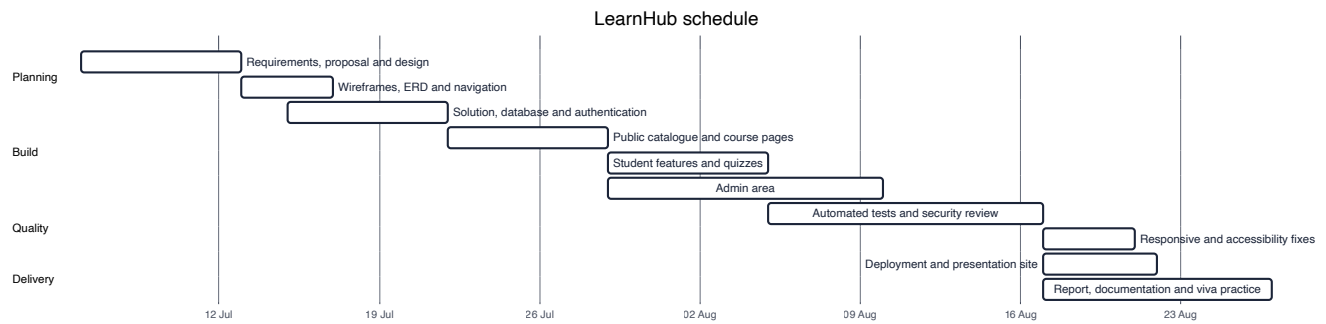
9. High-level architecture



10. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Little disk space for the .NET SDK	High	Medium	Build, test and generate migrations in GitHub Actions; only the repository is needed locally
Passwords or connection strings leak into Git	Medium	High	User secrets locally; environment variables on Railway; <code>.env</code> is git-ignored and only <code>.env.example</code> with placeholders is committed; CI generates throw-away passwords and masks them
The SQLite file is lost on redeploy because the volume is missing	Medium	High	The database, the Data Protection key ring and uploads all live under <code>/data</code> ; CI mounts a volume in the same place and restarts the container to prove migrations are not re-applied
A single SQLite instance cannot serve many concurrent writers	Low	Medium	The deployment is deliberately single-instance and read-heavy, which suits one file; the service layer is provider-agnostic should a server database ever be needed
Scope creep (payments, chat, certificates)	Medium	Medium	Fixed out-of-scope list; new ideas recorded as future enhancements
Security weaknesses in custom code	Medium	High	Use framework features (Identity, antiforgery, Razor encoding); security review; attack-style tests
Uneven contribution or merge conflicts	Medium	Medium	Clear ownership per area, short-lived branches, pull requests with CI checks
Cloud free-tier limits or cold starts during the demonstration	Medium	Low	The health check runs before the service accepts traffic; warm the site before presenting; screenshots and a local container run as backup

11. Schedule



12. Deliverables

1. Source code in the GitHub repository with a README.
2. The application packaged as a Docker image for Railway (`Dockerfile` , `railway.json` , `.env.example`) and the presentation site live on Firebase Hosting at <https://learnhub-wapp.web.app>.
3. Automated tests and the three CI jobs in `.github/workflows/ci.yml`.
4. Documentation: this proposal, the final report content, ERD, use cases, flowcharts, wireframes, navigation structure, testing and security documents, deployment guide, team responsibilities, Git workflow and viva preparation.

Requirement Traceability

Each requirement from [REQUIREMENTS_CHECKLIST.md](#) with its implementation, evidence and status. Paths without a prefix are relative to `src/LearnHub/`; documentation paths are relative to `docs/`. Every row below was re-verified by reading the file named in the *Implementation* column and by running the project, not copied from an earlier revision.

Measured evidence on the `feature/railway-firebase-sqlite` branch: **211 automated tests pass with 0 failures** (`dotnet test LearnHub.sln -c Release`), **42 browser checks pass** (14 Playwright tests × 3 viewports) and the seeded catalogue contains **8 categories, 12 courses (11 published, 1 draft), 64 learning resources, 11 quizzes, 54 questions and 216 answer options**. The build is warning-free because CI passes `-p:TreatWarningsAsErrors=true`. The presentation site is live on Firebase Hosting at <https://learnhub-wapp.web.app>.

Summary

	Total	Pass	Ready, not yet performed
Mandatory	110	108	2
Optional	6	6	0

Ready, not yet performed: DEP-01 and DEP-02. Everything they depend on is built and verified — `Dockerfile`, `railway.json`, `.env.example` and `scripts/set-app-url.sh` — and the `container` CI job builds the image and runs it exactly as Railway does (Production, `PORT=8080`, TLS terminated in front, a volume at `/data`), including a restart that proves the SQLite file survives. The Railway CLI is installed but **not authenticated**, so no Railway deployment has been made and no Railway URL is claimed anywhere in this repository. The steps that finish both are in [DEPLOYMENT.md](#).

Coverage of the assignment's requirement areas

Each area the brief asks for, the requirement IDs that carry it, and the one piece of evidence that settles it. The full rows follow.

Requirement area	IDs	How it is met	Verification
Web-based learning functionality	GUEST-01–10, STU-01–13, ADM-01–11	Browsers work without a plug-in: course catalogue, lessons, progress and quizzes are ordinary HTML pages served by MVC	<code>tests/LearnHub.Tests/Integration/StudentJourneyTests.A_new_student_</code>
Interlinked pages (working hyperlinks between pages)	UI-02, UI-04, TECH-11	Shared navigation, course cards, breadcrumbs, pagination and previous/next lesson links; routes declared in <code>Infrastructure/ApplicationBuilderExtensions.cs</code>	<code>NavigationCrawlTests</code> follows every internal link from each role's start page and fa
HTML5	TECH-06	Razor views use HTML5 doctype, <code><header></code> , <code><nav></code> , <code><main></code> , <code><footer></code> , <code><section></code> , <code><article></code> , <code><table></code> , <code><form></code> and HTML5 input types	<code>Views/Shared/_Layout.cshtml</code> ; <code>axe-core</code> scans in <code>tests/e2e/specs/acces</code>
CSS	TECH-07	<code>wwwroot/css/site.css</code> and <code>admin.css</code> (about 3,500 combined lines) restyle Bootstrap 5.3.8 through CSS custom properties and Grid/Flexbox	Both stylesheets are served and screenshotted by the browser tests
UI/UX	UI-01–04	One Interchange design system: consistent navigation, cards, dashboards, tables, forms, empty states and status messages	<code>tests/e2e/specs/</code> (public, student, admin) plus the screenshots published on the Fi
Database connectivity	TECH-03, TECH-04, DB-01–10	EF Core 10 with SQLite, one context, one migration, applied at start-up	<code>tests/LearnHub.Tests/Data/DatabaseModelTests.cs</code> ; the <code>container</code> CI job
CRUD	ADM-03–11, DB-09, STU-06	Create, read, update and delete through real forms for courses, categories, resources, quizzes, questions, enrolments, users and messages	<code>tests/LearnHub.Tests/Integration/AdminManagementTests.cs</code> (10 tests)
Registration	GUEST-08, AUTH-01, VAL-04	<code>AccountController.Register</code> with strong-password rules and a hashed password	<code>AuthenticationTests.Registration_creates_a_student_signs_them_in_and</code>

Requirement area	IDs	How it is met	Verification
Login / authentication	GUEST-09, AUTH-01-06, SEC-03	ASP.NET Core Identity cookie authentication, role claims, lockout and anti-forgery	<code>tests/LearnHub.Tests/Integration/AuthenticationTests.cs</code> (9 tests)
Registered-member functionality	STU-01-13	Dashboard, My Courses, lessons, progress, quizzes, results, profile, password change, logout	<code>StudentJourneyTests</code> (5 tests); <code>tests/e2e/specs/student.spec.js</code>
Administrator functionality	ADM-01-11	A protected Admin area with its own layout, dashboard and management pages	<code>tests/e2e/specs/admin.spec.js</code> ; <code>AuthorizationTests</code>
Database management (schema, constraints, migrations, seeds)	DB-01-10, TECH-04	Fluent API configuration, one migration creating 18 tables, idempotent seeding, <code>/health</code> database check	<code>DatabaseModelTests</code> ; <code>.github/workflows/ef-migrations.yml</code> ; <code>Infrastructure</code>
Form validation on the client	VAL-02, VAL-03	jQuery Validation Unobtrusive with custom adapters for the file and consent rules	<code>public.spec.js</code> contact and registration validation tests
Form validation on the server	VAL-01, VAL-04-06	DataAnnotations plus <code>ModelState</code> and service rules, so a tampered form is still rejected	<code>AuthenticationTests.Invalid_registration_is_rejected_by_server_side</code>
Client-side processing	TECH-08	Vanilla JavaScript and jQuery for the password meter, the quiz timer and counter, image previews and confirmation prompts	<code>public.spec.js</code> "registration form gives live password feedback"; <code>site.js</code>
Server-side processing	TECH-10, TECH-03	All routing, business rules, grading and persistence happen in C# controllers and services; no rule lives in the browser	<code>tests/LearnHub.Tests/Services/</code> (22 tests); <code>QuizAndCourseManagementTests.The_quiz_page_never_contains_the_correct</code>
Multimedia learning resources	TECH-09, GUEST-07, STU-08, ADM-05	Article, video (privacy-enhanced YouTube/Vimeo), PDF, image, link and exercise resource types, plus 12 SVG course covers and an interactive SVG map	<code>AdminManagementTests.Resources_are_created_validated_streamed_and_downloaded</code> ; <code>PublicSiteTests.Home_page_map_draws_one_line_per_category</code>
Functional navigation (menus, role-based links, protected routes, friendly 404)	UI-02, UI-04, AUTH-04, ERR-01	Role-aware navbar and admin sidebar, breadcrumbs, canonical redirects and a branded error page	<code>NavigationCrawlTests</code> (three roles); <code>PublicSiteTests.Unknown_pages_and_invalid_ids_return_a_friendly_404</code>
File organisation and naming	TECH-11, GIT-01	<code>src/</code> , <code>tests/</code> , <code>docs/</code> , <code>scripts/</code> , <code>FirebaseLanding/</code> ; consistent PascalCase types, one type per file, layered folders	Repository tree; ARCHITECTURE.md §2
.NET technologies	TECH-01-05, TECH-10	ASP.NET Core MVC, C#, Razor, EF Core 10, Identity — all on <code>net10.0</code>	<code>Directory.Build.props</code> ; the <code>build-and-test</code> CI job
Security	SEC-01-11, AUTH-02-06	Identity hashing, roles, anti-forgery, Razor encoding, CSP and security headers, IDOR and overposting guards, safe uploads, rate limiting	<code>tests/LearnHub.Tests/Unit/ContentSafetyTests.cs</code> ; <code>FileStorageService</code>
Error handling	ERR-01-05	Friendly 404 and 500 pages, business-error messages, unique-constraint handling, structured logs	<code>PublicSiteTests.Unknown_pages_and_invalid_ids_return_a_friendly_404</code>
Responsiveness	UI-05	Bootstrap 5.3 grid with project breakpoints, collapsing navbar, off-canvas admin menu, scrollable tables	42 browser checks at 1440x900, 768x1024 and Pixel 7 with no horizontal overflow
Accessibility	UI-06, VAL-03	Landmarks, skip link, labels, alt text, focus styles, reduced-motion support, AA contrast	axe-core WCAG 2.2 A/A scans of over 30 page views at three viewports report no violations outstanding (TESTING.md §7)
Entity Relationship Diagram	DB-01, DB-02, DB-04, DB-05, DB-10	Mermaid ERD with every entity, relationship, delete rule, key and index	ERD.md , checked against the migration

Requirement area	IDs	How it is met	Verification
Deployment	DEP-01-03, CI-01, CI-02, PAGES-01, PAGES-02	Docker image + <code>railway.json</code> + <code>/data</code> volume for the application; <code>FirebaseLanding/</code> on Firebase Hosting; three CI jobs	The <code>container</code> and <code>e2e</code> CI jobs; https://learnhub-wapp.web.app . The Railway depl

Technology and platform

ID	Requirement	Type	Implementation	Evidence
TECH-01	Built with .NET technologies – ASP.NET Core MVC on the latest stable LTS (.NET 10)	M	<code>LearnHub.csproj</code> with <code><TargetFramework>net10.0</TargetFramework></code> and <code>Program.cs</code> ; package versions pinned in <code>Directory.Build.props</code> / <code>Directory.Packages.props</code> (ASP.NET Core 10.0.12, EF Core 10.0.12)	CI job <code>build-and-test</code> ; <code>dotnet test</code> locally
TECH-02	C# for all server-side code	M	Every file in <code>Controllers/</code> , <code>Areas/Admin/Controllers/</code> , <code>Services/</code> , <code>Models/</code> , <code>ViewModels/</code> , <code>Infrastructure/</code> and <code>Data/</code> is C#; <code>LangVersion</code> is <code>latest</code>	Repository; build
TECH-03	Database connectivity through Entity Framework Core	M	<code>Data/ApplicationDbContext.cs</code> plus 23 classes under <code>Services/</code> that query through EF Core LINQ; <code>Services/CourseProjections.cs</code> keeps the catalogue queries projected	<code>tests/LearnHub.Tests/Data/DatabaseModelTests.cs</code> ; the integration tests r a real SQLite database
TECH-04	One database provider (SQLite) with code-first EF Core migrations	M	<code>Data/DatabaseServiceCollectionExtensions.cs</code> calls <code>UseSqlite</code> unconditionally and resolves the connection string from <code>DATABASE_CONNECTION_STRING</code> , then <code>ConnectionStrings:DefaultConnection</code> ; one migration folder, <code>Data/Migrations/</code>	<code>Data/DatabaseServiceCollectionExtensions.cs</code> ; CI job <code>container</code> applic migration on an empty volume
TECH-05	ASP.NET Core Identity for authentication	M	<code>Infrastructure/ServiceCollectionExtensions.cs</code> <code>AddLearnHubIdentity</code> with <code>ApplicationUser</code> , <code>IdentityRole</code> , hashing, lockout and cookie configuration	<code>tests/LearnHub.Tests/Integration/AuthenticationTests.cs</code> (9 tests)
TECH-06	Razor views, HTML5 semantic markup	M	72 <code>.cshtml</code> files under <code>Views/</code> and <code>Areas/Admin/Views/</code> ; <code>Views/Shared/_Layout.cshtml</code> uses <code><header></code> , <code><nav></code> , <code><main></code> , <code><footer></code> and a skip link	axe-core scans in <code>tests/e2e/specs/accessibility.spec.js</code> ; <code>Views/Shared/_Layout.cshtml</code>
TECH-07	CSS3 with Bootstrap 5.3 customised by a project design system	M	Bootstrap 5.3.8 in <code>wwwroot/lib/</code> restyled by <code>wwwroot/css/site.css</code> (about 2,800 lines) and <code>wwwroot/css/admin.css</code>	Both stylesheets, served through <code>UseLearnHubStaticFiles</code>
TECH-08	JavaScript for client-side processing where appropriate	M	Progressive enhancement in <code>wwwroot/js/site.js</code> (password-strength meter, quiz timer and counter, confirmations) and <code>wwwroot/js/validation.js</code> (custom adapters)	<code>tests/e2e/specs/public.spec.js</code> registration and contact tests; <code>tests/e2e/specs/student.spec.js</code> quiz test
TECH-09	Multimedia (images, embedded video, PDF documents, SVG illustrations)	M	12 SVG course covers in <code>wwwroot/images/courses/</code> ; embedded YouTube/Vimeo lessons rendered by <code>Services/Content/VideoEmbedParser.cs</code> and <code>Views/Resources/Details.cshtml</code> ; PDF cheat sheets; a data-driven SVG interchange map (<code>Infrastructure/InterchangeMap.cs</code> , <code>Views/Home/Index.cshtml</code>)	<code>AdminManagementTests.Resources_are_created_validated_streamed_and_ContentSafetyTests</code> ; <code>PublicSiteTests.Home_page_map_draws_one_line_per_category</code>

ID	Requirement	Type	Implementation	Evidence
TECH-10	Server-side processing (controllers, services, EF Core queries)	M	9 controllers in <code>Controllers/</code> and 10 in <code>Areas/Admin/Controllers/</code> (plus <code>AdminControllerBase</code>); 23 service classes; no business rule lives in JavaScript	<code>tests/LearnHub.Tests/Services/</code> (22 tests); <code>tests/LearnHub.Tests/Inte</code>
TECH-11	Proper project and file organisation	M	Top level: <code>src/</code> , <code>tests/</code> (<code>LearnHub.Tests</code> , <code>e2e</code>), <code>docs/</code> , <code>scripts/</code> , <code>FirebaseLanding/</code> , <code>.github/workflows/</code> ; inside the project a layered split of <code>Controllers/</code> , <code>Areas/</code> , <code>Services/</code> , <code>Models/</code> , <code>ViewModels/</code> , <code>Infrastructure/</code> , <code>Data/</code> , <code>Views/</code>	Repository tree; ARCHITECTURE.md §2

Guest

ID	Requirement	Type	Implementation	Evidence
GUEST-01	View Home page	M	<code>Controllers/HomeController.cs</code> <code>Index</code> , <code>Views/Home/Index.cshtml</code> (hero, interchange map, featured courses)	<code>PublicSiteTests.Public_pages_load_for_guests</code> ; <code>tests/e2e/specs/public</code> presents the catalogue"
GUEST-02	View About page	M	<code>HomeController.About</code> , <code>Views/Home/About.cshtml</code> ; routed at <code>/About</code> by <code>MapLearnHubRoutes</code>	<code>PublicSiteTests.Public_pages_load_for_guests</code> ; <code>public.spec.js</code> "about p
GUEST-03	Browse published courses	M	<code>Controllers/CoursesController.cs</code> <code>Index</code> , <code>Services/CourseCatalogService.cs</code> (published only)	<code>PublicSiteTests.Seeded_catalogue_is_shown_with_published_courses_only</code> ; <code>Draft_courses_are_not_visible_to_guests</code>
GUEST-04	Search courses (server-side)	M	Server-side <code>EF.Functions.Like</code> over title, description and lesson titles, with wildcards escaped by <code>Services/SearchPattern.cs</code>	<code>PublicSiteTests.Search_is_performed_on_the_server</code> , <code>Search_matches_lesson_titles_and_shows_the_matching_lessons</code> , <code>Search_matches_full_course_descriptions</code> ; <code>ContentSafetyTests.Wildcards_typed_by_users_are_escaped</code>
GUEST-05	Filter courses by category and difficulty	M	<code>CourseSearchQuery</code> filters by category and difficulty with sorting and paging (<code>ViewModels/Public/PublicViewModels.cs</code> , <code>ViewModels/Shared/PagedResult.cs</code>)	<code>PublicSiteTests.Category_filter_limits_results_to_that_category</code> , <code>Courses_beyond_the_first_page_are_reachable_by_paging</code> ; <code>CatalogueServiceTests.Search_returns_only_published_courses_matching</code>
GUEST-06	Open course details	M	<code>CoursesController.Details</code> with the <code>ViewModels/Learning/LearningViewModels.cs</code> course route	<code>PublicSiteTests.Course_details_show_route_and_enrolment_call_to_acti</code> "course details and a free preview lesson"
GUEST-07	View selected public (preview) learning resources	M	<code>Services/LearningResourceService.cs</code> releases resources with <code>IsPreview</code> ; <code>Views/Resources/Details.cshtml</code>	<code>PublicSiteTests.Preview_lessons_are_open_to_guests_but_other_lessons</code>
GUEST-08	Register	M	<code>Controllers/AccountController.cs</code> <code>Register</code> (GET/POST) with <code>ViewModels/Account/AccountViewModels.cs</code>	<code>AuthenticationTests.Registration_creates_a_student_signs_them_in_and</code> , <code>Invalid_registration_is_rejected_by_server_side_validation</code> , <code>Registering_with_an_existing_email_shows_a_single_clear_error</code>
GUEST-09	Login	M	<code>AccountController.Login</code> (GET/POST) with a local-return-URL guard	<code>AuthenticationTests.Students_and_admins_are_sent_to_their_own_dashbo</code> , <code>Wrong_password_shows_a_generic_error</code> , <code>Login_redirects_back_to_a_local_return_url_but_never_to_another_site</code>
GUEST-10	View Contact page and send a contact message	M	<code>HomeController.Contact</code> (GET/POST), <code>Services/ContactService.cs</code> stores the message and discards honeypot submissions; the POST is rate limited	<code>AdminManagementTests.Contact_messages_reach_the_admin_inbox</code> , <code>Spam_honeypot_submissions_are_not_stored</code> ; <code>public.spec.js</code> "contact form before submitting"

Student

ID	Requirement	Type	Implementation	Evidence
STU-01	Student dashboard with meaningful, data-driven information	M	<code>Controllers/StudentController.cs</code> <code>Dashboard</code> , <code>Services/DashboardService.cs</code> , <code>Views/Student/Dashboard.cshtml</code>	<code>tests/e2e/specs/student.spec.js</code> "dashboard, course progress, lesson and quiz"

ID	Requirement	Type	Implementation	Evidence
STU-02	Dashboard: enrolled courses, completed courses, available courses	M	Dashboard figures for enrolled, completed and available courses (<code>ViewModels/Learning/LearningViewModels.cs</code>)	<code>Student/Dashboard.cshtml</code> ; <code>CatalogueServiceTests.Progress_counts_completed_lessons_and_passed_quiz</code>
STU-03	Dashboard: recent activity, quiz results, learning progress	M	Recent activity feed, recent quiz results, progress bars and recommendations from <code>Services/ProgressService.cs</code>	<code>Student/Dashboard.cshtml</code> and <code>Views/Student/Progress.cshtml</code> ; <code>CatalogueServiceTests.Progress_counts_completed_lessons_and_passed_quiz</code> <code>Stored_progress_is_recalculated_and_completion_is_cleared_when_new_course</code>
STU-04	View and update profile	M	<code>Controllers/ProfileController.cs</code> <code>Index</code> (GET/POST), including a guarded e-mail change	<code>StudentJourneyTests.Profile_updates_are_validated_and_saved</code> , <code>Changing_the_email_address_needs_the_current_password_and_an_unused_ac</code>
STU-05	Browse courses and view course details	M	The catalogue and details pages accept anonymous and authenticated visitors	<code>NavigationCrawlTests.Student_navigation_has_no_broken_links</code> ; <code>student.</code>
STU-06	Enrol in a course (and leave a course)	M	<code>CoursesController.Enroll / Leave</code> (POST, <code>[Authorize(Roles = "Student")]</code>) , <code>Services/EnrollmentService.cs</code>	<code>CatalogueServiceTests.Enrolling_twice_is_rejected_and_unpublished_course</code> <code>Leaving_a_course_keeps_completed_lessons_for_later</code> ; <code>StudentJourneyTests.Leaving_a_course_removes_access_to_its_lessons</code>
STU-07	View My Courses	M	<code>StudentController.MyCourses</code> with an <code>MyCoursesFilter</code> (all, in progress, completed)	<code>Student/MyCourses.cshtml</code> ; <code>student.spec.js</code>
STU-08	Access learning resources of enrolled courses	M	<code>ResourcesController.Details</code> and <code>Open</code> (stream/download) enforce enrolment in <code>Services/LearningResourceService.cs</code> ; files are streamed from outside <code>wwwroot</code>	<code>StudentJourneyTests.A_new_student_enrols_studies_takes_the_quiz_and_se</code> <code>PublicSiteTests.Private_files_cannot_be_downloaded_by_guests</code>
STU-09	Track learning progress (mark resources complete)	M	<code>ResourcesController.ToggleComplete</code> , <code>Services/ProgressService.cs</code> recalculates the enrolment percentage	<code>CatalogueServiceTests.Progress_counts_completed_lessons_and_passed_quiz</code> <code>Draft_lessons_are_hidden_from_students_and_do_not_count_towards_progre</code>
STU-10	Take quizzes	M	<code>Controllers/QuizzesController.cs</code> <code>Index / Take / Submit</code> , <code>Services/QuizService.cs</code> and <code>Services/QuizGrader.cs</code> ; the page never contains the correct answers	<code>QuizAndCourseManagementTests.Submitting_answers_stores_a_graded_attempt</code> <code>The_quiz_page_never_contains_the_correct_answers</code> , <code>Students_who_are_not_enrolled_cannot_take_or_submit_the_quiz</code> ; <code>LogicTe</code>
STU-11	View quiz results and history	M	<code>QuizzesController.Result</code> and <code>History</code> , <code>Views/Quizzes/Result.cshtml</code> with <code>_QuizReview.cshtml</code> explanations	<code>QuizAndCourseManagementTests.Students_cannot_open_other_students_resul</code> <code>student.spec.js</code>
STU-12	Logout	M	POST-only <code>AccountController.Logout</code> with <code>[Authorize]</code> , so a link cannot sign a user out	<code>AuthenticationTests.Logout_ends_the_session</code>
STU-13	Change password	O	<code>ProfileController.ChangePassword</code> (GET/POST), rate limited, refreshes the sign-in cookie after a change	<code>ProfileController.cs</code> ; axe-core scan of <code>/Profile/ChangePassword</code> in <code>accessi</code>

Administrator

ID	Requirement	Type	Implementation	Evidence
ADM-01	Separate, protected Admin area	M	<code>Areas/Admin/</code> with its own layout; <code>Areas/Admin/Controllers/AdminControllerBase.cs</code> carries <code>[Authorize(Roles = AppRoles.Admin)]</code> , so every derived controller inherits it	<code>AuthorizationTests.Guests_are_sent_to_login_for_admin_pages</code> <code>Students_are_denied_access_to_admin_pages</code> , <code>Students_cannot_post_to_admin_actions_even_with_a_valid_token</code>
ADM-02	Admin dashboard with real platform statistics	M	<code>Areas/Admin/Controllers/DashboardController.cs</code> and <code>Services/DashboardService.cs</code> ; <code>Areas/Admin/Views/Dashboard/Index.cshtml</code>	<code>tests/e2e/specs/admin.spec.js</code> "dashboard and management pages"
ADM-03	Courses CRUD (create, list, details, edit, delete)	M	Admin <code>CoursesController</code> (index, details, create, edit, <code>TogglePublished</code> , delete/confirm) with <code>Services/CourseManagementService.cs</code>	<code>AdminManagementTests.Course_create_with_cover_edit_publish_a</code> <code>Category_with_courses_is_protected_from_deletion</code>

ID	Requirement	Type	Implementation	Evidence
ADM-04	Categories CRUD	M	Admin <code>CategoriesController</code> , <code>Services/CategoryService.cs</code> (unique name, protected delete)	<code>AdminManagementTests.Category_create_edit_and_delete</code> , <code>Category_validation_errors_are_shown</code> ; <code>CatalogueServiceTests.Creates_categories_and_rejects_duplicates</code> ; <code>A_category_that_still_has_courses_cannot_be_deleted</code> ; admin delete through the forms"
ADM-05	Learning resources CRUD (incl. secure file upload)	M	Admin <code>ResourcesController</code> , <code>Services/ResourceManagementService.cs</code> , <code>Services/Storage/FileStorageService.cs</code> and <code>Infrastructure/FileValidationAttributes.cs</code>	<code>AdminManagementTests.Resources_are_created_validated_streamed</code> , <code>FileStorageServiceTests</code> (7 tests)
ADM-06	Enrolments management (list, enrol a student, remove)	M	Admin <code>EnrollmentsController</code> (index, create, delete/confirm) with <code>Services/EnrollmentService.cs</code>	<code>AdminManagementTests.Enrolments_can_be_managed_by_administrator</code>
ADM-07	Quizzes CRUD	M	Admin <code>QuizzesController</code> (index, details, create, edit, <code>SetPublished</code> , delete) with <code>Services/QuizManagementService.cs</code>	<code>AdminManagementTests.Quiz_must_have_questions_before_it_can_be_created</code>
ADM-08	Quiz questions and answer options CRUD, set correct answer	M	Admin <code>QuestionsController</code> with <code>ViewModels/Admin/AdminLearningViewModels.cs</code> ; <code>Services/QuestionManagementService.cs</code> enforces 2–6 options with exactly one correct answer	<code>LogicTests.Question_needs_two_answers_one_marked_correct_and_one_wrong</code> ; <code>QuizAndCourseManagementTests.Option_ids_that_belong_to_another_quiz_cannot_be_reused</code>
ADM-09	Quiz results (attempts) view and delete	M	Admin <code>QuizAttemptsController</code> (index with filters, details, delete/confirm)	Link crawl as administrator (<code>NavigationCrawlTests.Admin_navigation_scan_of /Admin/QuizAttempts</code> — there is no dedicated automated test for this)
ADM-10	Users: list, view details, change role, deactivate/reactivate, safe delete	M	Admin <code>UsersController</code> (index, details, <code>SetRole</code> , <code>Deactivate</code> , <code>Reactivate</code> , delete/confirm) with <code>Services/UserManagementService.cs</code> , which protects the last administrator and self-deletion	<code>AdminManagementTests.User_roles_deactivation_and_self_protection</code>
ADM-11	Website content: contact message inbox	O	Admin <code>MessagesController</code> (index, details, <code>MarkUnread</code> , delete) with <code>Services/ContactService.cs</code>	<code>AdminManagementTests.Contact_messages_reach_the_admin_inbox</code>

Database

ID	Requirement	Type	Implementation	Evidence
DB-01	Relational model with primary keys and foreign keys	M	Fluent API configuration in <code>Data/Configurations/UserAndCatalogConfigurations.cs</code> and <code>Data/Configurations/QuizConfigurations.cs</code> over the 11 application entities in <code>Models/</code>	<code>DatabaseModelTests.Create_script_enforces_the_integrity_rules</code>
DB-02	Required properties and maximum lengths	M	<code>IsRequired()</code> and <code>HasMaxLength()</code> taken from <code>Models/FieldLengths.cs</code> , which the view models reuse so form rules cannot drift from the schema	<code>DatabaseModelTests.Create_script_enforces_the_integrity_rules</code>
DB-03	Unique constraints where appropriate	M	Unique indexes on category name, enrolment (user + course), resource completion (user + resource) and quiz answer (attempt + question)	<code>DatabaseModelTests.Create_script_enforces_the_integrity_rules</code> ; <code>CatalogueServiceTests.Creates_categories_and_rejects_duplicates</code>
DB-04	Indexes where useful	M	Indexes on search, filter, ordering and foreign-key columns, for example <code>Courses(IsPublished, CategoryId)</code> , <code>Courses(Title)</code> , <code>LearningResources(CourseId, SortOrder)</code> , <code>Enrollments(CourseId, CompletionPercentage)</code> , <code>QuizAttempts(CompletedAt)</code>	Index table in ERD.md ; 27 <code>CreateIndex</code> calls in the migration
DB-05	Intentional cascade / restrict delete behaviour	M	Restrict for <code>Course → Category</code> and both <code>QuizAnswer</code> links; cascade elsewhere; <code>Data/TransactionExtensions.cs</code> runs the ordered deletes in one transaction	<code>DatabaseModelTests.Delete_behaviour_is_intentional</code> ; <code>QuizAndCourseManagementTests.Deleting_a_course_removes_its_content</code>
DB-06	CreatedAt / UpdatedAt timestamps where useful	M	<code>Models/IHasTimestamps.cs</code> is applied in <code>ApplicationDbContext.SaveChanges</code> ; <code>Data/UtcDateTimeConverter.cs</code> stores UTC	<code>ApplicationDbContext.cs</code> ; ERD.md

ID	Requirement	Type	Implementation	Evidence
DB-07	EF Core migrations reproducible from an empty database	M	Single migration <code>Data/Migrations/20260914100832_InitialCreate.cs</code> creates 18 tables (11 application + 7 Identity); applied at start-up when <code>Database:ApplyMigrationsOnStartup</code> is true; <code>Data/DesignTimeDbContextFactory.cs</code> is the one design-time factory	The <code>container</code> CI job creates the schema on an empty volume, then restarts <code>tests/LearnHub.Tests/Infrastructure/TestDatabase.cs</code> migrates a f
DB-08	Realistic seed data (categories, courses, resources, quizzes, demo activity)	M	<code>Data/Seed/DatabaseInitializer.cs</code> → <code>DemoDataSeeder</code> → <code>DemoCatalog</code> (four partial files); seeded content measured: 8 categories, 12 courses (11 published, 1 draft), 64 resources, 11 quizzes, 54 questions, 216 answer options, 7 learners	The <code>container</code> CI job asserts the log line "Demo data seeded"; counts verified
DB-09	Insert, Display, Update, Delete operations demonstrated	M	Insert, display, update and delete are exercised through the student and admin screens rather than raw SQL	<code>AdminManagementTests</code> (10 tests) and <code>StudentJourneyTests</code> (5 tests) dr
DB-10	ERD that exactly matches the EF Core entities	M	ERD.md generated from <code>Models/</code> and <code>Data/Configurations/</code>	Entity, key, index and delete-behaviour tables checked against <code>Data/Migratio</code>

Authentication and authorisation

ID	Requirement	Type	Implementation	Evidence
AUTH-01	Register, Login, Logout	M	<code>Controllers/AccountController.cs</code> plus <code>Views/Account/Register.cshtml</code> and <code>Login.cshtml</code>	<code>AuthenticationTests</code> (9 tests); <code>public.spec.js</code> registration test
AUTH-02	Password hashing through Identity (no plain-text passwords)	M	<code>AddIdentity</code> with the default <code>PasswordHasher</code> ; no password field is ever stored or logged	<code>AuthenticationTests.Registration_creates_a_student_signs_them_</code>
AUTH-03	Roles: Guest, Student, Admin with role-based authorisation	M	<code>Models/AppRoles.cs</code> creates <code>Admin</code> and <code>Student</code> ; anonymous visitors are the guest role; <code>Infrastructure/LearnHubClaimsPrincipalFactory.cs</code> adds the role claim	<code>DatabaseInitializer.EnsureRolesAsync</code> ; <code>AuthorizationTests</code> (8 test
AUTH-04	Protected pages for students and admin-only controllers	M	<code>[Authorize(Roles = AppRoles.Student)]</code> on <code>StudentController</code> , <code>QuizzesController</code> and the student-only POST actions; <code>AdminControllerBase</code> covers the Admin area	<code>AuthorizationTests.Students_can_open_their_own_pages</code> , <code>Administrators_are_denied_student_only_pages</code> , <code>Access_denied_pa</code> <code>NavigationCrawlTests</code> crawls each role separately
AUTH-05	Initial admin seeded from secrets / environment variables (no password in Git)	M	<code>Data/Seed/DatabaseInitializer.cs</code> creates the administrator from <code>Seed:AdminEmail / Seed:AdminPassword</code> ; <code>appsettings.json</code> ships both passwords empty; <code>.env.example</code> documents them with placeholders only; in Development <code>DevelopmentSeedPasswords</code> writes generated passwords to the git-ignored <code>App_Data/demo-credentials.json</code>	<code>DevelopmentSeedPasswordsTests</code> (4 tests); the <code>container</code> CI job generat and asserts the log line "Administrator account ... created"
AUTH-06	Account lockout after repeated failed logins	O	<code>AddIdentity</code> lockout: 5 failed attempts, 15-minute lockout, enabled for new users; the failed attempt is logged	<code>AuthenticationTests.Account_is_locked_after_five_failed_attempt</code>

Validation

ID	Requirement	Type	Implementation	Evidence
VAL-01	Server-side validation with <code>DataAnnotations</code> and <code>ModelState</code>	M	<code>DataAnnotations</code> on every view model in <code>ViewModels/</code> ; controllers re-check <code>ModelState.IsValid</code> and services return <code>Services/OperationResult.cs</code>	<code>AuthenticationTests.Invalid_registration_is_rejected_by_server_si</code> <code>AdminManagementTests.Category_validation_errors_are_shown</code>

ID	Requirement	Type	Implementation	Evidence
VAL-02	Client-side validation with ASP.NET validation helpers	M	jQuery Validation Unobtrusive plus custom adapters in <code>wwwroot/js/validation.js</code> ; <code>Views/Shared/_ValidationScriptsPartial.cshtml</code>	<code>public.spec.js</code> "contact form validates in the browser before submitting" and "re gives live password feedback"
VAL-03	Bootstrap-styled, accessible validation feedback	M	Validation messages rendered with Bootstrap's <code>is-invalid</code> / <code>invalid-feedback</code> , <code>aria-describedby</code> hints and <code>aria-live</code> status regions	axe-core scans in <code>accessibility.spec.js</code> ; <code>Views/Shared/_Layout.cshtml</code>
VAL-04	Required, email, password, length and range rules	M	<code>[Required]</code> , <code>[EmailAddress]</code> , <code>[StringLength]</code> (from <code>Models/FieldLengths.cs</code>) , <code>[Range]</code> , <code>[Compare]</code> and the project's <code>MustBeTrueAttribute</code>	<code>LogicTests.Register_requires_matching_strong_passwords_and_accepts_a_valid_registration_has_no_errors</code> ; <code>LogicTests.Resource_rules_depend_on_the_resource_type</code>
VAL-05	File validation (type, size, content signature)	M	<code>Infrastructure/FileValidationAttributes.cs</code> (<code>[MaxFileSize]</code> , <code>[AllowedExtensions]</code>) plus signature and real-length checks in <code>Services/Storage/FileStorageService.cs</code> with limits in <code>Services/Storage/UploadRules.cs</code>	<code>FileStorageServiceTests</code> (7 tests)
VAL-06	Business-rule validation (e.g. exactly one correct answer)	M	Service rules: exactly one correct answer and 2–6 options, a quiz needs questions before publishing, unique category names, an e-mail change needs the current password, and the last administrator cannot be removed	<code>LogicTests.Question_needs_two_answers_one_marked_correct_and_no_duplicates</code> ; <code>AdminManagementTests.Quiz_must_have_questions_before_it_can_be_published</code> ; <code>User_roles_deactivation_and_self_protection</code> ; <code>CatalogueServiceTests</code>

Security

ID	Requirement	Type	Implementation	Evidence
SEC-01	No SQL injection (EF Core parameterised queries only)	M	EF Core LINQ everywhere; the only raw pattern is an escaped <code>LIKE</code> term built by <code>Services/SearchPattern.cs</code> , and no SQL string is concatenated	<code>ContentSafetyTests.Wildcards_typed_by_users_are_escaped</code> , <code>Very_long_search_terms_are_truncated</code> ; SECURITY.md
SEC-02	XSS protection (Razor encoding, no raw user HTML, CSP)	M	Razor encodes by default; the only markup renderer, <code>Services/Content/LessonContentRenderer.cs</code> , emits allow-listed elements; <code>Infrastructure/SecurityHeadersMiddleware.cs</code> sends a strict CSP with <code>script-src 'self'</code>	<code>ContentSafetyTests.Html_in_lesson_text_is_encoded_and_never_executed</code> ; <code>Formatting_syntax_produces_only_allow_listed_elements</code> , <code>Attack_attempts_through_formatting_markers_stay_as_text</code> ; <code>PublicSiteTests.Html_responses_carry_security_headers</code>
SEC-03	CSRF protection (anti-forgery tokens on all unsafe requests)	M	Global <code>AutoValidateAntiforgeryTokenAttribute</code> registered in <code>Infrastructure/ServiceCollectionExtensions.cs</code> ; every POST form emits a token	<code>AuthenticationTests.Posts_without_an_anti_forgery_token_are_rejected</code>
SEC-04	IDOR protection (ownership checks on attempts, resources, files)	M	Ownership and enrolment checks in <code>Services/QuizService.cs</code> , <code>LearningResourceService.cs</code> and <code>ResourceManagementService.cs</code> ; files are streamed through a controller action, never served from <code>wwwroot</code>	<code>QuizAndCourseManagementTests.Students_cannot_open_other_students_resources</code> ; <code>Students_who_are_not_enrolled_cannot_take_or_submit_the_quiz</code> ; <code>PublicSiteTests.Private_files_cannot_be_downloaded_by_guests</code>
SEC-05	Overposting protection (ViewModels, never bind entities)	M	Every action binds a view model from <code>ViewModels/</code> ; no <code>[Bind]</code> /entity binding anywhere	<code>StudentJourneyTests.Tampered_quiz_answers_are_graded_on_the_server</code> ; <code>QuizAndCourseManagementTests.Option_ids_that_belong_to_another_quiz_cannot_be_selected</code>
SEC-06	Safe file uploads (allow-list, magic bytes, size limit, random names, outside <code>wwwroot</code>)	M	Allow-list of extensions and MIME types, content-signature check, size limit, GUID file names, storage under <code>Storage:RootPath</code> (on Railway <code>/data/storage</code>), and path-traversal rejection	<code>FileStorageServiceTests</code> — including <code>Rejects_a_file_whose_content_does_not_match_its_extension</code> , <code>Rejects_files_over_the_size_limit_even_if_the_declared_length_is_within_limit</code> , <code>Paths_outside_the_storage_root_are_never_resolved</code>

ID	Requirement	Type	Implementation	Evidence
SEC-07	Safe video embedding (strict YouTube/Vimeo parsing, frame-src allow-list)	M	<code>Services/Content/VideoEmbedParser.cs</code> accepts only well-formed HTTPS YouTube/Vimeo links and builds a privacy-enhanced embed; CSP <code>frame-src</code> allows only <code>www.youtube-nocookie.com</code> and <code>player.vimeo.com</code>	<code>ContentSafetyTests</code> video cases (19 of the 21 inline data rows in that theory)
SEC-08	No secrets committed to Git	M	Passwords and connection strings come from environment variables (<code>DATABASE_CONNECTION_STRING</code> , <code>Seed__AdminPassword</code> , <code>Seed__DemoStudentPassword</code> , <code>DataProtection__KeysPath</code>); <code>.env</code> is git-ignored and only <code>.env.example</code> with placeholders is committed; CI generates throw-away passwords per run and masks them; the development credential file is git-ignored	<code>.gitignore</code> ; <code>.env.example</code> ; <code>SECURITY.md</code> §3
SEC-09	Admin routes protected by role authorisation	M	<code>AdminControllerBase</code> <code>[Authorize(Roles = AppRoles.Admin)]</code> is inherited by all 10 admin controllers	<code>AuthorizationTests.Students_are_denied_access_to_admin_pages</code> , <code>Students_cannot_post_to_admin_actions_even_with_a_valid_token</code>
SEC-10	Security headers, HTTPS/HSTS in production	O	<code>SecurityHeadersMiddleware</code> adds CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy and COOP; <code>Program.cs</code> adds HSTS and HTTPS redirection outside Development; <code>Infrastructure/ApplicationBuilderExtensions.cs</code> <code>UsePlatformProxyHeaders</code> reads X-Forwarded-Proto so the scheme is correct behind the Railway proxy	<code>PublicSiteTests.Html_responses_carry_security_headers</code> ; <code>scripts/s</code> headers on a running instance
SEC-11	Rate limiting on login, registration and contact form	O	Fixed-window limiter (10 requests per 60 s per client IP, <code>RateLimiting:PermitLimit</code>) applied by <code>[EnableRateLimiting(RateLimitPolicies.Forms)]</code> to registration, login, contact, profile update and password change	<code>Infrastructure/ServiceCollectionExtensions.cs</code> and the four attributed 429

Error handling and logging

ID	Requirement	Type	Implementation	Evidence
ERR-01	Custom 404 page	M	<code>Program.cs</code> <code>UseStatusCodePagesWithReExecute("/Error/{0}")</code> → <code>Controllers/ErrorController.cs</code> <code>Status</code> → <code>Views/Error/Status.cshtml</code> (branded, with a support reference)	<code>PublicSiteTests.Unknown_pages_and_invalid_ids_return_a_friendly_404</code> "unknown pages show the friendly 404 page"; <code>NavigationCrawlTests</code> fails on any
ERR-02	Custom 500 / error page without stack traces in production	M	<code>UseExceptionHandler("/Error")</code> outside Development; the friendly page never renders an exception, only a request id	The <code>container</code> CI job runs Production mode and fails on any <code>fail: / crit: log I</code> <code>ErrorController.ServerError</code>
ERR-03	User-friendly validation and business error messages	M	<code>Services/OperationResult.cs</code> messages surface through <code>TempData</code> (<code>Infrastructure/StatusMessages.cs</code>) and <code>ModelState</code> , plus empty states in the views	<code>AdminManagementTests.Category_validation_errors_are_shown</code> , <code>Quiz_must_have_questions_before_it_can_be_published</code> ; <code>CatalogueServ</code>
ERR-04	Database error handling (e.g. unique-constraint races)	M	<code>DbUpdateException</code> from a unique index is caught and turned into a friendly message in <code>Services/EnrollmentService.cs</code> , <code>CategoryService.cs</code> and <code>ResourceManagementService.cs</code>	<code>CatalogueServiceTests.Enrolling_twice_is_rejected_and_unpublished_c</code> <code>Creates_categories_and_rejects_duplicate_names_regardless_of_case</code>
ERR-05	Structured logging of important events	M	Structured <code>ILogger</code> events for start-up, migrations, seeding, sign-in, logout, enrolment, quiz submission, deletion, upload and honeypot rejection	The <code>container</code> CI job asserts the migration, seeding and administrator log lines and <code>Data/Seed/DatabaseInitializer.cs</code>

UI/UX, accessibility and responsiveness

ID	Requirement	Type	Implementation	Evidence	Status
UI-01	Coherent custom design system (not default Bootstrap)	M	The Interchange design system in <code>wwwroot/css/site.css</code> (colour, type, spacing and component tokens) plus <code>wwwroot/css/admin.css</code> ; the same tokens are reused by <code>FirebaseLanding/assets/site.css</code>	Both stylesheets; https://learnhub-wapp.web.app	PA
UI-02	Navigation bar per role, hero, course cards, dashboards, tables, forms, footer	M	<code>Views/Shared/_Layout.cshtml</code> renders a different navigation per role; <code>Views/Shared/_CourseCard.cshtml</code> , <code>_ProgressRoute.cshtml</code> , <code>_Pagination.cshtml</code> , <code>_StatusMessage.cshtml</code> ; <code>Areas/Admin/Views/Shared/_AdminLayout.cshtml</code> has an off-canvas sidebar	All public, student and admin screenshots; <code>admin.spec.js</code> , <code>student.spec.js</code>	PA
UI-03	Empty states and success/error messages	M	Empty states in the catalogue, My Courses, dashboard, progress and admin lists; flash messages through <code>Infrastructure/StatusMessages.cs</code> and <code>Views/Shared/_StatusMessage.cshtml</code> with <code>aria-live</code>	The views above; <code>AdminManagementTests</code> assert the resulting messages are shown	PA
UI-04	Breadcrumbs where useful	M	Breadcrumb trails on nested pages — course details, lesson, quiz take/result, profile and the admin detail/form pages	20 views contain a breadcrumb; NAVIGATION.md §3	PA
UI-05	Responsive on mobile, tablet and desktop; collapsing navigation; usable tables	M	Bootstrap 5.3 grid plus project breakpoints; collapsing navbar, off-canvas admin menu, and <code>.table-wrap</code> containers that scroll horizontally and stack into label/value rows on phones (<code>table-stack</code>)	42 browser checks (14 tests × desktop 1440×900, tablet 768×1024, mobile Pixel 7) assert no horizontal overflow; <code>public.spec.js</code> "navigation menu works on every screen size"	PA
UI-06	Semantic HTML, labels, alt text, keyboard access, contrast, heading structure	M	Landmarks and a skip link in <code>_Layout.cshtml</code> , explicit labels and <code>aria-describedby</code> hints, decorative images <code>aria-hidden</code> with empty <code>alt</code> , visible <code>:focus-visible</code> styles, and <code>prefers-reduced-motion</code> handling	axe-core WCAG 2.2 A/Aa scans of the public, student and admin pages at all three viewports report no violations; Keyboard and screen-reader passes are still listed as "To do" in TESTING.md §7	PA
UI-07	No placeholder/template text, no broken images or dead links	M	Every seeded course, lesson, quiz and image is real content; no lorem ipsum and no template placeholders remain	<code>NavigationCrawlTests</code> crawls all three roles with no broken links; <code>PublicSiteTests.Static_assets_and_seeded_covers_are_served</code> ; the browser tests fail on console errors and broken images. The four names in the team table of PROPOSAL.md and TEAM.md are intentional placeholders.	PA

Testing and quality

ID	Requirement	Type	Implementation	Evidence	Status
TEST-01	Automated unit tests for services and helpers	M	<code>tests/LearnHub.Tests/Unit/</code> (5 files: logic, content safety, file storage, seed passwords) and <code>tests/LearnHub.Tests/Services/</code> (2 files)	211 tests pass, 0 fail, in <code>dotnet test LearnHub.sln -c Release</code>	PASS
TEST-02	Automated integration tests (auth, protected routes, CRUD, enrolment, quiz, search, validation, 404)	M	<code>tests/LearnHub.Tests/Integration/</code> — <code>PublicSiteTests</code> , <code>AuthenticationTests</code> , <code>AuthorizationTests</code> , <code>AdminManagementTests</code> , <code>StudentJourneyTests</code> , <code>NavigationCrawlTests</code> ; <code>tests/LearnHub.Tests/Infrastructure/LearnHubWebApplicationFactory.cs</code> runs the real pipeline in the <code>Testing</code> environment, and <code>tests/LearnHub.Tests/Infrastructure/TestDatabase.cs</code> gives each test its own migrated SQLite database	Same run: 211 pass, 0 fail	PASS
TEST-03	Browser end-to-end tests across mobile, tablet and desktop viewports	O	<code>tests/e2e/specs/</code> — <code>public</code> , <code>student</code> , <code>admin</code> and <code>accessibility</code> specs; <code>tests/e2e/playwright.config.js</code> defines the desktop, tablet and mobile projects; the <code>e2e</code> CI job serves a Production instance over HTTPS	42/42 browser checks pass (14 tests × 3 viewports)	PASS
TEST-04	Manual testing checklist with recorded results	M	TESTING.md §7 "Manual testing checklist" records 13 results	13 items passed. Four remain "To do" and need a person or a real deployment: keyboard-only journeys, a screen-reader pass, real devices, and the live production check	PASS

ID	Requirement	Type	Implementation	Evidence	Status
TEST-05	Build without errors and without warnings	M	The <code>build-and-test</code> CI job runs <code>dotnet build LearnHub.sln -c Release -p:TreatWarningsAsErrors=true</code> ; <code>Directory.Build.props</code> keeps package-vulnerability advisories (<code>NU1901</code> – <code>NU1904</code>) visible as warnings without breaking unrelated builds	CI build: 0 warnings, 0 errors	PASS

Repository, CI/CD and deployment

ID	Requirement	Type	Implementation	Evidence	Status
GIT-01	Git repository with a professional structure and <code>.gitignore</code>	M	https://github.com/KhayitOff/LearnHub on branch <code>feature/railway-firebase-sqlite</code> ; <code>.gitignore</code> covers build output, <code>App_Data/</code> , <code>.env*</code> (except <code>.env.example</code>) and generated credentials: <code>.gitattributes</code> normalises line endings	Repository	PASS
GIT-02	Sensible commit history and documented branching strategy	M	Conventional Commits history with scoped subjects; the branching strategy is in GIT_WORKFLOW.md	<code>git log</code> ; GIT_WORKFLOW.md	PASS
GIT-03	README with all required sections	M	<code>README.md</code> — overview, technology, structure, setup, tests and a documentation index	<code>README.md</code> . Its badges, hosting table and deployment section still describe the earlier Azure hosting and are not yet updated.	PASS
CI-01	GitHub Actions CI: restore, build, test, fail on error	M	<code>.github/workflows/ci.yml</code> job <code>build-and-test</code> — checkout, .NET 10, cached restore, <code>dotnet build -p:TreatWarningsAsErrors=true</code> , the whole test suite, a result summary and the TRX artifact; runs on every branch push and on pull requests to <code>main</code> / <code>develop</code> , ignoring <code>docs/</code> , <code>FirebaseLanding/</code> and Markdown-only changes	The job file; the latest CI run	PASS
CI-02	GitHub Actions container workflow that reproduces the Railway runtime	M	<code>.github/workflows/ci.yml</code> job <code>container</code> — builds the Docker image, starts it with Production settings, <code>PORT=8080</code> and a volume at <code>/data</code> , waits for <code>/health</code> , runs <code>scripts/smoke-test.sh</code> , asserts the start-up log shows migrations applied, demo data seeded and the administrator created with no <code>fail: / crit:</code> lines, then restarts the container and asserts migrations are not re-applied	The job file; the uploaded <code>container</code> artifact (both logs)	PASS
DEP-01	Production ASP.NET application packaged and ready to run on Railway	M	<code>Dockerfile</code> (multi-stage <code>sdk:10.0</code> → <code>aspnet:10.0</code>), <code>railway.json</code> (DOCKERFILE builder, health check <code>/health</code> , 300 s timeout, restart on failure), <code>Infrastructure/ApplicationBuilderExtensions.cs</code> <code>UsePlatformPort</code> binding <code>http://0.0.0.0:{PORT}</code> and <code>UsePlatformProxyHeaders</code> for the TLS-terminating proxy, and <code>.env.example</code> documenting every variable	The <code>container</code> CI job runs this image the way Railway does; Railway CLI is unauthenticated, so no Railway deployment has been performed	READY – not deployed
DEP-02	SQLite production database on a persistent volume, configured through environment variables	M	<code>DATABASE_CONNECTION_STRING=Data Source=/data/learnhub.db</code> , <code>DataProtection__KeysPath=/data/keys</code> and <code>Storage__RootPath=/data/storage</code> ; <code>Data/DatabaseServiceCollectionExtensions.cs</code> prefers the environment variable over <code>ConnectionStrings:DefaultConnection</code> and creates relative folders; migrations are applied at start-up	The <code>container</code> CI job mounts a volume at <code>/data</code> , seeds it, restarts the container and proves the SQLite file survived; no Railway volume has been created yet	READY – not deployed
DEP-03	Documented migration strategy and production verification	M	Start-up migrations guarded by <code>Database:ApplyMigrationsOnStartup</code> ; <code>.github/workflows/ef-migrations.yml</code> generates a migration in the cloud and commits it; the health endpoint <code>/health</code> uses <code>Infrastructure/DatabaseHealthCheck.cs</code> ; <code>scripts/smoke-test.sh</code> verifies a running instance	DEPLOYMENT.md ; <code>.github/workflows/ef-migrations.yml</code> ; <code>scripts/smoke-test.sh</code>	PASS
PAGES-01	Separate static presentation site with an "Open Learning System" call to action	M	<code>FirebaseLanding/</code> — <code>index.html</code> , <code>404.html</code> and <code>assets/</code> (design system, Overpass webfont and eight real screenshots); its buttons link to the Railway application, the GitHub repository and the documentation	Live at https://learnhub-wapp.web.app (<code>/ 200</code> , <code>/assets/site.css</code> 200, <code>/assets/screens/home-desktop.png</code> 200, unknown path 404). The buttons carry the conventional Railway address until <code>scripts/set-app-url.sh</code> rewrites it after the first deploy.	PASS
PAGES-02	Presentation site deployment automated and reproducible	M	<code>firebase.json</code> at the repository root sets <code>"public": "FirebaseLanding"</code> with <code>cleanUrls</code> and asset caching headers; <code>.firebaserc</code> sets the default project to <code>learnhub-wapp</code> ; deployment is one command, <code>firebase deploy --only hosting</code>	<code>firebase.json</code> , <code>.firebaserc</code> ; the live site. The former GitHub Pages site and its <code>pages.yml</code> workflow were deleted, so no heading or file named <code>pages.yml</code> remains.	PASS

Documentation

ID	Requirement	Type	Implementation	Evidence	Status
DOC-01	Proposal: title, objectives, mission, audience modelling, scope	M	PROPOSAL.md	File	PASS
DOC-02	Final report content (full structure from the brief)	M	REPORT_NOTES.md	File	PASS
DOC-03	ERD (Mermaid)	M	ERD.md	File	PASS
DOC-04	Use cases	M	USE_CASES.md	File	PASS
DOC-05	Flowcharts (registration, login, enrolment, quiz, admin course CRUD)	M	FLOWCHARTS.md — registration, login, enrolment, lesson study, quiz attempt, admin course CRUD and more	File	PASS
DOC-06	Wireframes	M	WIREFRAMES.md	File	PASS
DOC-07	Navigation structure diagram	M	NAVIGATION.md	File	PASS
DOC-08	Testing documentation	M	TESTING.md	File	PASS
DOC-09	Deployment documentation	M	DEPLOYMENT.md	File	PASS
DOC-10	Team responsibilities and Git workflow	M	TEAM.md , GIT_WORKFLOW.md	Files	PASS
DOC-11	Viva preparation questions	M	VIVA.md	File	PASS

Requirements Checklist

Module: **CT050-3-2-WAPP – Web Applications** (Group Assignment)

This checklist is the traceability baseline for the project. Every requirement has a stable ID. The final audit ([REQUIREMENT_TRACEABILITY.md](#)) maps each ID to its implementation, evidence and status.

Legend: **M** = mandatory (assignment brief), **O** = optional improvement.  = implemented and verified against the code; **PENDING** = built and documented, but the hosting step itself has not been performed yet (see §14).

Every requirement below was re-checked against the code on the `feature/railway-firebase-sqlite` branch. The measured evidence behind the ticks is **211 automated tests passing, 42 browser checks passing** (14 Playwright tests x desktop, tablet and mobile) and a seeded catalogue of **8 categories, 12 courses, 64 learning resources, 11 quizzes, 54 questions and 216 answer options**. File-level implementation and evidence for each ID are in [REQUIREMENT_TRACEABILITY.md](#).

1. Technology and platform

ID	Requirement	Type	Status
TECH-01	Built with .NET technologies – ASP.NET Core MVC on the latest stable LTS (.NET 10)	M	
TECH-02	C# for all server-side code	M	
TECH-03	Database connectivity through Entity Framework Core	M	
TECH-04	One database provider (SQLite) with code-first EF Core migrations	M	
TECH-05	ASP.NET Core Identity for authentication	M	
TECH-06	Razor views, HTML5 semantic markup	M	
TECH-07	CSS3 with Bootstrap 5.3 customised by a project design system	M	
TECH-08	JavaScript for client-side processing where appropriate	M	
TECH-09	Multimedia (images, embedded video, PDF documents, SVG illustrations)	M	
TECH-10	Server-side processing (controllers, services, EF Core queries)	M	
TECH-11	Proper project and file organisation	M	

2. Guest (anonymous visitor)

ID	Requirement	Type	Status
GUEST-01	View Home page	M	
GUEST-02	View About page	M	
GUEST-03	Browse published courses	M	
GUEST-04	Search courses (server-side)	M	
GUEST-05	Filter courses by category and difficulty	M	
GUEST-06	Open course details	M	
GUEST-07	View selected public (preview) learning resources	M	
GUEST-08	Register	M	
GUEST-09	Login	M	
GUEST-10	View Contact page and send a contact message	M	

3. Registered user (Student)

ID	Requirement	Type	Status
STU-01	Student dashboard with meaningful, data-driven information	M	
STU-02	Dashboard: enrolled courses, completed courses, available courses	M	
STU-03	Dashboard: recent activity, quiz results, learning progress	M	
STU-04	View and update profile	M	
STU-05	Browse courses and view course details	M	
STU-06	Enrol in a course (and leave a course)	M	

ID	Requirement	Type	Status
STU-07	View My Courses	M	✓
STU-08	Access learning resources of enrolled courses	M	✓
STU-09	Track learning progress (mark resources complete)	M	✓
STU-10	Take quizzes	M	✓
STU-11	View quiz results and history	M	✓
STU-12	Logout	M	✓
STU-13	Change password	O	✓

4. Administrator

ID	Requirement	Type	Status
ADM-01	Separate, protected Admin area	M	✓
ADM-02	Admin dashboard with real platform statistics	M	✓
ADM-03	Courses CRUD (create, list, details, edit, delete)	M	✓
ADM-04	Categories CRUD	M	✓
ADM-05	Learning resources CRUD (incl. secure file upload)	M	✓
ADM-06	Enrolments management (list, enrol a student, remove)	M	✓
ADM-07	Quizzes CRUD	M	✓
ADM-08	Quiz questions and answer options CRUD, set correct answer	M	✓
ADM-09	Quiz results (attempts) view and delete	M	✓
ADM-10	Users: list, view details, change role, deactivate/reactivate, safe delete	M	✓
ADM-11	Website content: contact message inbox	O	✓

5. Database

ID	Requirement	Type	Status
DB-01	Relational model with primary keys and foreign keys	M	✓
DB-02	Required properties and maximum lengths	M	✓
DB-03	Unique constraints where appropriate	M	✓
DB-04	Indexes where useful	M	✓
DB-05	Intentional cascade / restrict delete behaviour	M	✓
DB-06	CreatedAt / UpdatedAt timestamps where useful	M	✓
DB-07	EF Core migrations reproducible from an empty database	M	✓
DB-08	Realistic seed data (8 categories, 12 courses, 64 resources, 11 quizzes, demo activity)	M	✓
DB-09	Insert, Display, Update, Delete operations demonstrated	M	✓
DB-10	ERD that exactly matches the EF Core entities	M	✓

6. Authentication and authorisation

ID	Requirement	Type	Status
AUTH-01	Register, Login, Logout	M	✓
AUTH-02	Password hashing through Identity (no plain-text passwords)	M	✓
AUTH-03	Roles: Guest, Student, Admin with role-based authorisation	M	✓
AUTH-04	Protected pages for students and admin-only controllers	M	✓
AUTH-05	Initial admin seeded from secrets / environment variables (no password in Git)	M	✓
AUTH-06	Account lockout after repeated failed logins	O	✓

7. Validation

ID	Requirement	Type	Status
VAL-01	Server-side validation with DataAnnotations and ModelState	M	✓
VAL-02	Client-side validation with ASP.NET validation helpers	M	✓
VAL-03	Bootstrap-styled, accessible validation feedback	M	✓
VAL-04	Required, email, password, length and range rules	M	✓
VAL-05	File validation (type, size, content signature)	M	✓
VAL-06	Business-rule validation (e.g. exactly one correct answer)	M	✓

8. Security

ID	Requirement	Type	Status
SEC-01	No SQL injection (EF Core parameterised queries only)	M	✓
SEC-02	XSS protection (Razor encoding, no raw user HTML, CSP)	M	✓
SEC-03	CSRF protection (anti-forgery tokens on all unsafe requests)	M	✓
SEC-04	IDOR protection (ownership checks on attempts, resources, files)	M	✓
SEC-05	Overposting protection (ViewModels, never bind entities)	M	✓
SEC-06	Safe file uploads (allow-list, magic bytes, size limit, random names, outside wwwroot)	M	✓
SEC-07	Safe video embedding (strict YouTube/Vimeo parsing, frame-src allow-list)	M	✓
SEC-08	No secrets committed to Git	M	✓
SEC-09	Admin routes protected by role authorisation	M	✓
SEC-10	Security headers, HTTPS/HSTS in production	O	✓
SEC-11	Rate limiting on login, registration and contact form	O	✓

9. Error handling and logging

ID	Requirement	Type	Status
ERR-01	Custom 404 page	M	✓
ERR-02	Custom 500 / error page without stack traces in production	M	✓
ERR-03	User-friendly validation and business error messages	M	✓
ERR-04	Database error handling (e.g. unique-constraint races)	M	✓
ERR-05	Structured logging of important events	M	✓

10. UI / UX, accessibility and responsiveness

ID	Requirement	Type	Status
UI-01	Coherent custom design system (not default Bootstrap)	M	✓
UI-02	Navigation bar per role, hero, course cards, dashboards, tables, forms, footer	M	✓
UI-03	Empty states and success/error messages	M	✓
UI-04	Breadcrumbs where useful	M	✓
UI-05	Responsive on mobile, tablet and desktop; collapsing navigation; usable tables	M	✓
UI-06	Semantic HTML, labels, alt text, keyboard access, contrast, heading structure	M	✓
UI-07	No placeholder/template text, no broken images or dead links	M	✓

11. Testing and quality

ID	Requirement	Type	Status
TEST-01	Automated unit tests for services and helpers	M	✓
TEST-02	Automated integration tests (auth, protected routes, CRUD, enrolment, quiz, search, validation, 404)	M	✓

ID	Requirement	Type	Status
TEST-03	Browser end-to-end tests across mobile, tablet and desktop viewports	O	✓
TEST-04	Manual testing checklist with recorded results	M	✓
TEST-05	Build without errors and without warnings	M	✓

12. Repository, CI/CD and deployment

ID	Requirement	Type	Status
GIT-01	Git repository with a professional structure and <code>.gitignore</code>	M	✓
GIT-02	Sensible commit history and documented branching strategy	M	✓
GIT-03	README with all required sections	M	✓
CI-01	GitHub Actions CI: restore, build, test, fail on error	M	✓
CI-02	GitHub Actions container workflow that reproduces the Railway runtime	M	✓
DEP-01	Production ASP.NET application packaged and ready to run on Railway	M	PENDING
DEP-02	SQLite production database on a persistent volume, configured through environment variables	M	PENDING
DEP-03	Documented migration strategy and production verification	M	✓
PAGES-01	Separate static presentation site with an "Open Learning System" call to action	M	✓
PAGES-02	Presentation site deployment automated and reproducible	M	✓

13. Documentation (university deliverables)

ID	Requirement	Type	Status
DOC-01	Proposal: title, objectives, mission, audience modelling, scope	M	✓
DOC-02	Final report content (full structure from the brief)	M	✓
DOC-03	ERD (Mermaid)	M	✓
DOC-04	Use cases	M	✓
DOC-05	Flowcharts (registration, login, enrolment, quiz, admin course CRUD)	M	✓
DOC-06	Wireframes	M	✓
DOC-07	Navigation structure diagram	M	✓
DOC-08	Testing documentation	M	✓
DOC-09	Deployment documentation	M	✓
DOC-10	Team responsibilities and Git workflow	M	✓
DOC-11	Viva preparation questions	M	✓

14. Notes on partial or pending items

Everything above is either verified in the code or listed here. No requirement is claimed that the code does not support.

ID	Note
DEP-01, DEP-02	The Docker image, <code>railway.json</code> , the environment-variable contract and <code>scripts/set-app-url.sh</code> are complete, and the <code>container</code> CI job runs the image the way Railway does, including a restart that proves the SQLite file on <code>/data</code> survives. The Railway CLI is installed but not authenticated , so no Railway deployment has been made and no Railway URL is claimed.
ADM-09	The three attempt pages exist and are crawled and scanned by the test suite, but there is no dedicated automated test for deleting a single attempt.
TEST-04	TESTING.md §7 records 13 passed checks. Keyboard-only journeys, a screen-reader pass, real devices and the live production check remain "To do" because they need a person or a deployment.
UI-06	Automated axe-core WCAG 2.2 A/AA scans pass on the public, student and admin pages at all three viewports. This does not replace the manual keyboard and screen-reader pass above.
PAGES-01	The presentation site is live at https://learnhub-wapp.web.app . Its buttons carry the conventional Railway address until <code>scripts/set-app-url.sh</code> rewrites it after the first deployment.
GIT-03	README.md has all required sections, but its badges, hosting table and deployment section still describe the earlier Azure hosting and are not yet updated.

15. Explicitly out of scope

To prevent scope creep, the following are **not** part of LearnHub v1.0: AI tutor, live video classrooms, real-time chat, payment processing, certificates, e-mail delivery (confirmation / password reset e-mails), instructor self-service role, discussion forums, mobile apps, multi-language UI.

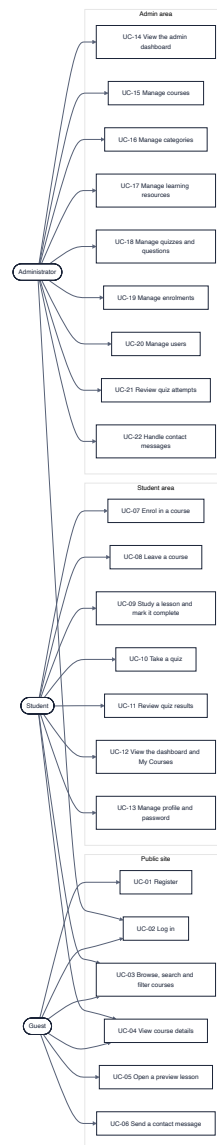
Use Cases

1. Actors

Actor	Description
Guest	Anyone who is not signed in.
Student	A registered user with the <code>Student</code> role (every new registration).
Administrator	A user with the <code>Admin</code> role, seeded from configuration or promoted by another administrator.
System	Automated behaviour: migrations and seeding at start-up, grading, progress calculation.

A signed-in student can do everything a guest can; administrators use the separate `/Admin` area.

2. Overview



3. Use case index

ID	Use case	Primary actor	Requirements	Implemented in
UC-01	Register	Guest	GUEST-08, AUTH-01, AUTH-02, VAL-01-04	<code>Controllers/AccountController.cs</code> (Register)
UC-02	Log in and log out	Guest, Student, Administrator	GUEST-09, AUTH-01, AUTH-06, STU-12	<code>AccountController</code> (Login, Logout)

ID	Use case	Primary actor	Requirements	Implemented in
UC-03	Browse, search and filter courses	Guest, Student	GUEST-03–05, STU-05	<code>CoursesController.Index</code> , <code>Services/CourseCatalogService.cs</code>
UC-04	View course details	Guest, Student	GUEST-06	<code>CoursesController.Details</code>
UC-05	Open a preview lesson	Guest	GUEST-07	<code>ResourcesController.Details</code> , <code>Services/LearningResourceService.cs</code>
UC-06	Send a contact message	Guest, Student	GUEST-10	<code>HomeController.Contact</code> , <code>Services/ContactService.cs</code>
UC-07	Enrol in a course	Student	STU-06	<code>CoursesController.Enroll</code> , <code>Services/EnrollmentService.cs</code>
UC-08	Leave a course	Student	STU-06	<code>CoursesController.Leave</code>
UC-09	Study a lesson and mark it complete	Student	STU-08, STU-09	<code>ResourcesController</code> (Details, Open, ToggleComplete)
UC-10	Take a quiz	Student	STU-10	<code>QuizzesController.Take</code> (GET and POST), <code>Services/QuizService.cs</code> , <code>Services/QuizGrader.cs</code>
UC-11	Review quiz results	Student	STU-11	<code>QuizzesController</code> (Result, History)
UC-12	View the dashboard and My Courses	Student	STU-01–03, STU-07	<code>StudentController</code> , <code>Services/DashboardService.cs</code>
UC-13	Manage profile and password	Student	STU-04, STU-13	<code>ProfileController</code>
UC-14	View the admin dashboard	Administrator	ADM-01, ADM-02	<code>Areas/Admin/Controllers/DashboardController.cs</code>
UC-15	Manage courses	Administrator	ADM-03	<code>Areas/Admin/Controllers/CoursesController.cs</code> , <code>Services/CourseManagementService.cs</code>
UC-16	Manage categories	Administrator	ADM-04	<code>Areas/Admin/Controllers/CategoriesController.cs</code> , <code>Services/CategoryService.cs</code>
UC-17	Manage learning resources	Administrator	ADM-05, VAL-05	<code>Areas/Admin/Controllers/ResourcesController.cs</code> , <code>Services/ResourceManagementService.cs</code>
UC-18	Manage quizzes and questions	Administrator	ADM-07, ADM-08, VAL-06	<code>Areas/Admin/Controllers/QuizzesController.cs</code> , <code>QuestionsController.cs</code>
UC-19	Manage enrolments	Administrator	ADM-06	<code>Areas/Admin/Controllers/EnrollmentsController.cs</code>
UC-20	Manage users	Administrator	ADM-10	<code>Areas/Admin/Controllers/UsersController.cs</code> , <code>Services/UserManagementService.cs</code>
UC-21	Review quiz attempts	Administrator	ADM-09	<code>Areas/Admin/Controllers/QuizAttemptsController.cs</code>
UC-22	Handle contact messages	Administrator	ADM-11	<code>Areas/Admin/Controllers/MessagesController.cs</code>

Controller paths are relative to `src/LearnHub/`.

4. Detailed use cases

UC-01 Register

Goal	Create a student account and start learning.
Actor	Guest
Preconditions	The visitor is not signed in.
Trigger	The guest selects Register or Create a free account .

Main success scenario

1. The system shows the registration form (full name, email, password, confirm password, terms checkbox).
2. The guest fills in the form. The browser validates it as they type and shows password strength.
3. The guest submits the form (with its antiforgery token).

4. The system validates the view model on the server again.
5. The system creates the Identity user with a hashed password and adds the `Student` role.
6. The system signs the student in and opens their dashboard.

Alternative and exception flows

- 2a/4a. A field is missing or invalid (email format, weak password, passwords differ, terms not accepted): the form is shown again with messages beside the fields and the entered values kept (except passwords).
- 5a. The email is already registered: one clear error is shown; no account is created.
- 3a. More than 10 submissions in a minute from the same address: the system returns `429 Too Many Requests`.
- 3b. The request has no valid antiforgery token: the system rejects it with `400 Bad Request`.

Postconditions: a new student exists and is signed in. **Tests:**

`AuthenticationTests.Registration_creates_a_student_signs_them_in_and_hashes_the_password`,
`Invalid_registration_is_rejected_by_server_side_validation`, `Registering_with_an_existing_email_shows_a_single_clear_error`.

UC-02 Log in and log out

Main success scenario

1. The user opens **Log in** and enters email and password (optionally "remember me").
2. The system checks the password with Identity (failed attempts are counted).
3. On success the system redirects to the local `returnUrl` if one was given; otherwise administrators go to `/Admin` and students to `/Student/Dashboard`.
4. **Log out** is a POST form in the user menu; the system ends the session and shows the home page.

Alternative and exception flows

- 2a. Wrong email or password: a generic message is shown that does not reveal whether the email exists.
- 2b. Five failed attempts: the account is locked for 15 minutes; the same message is used for a deactivated account.
- 3a. `returnUrl` points to another site: it is ignored and the default dashboard is used.
- 1a. Rate limit exceeded: `429 Too Many Requests`.

Tests: `AuthenticationTests.Students_and_admins_are_sent_to_their_own_dashboards_after_login`, `Wrong_password_shows_a_generic_error`,
`Account_is_locked_after_five_failed_attempts`, `Login_redirects_back_to_a_local_return_url_but_never_to_another_site`, `Logout_ends_the_session`.

UC-03 Browse, search and filter courses

Main success scenario

1. The user opens **Courses** (or searches from the home page).
2. The system lists published courses, nine per page, with category, difficulty, duration, lesson count and instructor.
3. The user types a term and/or chooses a category, difficulty or sort order and submits.
4. The system runs the search in the database (title, short description, instructor and category name) and shows the matching count and results; filters are kept in the URL so the page can be shared or bookmarked.

Alternative flows

- 4a. No course matches: an empty state ("No courses match these filters") suggests a shorter term and offers **Show all courses**.
- 3a. Wildcard characters such as `%` or `_` are treated as normal text.

Tests: `PublicSiteTests.Search_is_performed_on_the_server`, `Category_filter_limits_results_to_that_category`,
`CatalogueServiceTests.Search_returns_only_published_courses_matching_the_term_and_filters`.

UC-04 View course details

Main success scenario

1. The user opens a course from the catalogue, the home page or the dashboard.
2. The system shows the title, category, difficulty, duration, instructor, description, learning outcomes and the **course route**: lessons and quizzes in order.
3. For a guest, preview lessons are links and the other stops are marked as locked; the side panel offers **Create a free account** and **Log in to enrol**.
4. For an enrolled student, every stop is available, completed stops are ticked and progress is shown.

Exception flows

- 1a. The course does not exist or is a draft: the friendly 404 page is shown (administrators can still preview drafts in the admin area).

Tests: `PublicSiteTests.Course_details_show_route_and_enrolment_call_to_action`, `Draft_courses_are_not_visible_to_guests`.

UC-05 Open a preview lesson

1. A guest selects a lesson marked **Free preview** on the course route.
 2. The system shows the lesson content with a notice that it is a free preview and an invitation to enrol.
- Alternative: the guest opens a non-preview lesson URL directly → redirect to **Log in** with a return URL.

Tests: `PublicSiteTests.Preview_lessons_are_open_to_guests_but_other_lessons_require_login`.

UC-06 Send a contact message

1. The user opens **Contact** and enters name, email, subject and message.
2. The browser and then the server validate the fields.
3. The system stores the message for administrators and confirms that it was sent.

Alternative flows

- 2a. Invalid fields: messages are shown beside them.
- 3a. The hidden honeypot field is filled in (typical of spam bots): the system shows the same confirmation but does not store anything.
- 1a. Rate limit exceeded: `429 Too Many Requests`.

Tests: `AdminManagementTests.Contact_messages_reach_the_admin_inbox`, `Spam_honeypot_submissions_are_not_stored`.

UC-07 Enrol in a course

Actor	Student
Preconditions	Signed in as a student; the course is published.

Main success scenario

1. On the course page the student selects **Enrol**. The browser posts the form with its antiforgery token.
2. The system checks that the course exists and is published.
3. The system checks that the student is not already enrolled and stores the enrolment.
4. The course page shows "You're enrolled. Your route starts at the first lesson." and all stops become available.

Alternative and exception flows

- 2a. Course missing or unpublished: 404.
- 3a. Already enrolled: the page shows "You are already enrolled in this course."
- 3b. Two submissions arrive at the same moment: the unique `(UserId, CourseId)` index rejects the second insert and the student sees the same "already enrolled" message.
- 1a. A guest or administrator posts the form: the student role is required (login redirect or access denied).

Tests: `CatalogueServiceTests.Enrolling_twice_is_rejected_and_unpublished_courses_cannot_be_joined`, `StudentJourneyTests.A_new_student_enrols_studies_takes_the_quiz_and_sees_the_result`.

UC-08 Leave a course

1. On an enrolled course page the student selects **Leave this course** and confirms the prompt.
2. The system removes the enrolment; completed lessons and quiz attempts are kept.
3. The lessons of that course are locked again; if the student enrolls again, their progress returns.

Tests: `StudentJourneyTests.Leaving_a_course_removes_access_to_its_lessons`, `CatalogueServiceTests.Leaving_a_course_keeps_completed_lessons_for_later`.

UC-09 Study a lesson and mark it complete

Main success scenario

1. The student opens a lesson from the course route, the dashboard (**Resume**) or the outline.
2. The system checks access (preview, enrolment or administrator) and shows the lesson: article text, an embedded video, a PDF or image opened through a protected link, or an external link, with the course outline and previous and next lessons.
3. The student selects **Mark as complete**.
4. The system records the completion, recalculates progress and moves the student to the next lesson with "Lesson complete. On to the next stop."

Alternative flows

- 3a. The lesson is already complete: the button reads **Mark as not complete** and removes the record.
- 2a. Not enrolled: redirect to the course page, which offers enrolment.
- 2b. A file link is opened without access: the file is not served.

Tests: `StudentJourneyTests.A_new_student_enrols_studies_takes_the_quiz_and_sees_the_result`, `CatalogueServiceTests.Progress_counts_completed_lessons_and_passed_quizzes`, `AdminManagementTests.Resources_are_created_validated_streamed_and_deleted`.

UC-10 Take a quiz

Actor	Student
-------	---------

Preconditions	Enrolled in the course; the quiz is published and has questions.
---------------	--

Main success scenario

1. The student opens the quiz from the course route.
2. The system shows every question with its options; the correct answers are not part of the page.
3. The student chooses answers; a counter shows how many are answered.
4. The student submits. If some questions are unanswered, the browser asks for confirmation first.
5. The system grades on the server: only options that belong to each question count, unanswered questions are incorrect, and the score is rounded down.
6. The system stores the attempt with a score snapshot and every answer, and opens the result page.

Alternative and exception flows

- 1a/5a. Not enrolled: redirect to the course page.
- 1b. Quiz missing or unpublished: 404.
- 5b. Tampered form values (for example an option id from another question): ignored and marked incorrect.

Postconditions: a new attempt; a passed quiz counts towards course progress. Quizzes can be retaken. **Tests:**

```
StudentJourneyTests.Tampered_quiz_answers_are_graded_on_the_server ,
QuizAndCourseManagementTests.Submitting_answers_stores_a_graded_attempt_with_every_answer , The_quiz_page_never_contains_the_correct_answers ,
LogicTests.Unanswered_questions_count_as_incorrect .
```

UC-11 Review quiz results

1. After submitting, or from **Quiz results** history, the student opens an attempt.
 2. The system shows the score, pass or fail against the pass mark, submission time and, for every question, the chosen answer, the correct answer and the explanation.
- Exception: the attempt belongs to someone else → 404.

Tests: `QuizAndCourseManagementTests.Students_cannot_open_other_students_results_but_admins_can` .

UC-12 View the dashboard and My Courses

1. After login, or from the navigation, the student opens **Dashboard**.
 2. The system shows figures (enrolled courses, completed courses, lessons completed, quizzes passed, courses still available), courses in progress with progress bars and **Resume**, recent quiz results with the average best score, recent activity and three recommended courses.
 3. **My Courses** lists every enrolled course with progress and the next lesson.
- Alternative: a new student with no enrolments sees empty states that link to the catalogue.

UC-13 Manage profile and password

1. The student opens **Profile**, edits full name and bio, and saves; the system validates and saves.
 2. From **Change password** the student enters the current password and a new one twice; the system checks the current password and the policy, changes it and refreshes the sign-in.
- Alternatives: validation errors are shown beside the fields; a wrong current password shows an error; rate limit applies to password changes.

Tests: `StudentJourneyTests.Profile_updates_are_validated_and_saved` .

UC-14 View the admin dashboard

The administrator opens `/Admin` and sees live figures (published and draft courses, students and new students in the last 30 days, enrolments, quiz attempts with pass rate and average score, unread messages), most enrolled courses, enrolments by category, recent enrolments, recent quiz attempts and content warnings such as a published course without lessons. Non-administrators are redirected to login or shown **Access denied**.

UC-15 Manage courses**Main success scenario (create)**

1. The administrator opens **Courses** → **New course**.
2. The administrator enters title, short description, description, learning outcomes, category, difficulty, duration, instructor and optionally a cover image, and chooses whether to publish.
3. The system validates the form and the image (type, size and signature), stores the image under a random name, saves the course and shows the course details.

Other flows

- Edit: the same form, pre-filled; replacing the cover deletes the old uploaded image.
- Publish or unpublish from the list or details page.
- Delete: the confirmation page shows how many lessons, quizzes, enrolments and attempts will be removed; on confirmation the system deletes the quiz answers first and then the course in one transaction, then removes files.

- Validation errors keep the form open with messages.

Tests: `AdminManagementTests.Course_create_with_cover_edit_publish_and_delete` ,
`QuizAndCourseManagementTests.Deleting_a_course_removes_its_content_attempts_and_answers_in_the_right_order` .

UC-16 Manage categories

1. The administrator creates or edits a category with a unique name, a description and an icon from the allowed list.
2. Deleting a category that still has courses is refused with an explanation; an empty category can be deleted.

- Alternative: a name that already exists (ignoring case) is rejected.

Tests: `AdminManagementTests.Category_create_edit_and_delete` , `Category_with_courses_is_protected_from_deletion` ,
`CatalogueServiceTests.Creates_categories_and_rejects_duplicate_names_regardless_of_case` .

UC-17 Manage learning resources

1. The administrator creates a resource for a course: title, summary, type, order, estimated minutes and preview flag.
2. Depending on the type, the form shows the article body, the video URL (YouTube or Vimeo), the external link, or a file upload (PDF or image).
3. The system validates the type-specific fields, checks the file, stores it privately and saves the resource.

- Alternatives: an unsupported video address, a wrong file type, a file whose content does not match its extension or a file over the limit is rejected with a message; delete removes the record and the file.

Tests: `AdminManagementTests.Resources_are_created_validated_streamed_and_deleted` ,
`FileStorageServiceTests.Rejects_a_file_whose_content_does_not_match_its_extension` .

UC-18 Manage quizzes and questions

1. The administrator creates a quiz for a course with a title, description and pass mark (0–100).
2. On the quiz details page the administrator adds questions with an explanation and two to six answer options, marking exactly one as correct.
3. The administrator publishes the quiz.

- Alternatives: fewer than two options, no or several correct options, or duplicate options are rejected; publishing a quiz without questions shows "Add at least one question before publishing this quiz."; deleting a question or quiz removes the related student answers first.

Tests: `AdminManagementTests.Quiz_must_have_questions_before_it_can_be_published` ,
`LogicTests.Question_needs_two_answers_one_marked_correct_and_no_duplicates` ,
`QuizAndCourseManagementTests.Removing_an_answer_that_students_selected_keeps_their_attempts` .

UC-19 Manage enrolments

1. The administrator lists enrolments (search by student, filter by course), or selects **Enrol a student**, enters the student's email and chooses a course.
2. The system checks that the account exists and is a student and that no enrolment exists, then saves it.
3. The administrator can remove an enrolment after confirmation.

- Alternatives: "No account uses that email address.", "Only student accounts can be enrolled in courses.", "This student is already enrolled in that course."

Tests: `AdminManagementTests.Enrolments_can_be_managed_by_administrators` .

UC-20 Manage users

1. The administrator searches the user list (filter by role) and opens a user's details (enrolments, attempts, status).
2. The administrator can change the role, deactivate or reactivate the account, or delete it after confirmation.

- Exceptions: an administrator cannot change, deactivate or delete their own account; the last active administrator cannot be demoted, deactivated or deleted. A deactivated user cannot sign in, and an existing session ends within five minutes.

Tests: `AdminManagementTests.User_roles_deactivation_and_self_protection` .

UC-21 Review quiz attempts

The administrator searches attempts by student and filters by quiz, opens one to see every answer, and may delete an attempt after confirmation (for example a test attempt).

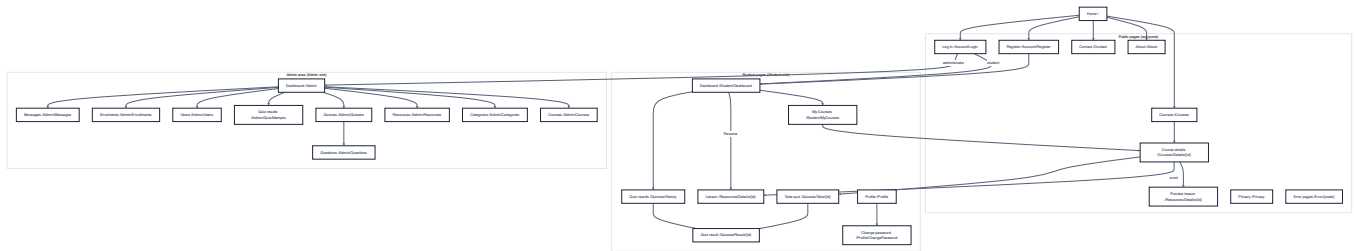
UC-22 Handle contact messages

The administrator sees the inbox newest first, can show unread messages only (the sidebar shows the unread count), opens a message (which marks it read), can mark it unread again, and can delete it after confirmation.

Navigation Structure

Navigation is defined in `src/LearnHub/Views/Shared/_Layout.cshtml` (public and student pages) and `src/LearnHub/Areas/Admin/Views/Shared/_AdminLayout.cshtml` (admin area). The current section is highlighted by the `lh-nav` tag helper, which also sets `aria-current="page"`.

1. Site map



2. Main navigation by role

Element	Guest	Student	Administrator
Brand (LearnHub mark)	Home	Home	Home
Home, Courses	Yes	Yes	Yes
My Courses, Dashboard	–	Yes	–
About, Contact	Yes	In the footer	In the footer
Admin	–	–	Yes (opens <code>/Admin</code>)
Right-hand side	Log in, Register	User menu: Dashboard, Quiz results, Profile, Log out	User menu: Admin dashboard, Profile, Log out

Log out is a POST form inside the user menu, so it cannot be triggered by a link on another site.

Admin area sidebar

Group	Links
–	Dashboard
Catalogue	Courses, Categories, Resources
Assessment	Quizzes (also highlighted on question pages), Quiz results
People	Users, Enrolments, Messages (with the unread count)
Website	Main website, Log out

The admin top bar shows the page title, **View site** and the administrator's profile link.

Footer

Column	Links
Explore	All courses, About LearnHub, Contact
Account	Guest: Create an account, Log in, Privacy. Signed in: Profile, Privacy
Project	Module and university information

3. Breadcrumbs

Breadcrumbs appear where pages are nested more than one level deep:

- **Public and student:** course catalogue and course details (Home / Courses / category / course), lessons (Courses / course / lesson), quiz and quiz result pages, and Change password.
- **Admin:** create, edit, details and delete pages for courses, categories, resources, quizzes, questions, quiz attempts, enrolments, users and messages, for example Courses / *course title* / Edit.

4. Routes

URL	Page	Access	Methods
/	Home	Everyone	GET
/About , /Privacy	About, Privacy	Everyone	GET
/Contact	Contact form	Everyone (rate limited)	GET, POST
/Courses?q=&categoryId=&difficulty=&sort=&page=	Course catalogue	Everyone	GET
/Courses/Details/{id}	Course details	Everyone (published courses)	GET
/Courses/Enroll/{id} , /Courses/Leave/{id}	Enrol, leave	Student	POST
/Resources/Details/{id}	Lesson	Preview: everyone; otherwise enrolled students and administrators	GET
/Resources/Open/{id}	Lesson file (PDF or image)	Same as the lesson	GET
/Resources/ToggleComplete/{id}	Mark complete or not complete	Enrolled student	POST
/Quizzes/Take/{id}	Quiz form and submission	Enrolled student	GET, POST
/Quizzes/Result/{id}	Quiz result	The student who submitted it	GET
/Quizzes/History	Quiz results history	Student	GET
/Student/Dashboard , /Student/MyCourses	Dashboard, My Courses	Student	GET
/Profile	Profile	Signed-in user	GET, POST
/Profile/ChangePassword	Change password	Signed-in user (rate limited)	GET, POST
/Account/Register , /Account/Login	Register, log in	Everyone (rate limited)	GET, POST
/Account/Logout	Log out	Signed-in user	POST
/Account/AccessDenied	Access denied (status 403)	Everyone	GET
/Error , /Error/{statusCode}	Friendly error pages	Everyone	GET
/health	Database health check	Everyone	GET
/Admin	Admin dashboard	Admin	GET
/Admin/Courses + /Create , /Edit/{id} , /Details/{id} , /Delete/{id} , /TogglePublished/{id}	Course management	Admin	GET, POST
/Admin/Categories + /Create , /Edit/{id} , /Delete/{id}	Category management	Admin	GET, POST
/Admin/Resources + /Create , /Edit/{id} , /Delete/{id}	Learning resource management	Admin	GET, POST
/Admin/Quizzes + /Create , /Edit/{id} , /Details/{id} , /Delete/{id} , /SetPublished/{id}	Quiz management	Admin	GET, POST
/Admin/Questions/Create?quizId= + /Edit/{id} , /Delete/{id}	Question management	Admin	GET, POST
/Admin/QuizAttempts + /Details/{id} , /Delete/{id}	Quiz results	Admin	GET, POST
/Admin/Enrollments + /Create , /Delete/{id}	Enrolment management	Admin	GET, POST
/Admin/Users + /Details/{id} , /SetRole/{id} , /Deactivate/{id} , /Reactivate/{id} , /Delete/{id}	User management	Admin	GET, POST
/Admin/Messages + /Details/{id} , /MarkUnread/{id} , /Delete/{id}	Contact messages	Admin	GET, POST

Every state-changing action is a POST protected by an antiforgery token.

5. Redirects and error behaviour

Situation	Result
A guest opens a page that needs sign-in	302 to <code>/Account/Login?ReturnUrl=...</code> ; after login the user returns to that page if it is on this site
A signed-in user opens a page for another role	302 to <code>/Account/AccessDenied</code> , which responds with status 403
An unknown URL, a draft course or a non-existent id	Friendly 404 page ("This stop is not on the map") with links back to courses
Another student's quiz result	404, so result ids cannot be probed
A student opens a lesson or quiz of a course they have not joined	Redirect to the course page, which offers Enrol
An unexpected error in production	Friendly 500 page without technical details; the error is logged
Too many form submissions	429 Too Many Requests

6. Responsive behaviour

- **Main navigation:** below 992 px the menu collapses behind a toggle button labelled "Toggle navigation".
- **Admin sidebar:** below 992 px it becomes an off-canvas panel opened from the top bar ("Open navigation").
- **Admin tables:** below 768 px they become stacked records, each value shown with its column label; wider tables scroll inside their own container.
- **Course grids:** one column on phones, two on tablets, and three or four on desktops.
- **Skip links:** every layout starts with "Skip to main content" for keyboard users.

Wireframes

Low-fidelity wireframes of the main pages, matching the implemented layouts. The finished pages can be seen in the screenshots in [FirebaseLanding/assets/screens/](#) and on the [presentation site](#).

The ten sections cover the main journeys rather than every page. The admin sections stand for the whole area, so Categories, Resources, Quizzes, Quiz results, Enrolments, Users and Messages reuse the list and form layouts drawn in sections 9 and 10; on the public side, Categories, About, Contact, Privacy, Profile, My Courses and Progress reuse the page patterns drawn above.

Design concept

LearnHub is an interchange. Every category is a coloured transit line, a course is a route, lessons and quizzes are stops, and a student's progress is how far along the route they are. The idea shapes real components: the home page map where category lines meet at the LearnHub hub, the "course route" on course pages, the lesson outline, the progress bars with a signal-yellow marker, and the admin sidebar where the current section has a yellow platform edge.

Token	Value	Use
Ink	#1b2438	Text, primary buttons, admin sidebar
Paper	#f5f7fa	Page background of content areas
Surface	#ffffff	Cards, panels, forms
Rail	#dde3eb	Borders and dividers
Muted	#566076	Secondary text
Signal	#ffc53d	Main calls to action, current step, focus halo
Go / Stop	#107a55 / #c12f3c	Success and error states
Line colours	#4b5a73 , #c9344a , #157f8e , #6247c2 , #b0520c , #587414 , #2865c9 , #a8337d	One per category
Typeface	Overpass (variable weight)	Headings and body

Breakpoints follow Bootstrap: phones below 768 px, tablets 768–991 px, desktops from 992 px. The navigation collapses and the admin sidebar becomes an off-canvas panel below 992 px; admin tables become stacked records below 768 px.

1. Home

Desktop

```

+-----+
| [mark] LearnHub   Home  Courses  Categories  About                               Log in  [Register] |
+-----+
|
|                               Web Development      Programming |
| Build real computing      | 3 courses          2 courses |
| skills, one lesson at a time. | o====o\          /o====o |
|                               \          / |
| Short introduction to the catalogue | Cloud      ( LearnHub hub ) Security |
|                               o====o====(          )====o====o |
| +-----+ |                               /          \ | | |
| | (search) Search for SQL, security...[Go]| | o====o/          \o====o |
| +-----+ | Databases          Networking |
| 11 courses, 64 lessons and 11 quizzes ... |
+-----+
| Featured courses                               [Browse all courses] |
| +-----+ +-----+ +-----+ |
| | [cover: line+icon] | | [cover] | | [cover] | |
| | Category          | | Category | | Category | |
| | Course title       | | Course title | | Course title | |
| | Short description  | | Short description | | Short description | |
| | (Level) 10 h 6 les | | (Level) 8 h 5 les | | (Level) 12 h 7 les | |
| | Instructor 4 lnr | | Instructor 4 lnr | | Instructor 3 lnr | |
| +-----+ +-----+ +-----+ |
| (second row of three cards) |
+-----+
| How learning works on LearnHub |
| (1)------(2)------(3)------(4) |
| Find a course   Enrol for free   Work through   Check your understanding |
+-----+
| Ready to start your first course? [Create a free account] [Log in] |
+-----+
| Footer: brand and line colours | Explore | Account | Project |
+-----+

```

Phone

```

+-----+
| [mark] LearnHub      [===] |
+-----+
| Build real computing |
| skills, one lesson at a time. |
| Introduction         |
| +-----+ |
| | Search...      [Go] | |
| +-----+ |
| (o) Cloud Computing 1 |
| (o) Cybersecurity   1 |
| (o) Databases 1 (o) Networking |
+-----+
| Popular courses      |
| [Browse all courses] |
| +-----+ |
| | [cover] | |
| | Course title ... | |
| +-----+ |
| (cards stacked) |
+-----+

```

- The map is decorative on small screens, so phones and tablets get the category tiles instead, each a link to the filtered catalogue.
- The featured courses grid shows complete rows: one, two or three columns.
- The four "how it works" steps are a real sequence, so they sit on one numbered line.
- The header navigation depends on the role: a guest or a student sees Home, Courses, Categories and About, while an administrator sees Admin Dashboard; **Contact** lives in the footer rather than the header. The hero search box accepts the real placeholder text ("Search for SQL, binary, ASP.NET...").
- The real home page also carries "Browse by category" tiles, "Why learn with LearnHub", "Every kind of lesson in one place" (with a free preview link and a sample quiz question) and a closing call to action; they sit between the featured courses and the "how learning works" steps drawn above.

2. Course catalogue

Desktop				Phone			
+-----+				+-----+			
Courses				Courses			
Search titles, descriptions and lessons				Search titles...			
[Search courses.....] [Search]				[Search.....][Search]			
+-----+				+-----+			
Filter courses 11 courses found				Filter courses			
Category +-----+ +-----+ +-----+				All categories 9			
All 9 card card card				Cloud Computing 1			
Cloud 1 +-----+ +-----+ +-----+				Cybersecurity 1			
Security 1 +-----+ +-----+ +-----+				Level [v] Sort [v]			
Databases 1 card card card				[Apply]			
... +-----+ +-----+ +-----+				+-----+			
Level [v] (up to 9 per page)				11 courses found			
Sort [v]				+-----+			
[Apply] [< Previous] 1 2 [Next >]				card			
+-----+				+-----+			
				(stacked)			
				+-----+			

- Categories are coloured line links with course counts; the selected line is highlighted and marked with `aria-current`.
- The search box matches course titles, descriptions, categories and lesson titles (not instructors), and the page lists the courses whose lessons matched.
- Level and sort submit automatically when changed (the **Apply** button covers browsers without JavaScript). The sort options are Newest, Most popular, Title (A–Z) and Shortest first.
- Active filters appear as removable chips above the results, with a **Clear all** link.
- All filters are GET parameters, so results can be bookmarked and the back button works.
- An empty result shows "No courses match these filters" with **Show all courses**.

3. Course details

+-----+							
Home / Courses / Networking / Networking Fundamentals							
(line) Networking				+-----+			
Networking Fundamentals				[large cover: line + icon]			
Short description							
(Beginner) 9 h 5 lessons 1 quiz 2 learners				+-----+			
Taught by Ms. Priyanka Rao							
+-----+							
About this course				+-----+			
Description paragraphs				Start learning			
				[Create a free account]			
What you will learn				[Log in to enrol]			
(v) outcome (v) outcome				or, when enrolled:			
(v) outcome (v) outcome				Your progress [====>]			
				[Resume] Leave this course			
Course route				Beginner level			
(o)--- How data travels Article 10 min [Free preview]				About 9 h			
				Updated Jul 21, 2026			
(lock) The OSI model Image 5 min				+-----+			
(lock) ...							
(lock) Networking quiz Quiz with 5 questions Pass mark 60%							
+-----+							
More in Networking: related course cards							
+-----+							

- The route line uses the category colour; stops show done (tick), current (yellow) or locked states.
- From 992 px the enrolment panel is a sticky column beside the content; on phones and tablets it follows the course route, after the description.
- The panel has four real variants: a guest sees **Start learning** with *Create a free account* and *Log in to enrol*; a student sees **Enrol in this course**; an enrolled student sees **Your progress** with *Continue learning* and *Leave this course*; an administrator sees **Administrator view** with *Manage course*.
- Status chips mark a free preview, a draft ("visible to admins only") and the student's best quiz score. "What you will learn" appears only when outcomes are set, and "More in {Category}" lists related courses below the route.

4. Lesson player

Desktop	Phone
Courses / ASP.NET Core MVC / Models... [=====] 75% Course route (v) How MVC... (v) Pipeline (*) Models... () EF Core () Status codes [Course overview] Article 15 min Models, validation and ModelState Summary Content: headings, paragraphs, code blocks, notes, or video / PDF image / external link [Complete and continue] [< Previous lesson] [Next lesson >]	Courses / ... / Lesson [=====] 75% Article 15 min Lesson title Content [code block scrolls ->] [Complete and continue] [< Previous] [Next >] Course route (below)

- The outline is sticky on desktop and follows the content on phones.
- The lesson action reads **Complete and continue**; a completed lesson shows a *Completed* flag with **Mark as not complete**, and a guest sees the free-preview note instead.
- A PDF lesson adds **Download PDF (size)** and **Open in a new tab**, an exercise adds a **Show solution** disclosure, and a video link that fails validation shows a warning banner rather than a player.
- Wide code blocks scroll inside their own box (and can be focused with the keyboard) instead of widening the page.

5. Quiz and result

Take quiz	Result
Dashboard / Course / MVC architecture quiz MVC architecture quiz 5 questions 12 points Pass mark 70% Choose one answer for each question... Question 1 () option () option () option () option (one fieldset per question) 3 of 5 answered [Cancel] [Submit answers]	Quizzes / Course / MVC architecture quiz MVC architecture quiz [Retake quiz] [Back] 100% Passed Pass mark 70% 5 of 5 points (5 questions correct). Answer review Question 1: correct () option (v) chosen and correct Why: explanation

- Each question is a `fieldset` with a `legend`, so screen readers announce the question with its options.
- Submitting with unanswered questions asks for confirmation first; grading happens only on the server.
- Correct answers are green, a wrong choice is red with the correct answer shown, and every question explains why.
- The quiz page shows the total points as well as the question count, and the submit bar carries a **Cancel** link back to the course. The result page offers **Retake quiz** (only while the quiz is published and has questions) and **Back to course**, and its score panel reads "X of Y points (C of Q questions correct), which is N%" with the pass mark.

6. Student dashboard

```

+-----+
| Welcome back, Aisyah                                     [Browse courses] |
| You have completed 63% of your enrolled courses. Keep going.           |
+-----+
| [3 Enrolled] [1 Completed] [12 Lessons completed] [2 Quizzes passed] [6 Can still join] |
+-----+
| Continue learning                                     All my courses | Recent activity | | |
| +-----+ +-----+ +-----+ | (v) Completed "... |
| | [cover] Course title [=====] 14% [Resume] | | date |
| | Next: lesson title | | (+) Enrolled in ... |
| +-----+ +-----+ +-----+ | (*) Scored 80% in ... |
| (up to 3 courses) | | (list continues) |
| | |
| Recent quiz results                                     All results |
| | Quiz | Submitted | Score | Review | |
+-----+
| Recommended for you                                     [Browse all courses] |
| +-----+ +-----+ +-----+ +-----+ |
| | course card | | course card | | course card | |
| +-----+ +-----+ +-----+ +-----+ |
+-----+

```

- On phones the figure strip becomes two columns and every section stacks: continue learning, results, activity, recommendations.
- Empty states (no enrolments, no attempts) explain the next step and link to the catalogue.
- Two sibling pages reuse these patterns: **My courses** (the same rows with All, In progress and Completed filters) and **Progress** (an overall bar plus a per-course breakdown of lessons completed and quizzes passed).

7. Login and registration

```

+-----+
| +-----+ What your account includes |
| | Create your student account | (*) Enrol in any published course for free |
| | Already registered? Log in. | |
| | Full name [.....] | (o) Open every lesson, video and PDF |
| | 0 / 100 | |
| | Email address [.....] | (o) Take quizzes and review answers |
| | Password [.....] | |
| | [strength meter: ####-----] | (o) Track your progress on a dashboard |
| | Use at least 8 characters with ... |
| | Confirm password [.....] |
| | [ ] I will use LearnHub for learning |
| | and I have read the privacy notice |
| | [ Create account ] |
| +-----+ |
+-----+

```

- The login page uses the same card with email, password, "remember me" and a link to registration.
- Validation messages appear under each field with an icon; the password hint is hidden while the same rule is shown as the error.
- On phones the benefits list moves below the form.
- There is no password-reset page: the login page points at **Contact us**, and a forgotten password is reset by an administrator. Roles are managed by administrators too, so the profile page never offers them.

8. Admin dashboard

Desktop	Phone
[mark] LH Admin Dashboard Live figures for the catalogue [Users] [Courses] [Categories] [Resources] [Enrolments] [Quizzes] + four activity figures Needs attention (if any) Most enrolled courses (bars) Recent enrolments Recent quiz attempts Quiz results Messages 2 Main website Log out	[=] Dashboard [->] (LA) Dashboard [New quiz] [New course] [9 Published] [7 Students] [21 Enrol.] [7 Attempts] [2 Unread messages] Most enrolled (bars) By category (bars) Recent enrolments Recent attempts

- The dark sidebar keeps the admin area visually separate from the public site. It is a flat list — Dashboard, Users, Courses, Categories, Resources, Quizzes, Enrolments — followed by a single **More** group holding Quiz results, Messages (with an unread-count chip), Main website and Log out. There are no "Catalogue", "Assessment", "People" or "Website" headings, and Users sits directly under Dashboard.
- Below 992 px the sidebar becomes an off-canvas panel opened from the hamburger in the top bar; it closes from its own button, and the top bar also carries View site and the signed-in user's avatar and name.
- The dashboard shows two figure strips (catalogue figures, then learner activity) rather than one, then a **Needs attention** panel when content is missing, then the charts and the two tables, and closes with a catalogue summary line. On phones the figures drop to two per row.

9. Admin list page (for example Courses)

Desktop	Phone (stacked records)
Courses 10 courses match the current filters. [Search.....] [Category v] [Status v] [Apply] Reset	Courses [Search.....] [Category v] [Status v] [Apply] Reset
Cover Course Level Status Les Quiz Lrn Upd [img] Title Category	Course title Category Level Status Lessons
3 0 0 ... [Edit] [Delete]	Beginner Draft 3 [Edit] [Delete]
Showing 1–10 of 10 [pages]	[Edit] [Delete]

- Status uses badges with text (not colour alone); besides Draft and Published there are "Published, no questions" on quizzes, "Deactivated", "Locked" and "Active" on users, "Unread" on messages, and "Draft", "Free preview" and "Enrolled only" on resources.
- Every list page carries a filter bar, a "Showing X–Y of Z" footer and paging with 20 rows per page.
- Below 768 px the tables become stacked records, with each cell labelled from its column heading.
- Deleting always opens a confirmation page that explains the consequences, not a modal.

10. Admin form page (for example New course)

+-----+	
Courses / New course	
New course	
+-----+	
Course details	Cover image
Title [.....]	[current image]
Short description [.....]	[] Remove the image
Description [textarea.....]	Replace the image
Learning outcomes [textarea.....]	[Choose file] [preview]
-----	JPG, PNG or WebP, 2 MB
Organisation	Publishing
Category [v] Difficulty [v] Duration (minutes) []	[on/off] Published
Instructor [.....]	Drafts are hidden
[Cancel] [Create course]	
+-----+	

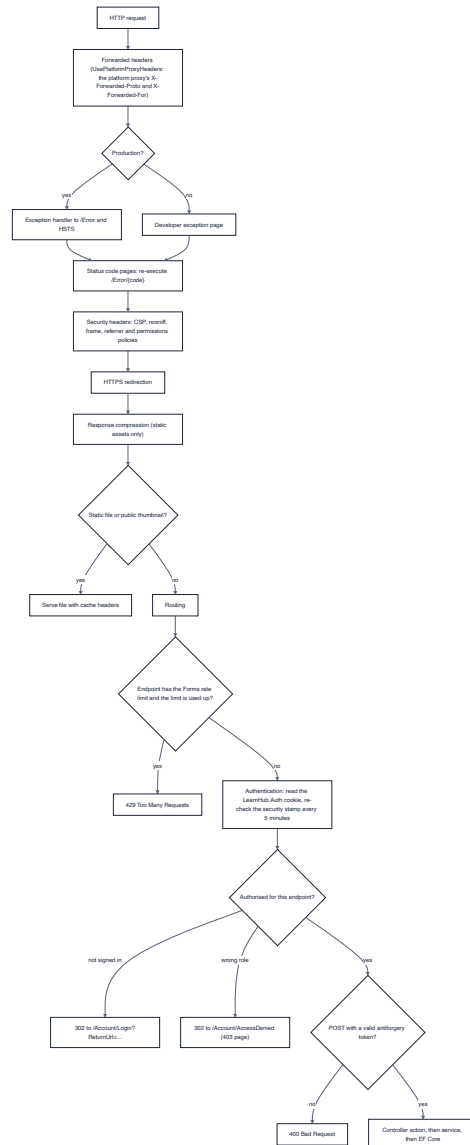
- From 1200 px the side panel stays visible next to the form; below that it follows the form sections. The aside is two panels: **Cover image** and **Publishing**, the second holding the Published switch and the note "Drafts are only visible to administrators."
- Character counters, image preview and type-specific resource fields are progressive enhancements; the server repeats every check.

Flowcharts

Each flowchart follows the real code path. File paths are relative to `src/LearnHub/`.

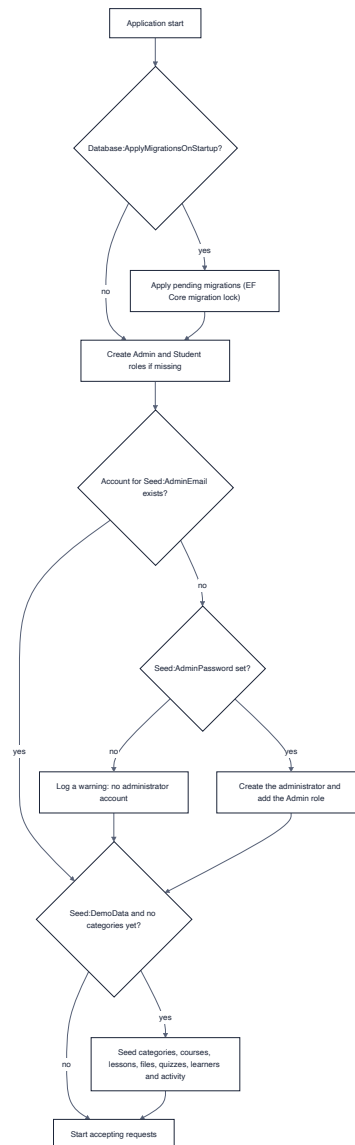
1. Request pipeline

Every request passes through the middleware configured in `Program.cs`. Railway terminates TLS at its edge proxy and forwards plain HTTP with `X-Forwarded-Proto` and `X-Forwarded-For`, so `UsePlatformProxyHeaders()` runs first and turns those headers into `Request.IsHttps` and the real client address.



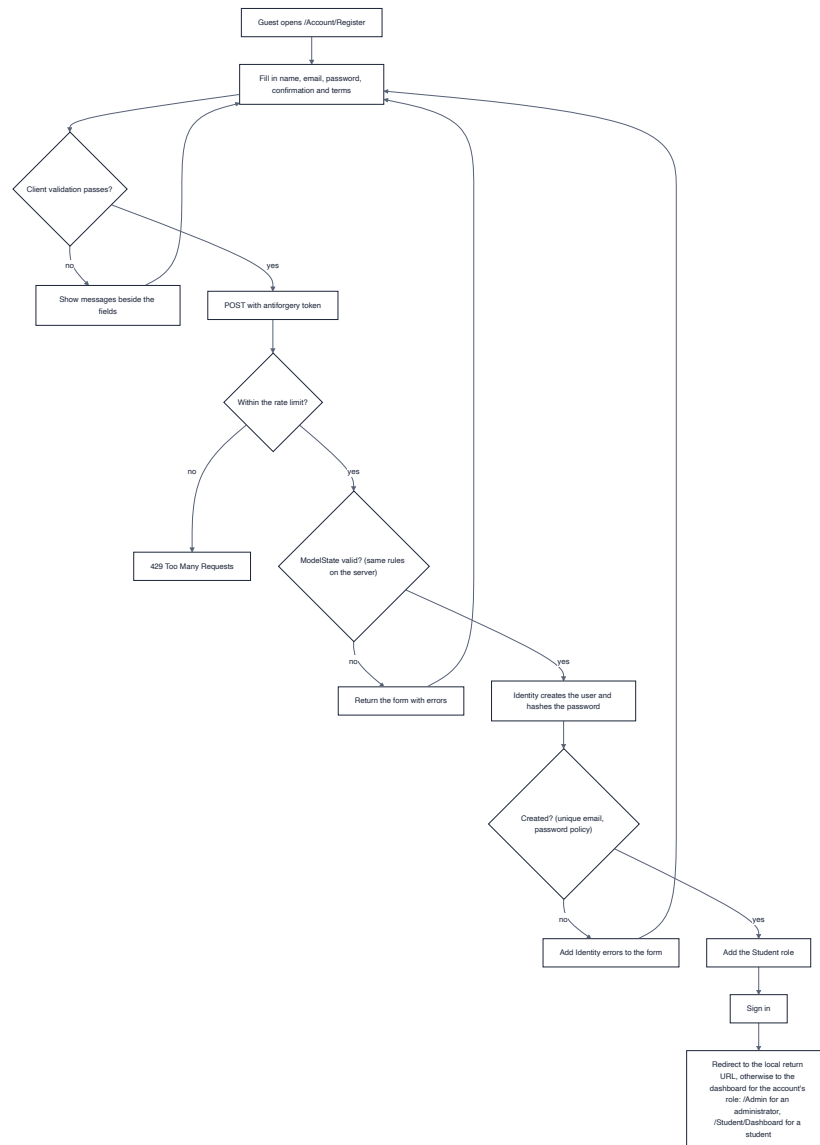
2. Start-up: database initialisation

`Data/Seed/DatabaseInitializer.cs` runs once before the application starts listening.



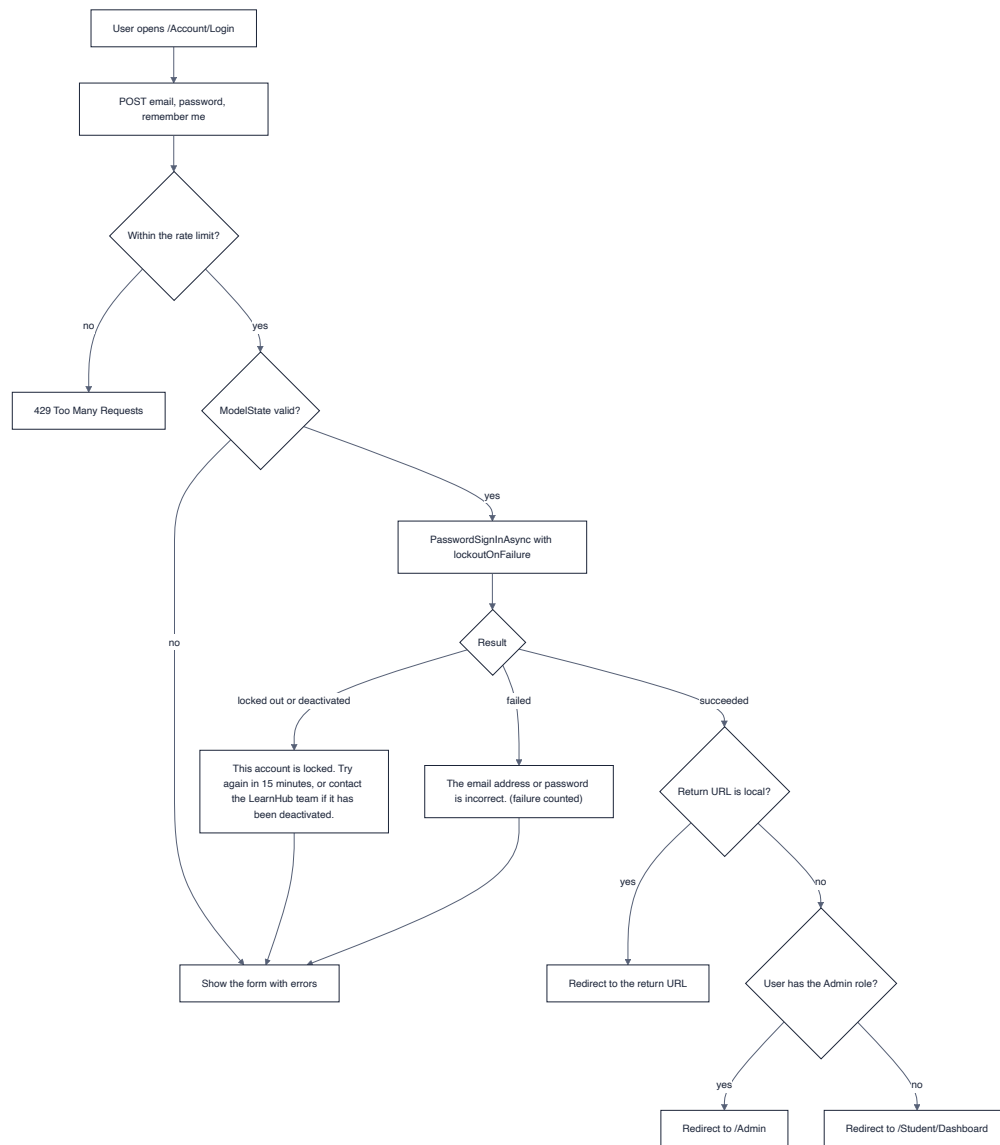
3. Registration

`Controllers/AccountController.cs` (Register) and `ViewModels/Account/AccountViewModels.cs`.



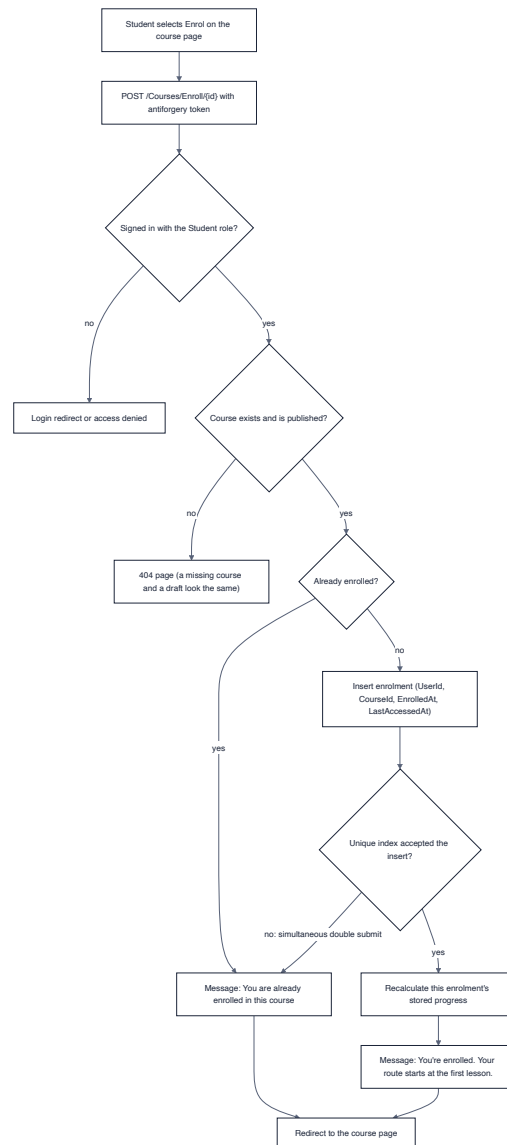
4. Login

`Controllers/AccountController.cs` (Login). Identity options: 5 failed attempts lock the account for 15 minutes.



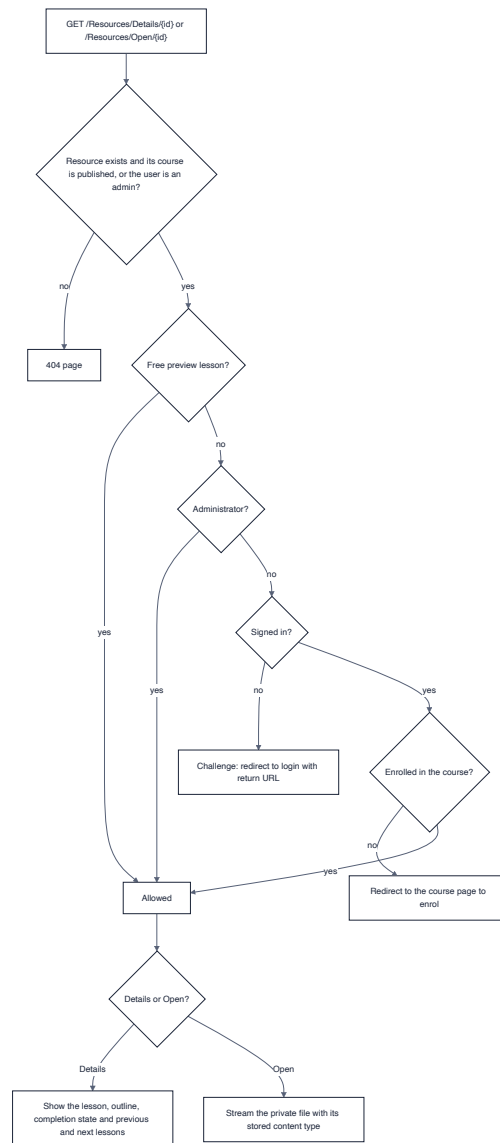
5. Enrolling in a course

`Controllers/CoursesController.cs` (Enroll) and `Services/EnrollmentService.cs`.



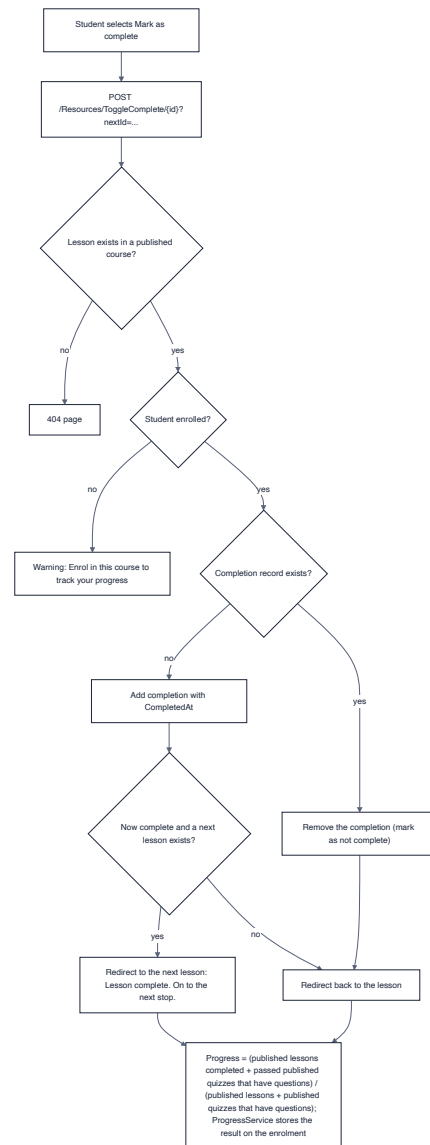
6. Opening a lesson or a lesson file

`Controllers/ResourcesController.cs` (Details, Open) and `Services/LearningResourceService.cs`.



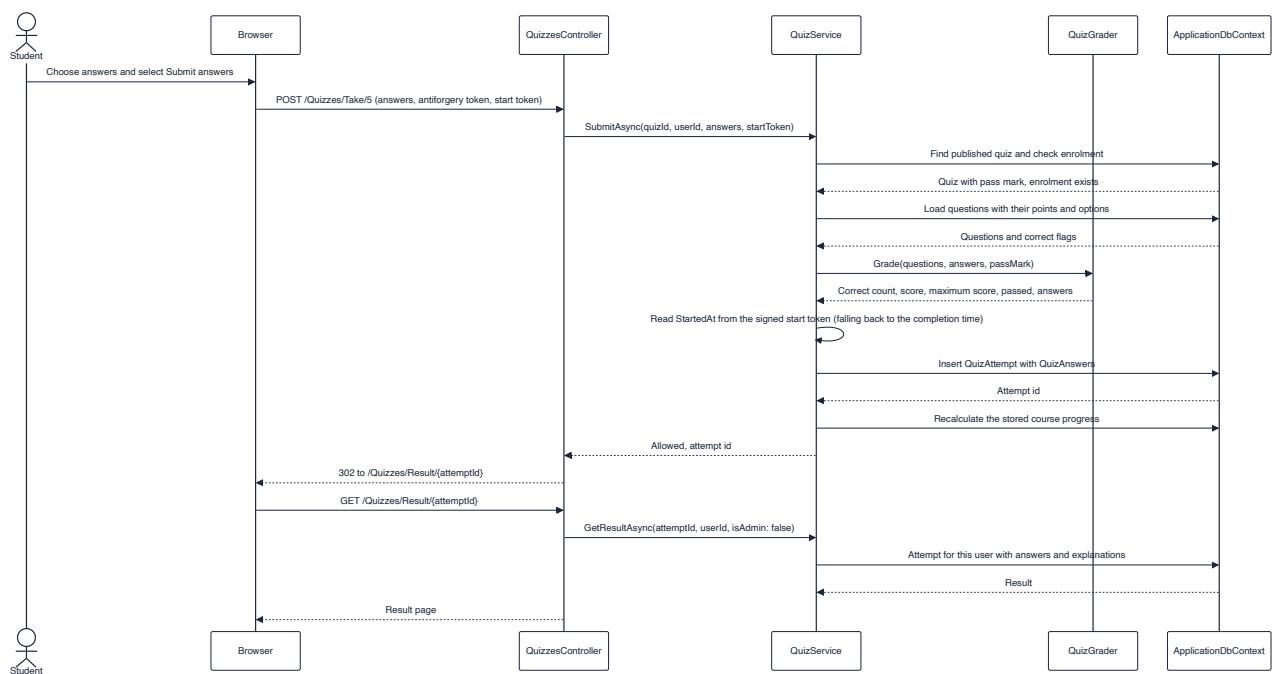
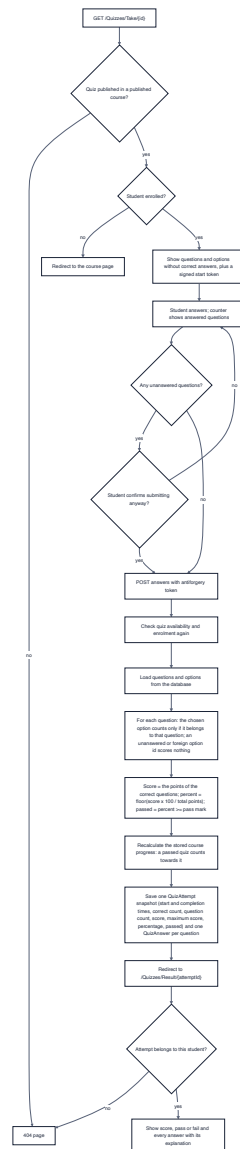
7. Marking a lesson complete and calculating progress

Controllers/ResourcesController.cs (ToggleComplete), Services/LearningResourceService.cs and Services/ProgressService.cs.



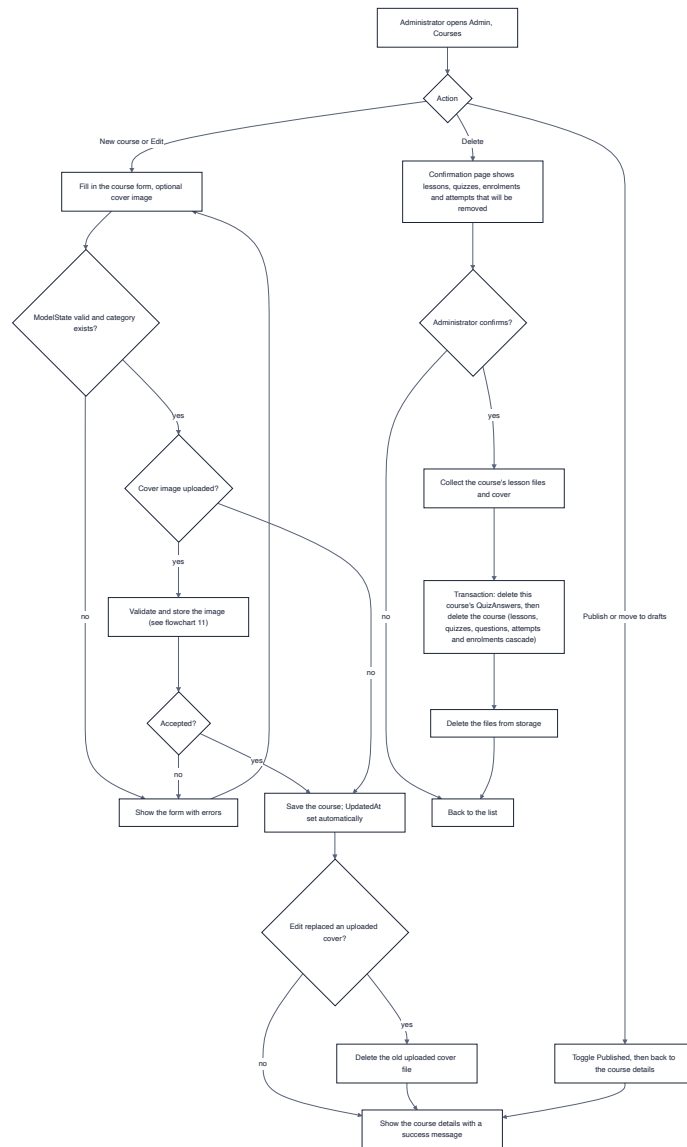
8. Taking and grading a quiz

`Controllers/QuizzesController.cs` (Take, Result), `Services/QuizService.cs` and `Services/QuizGrader.cs`.



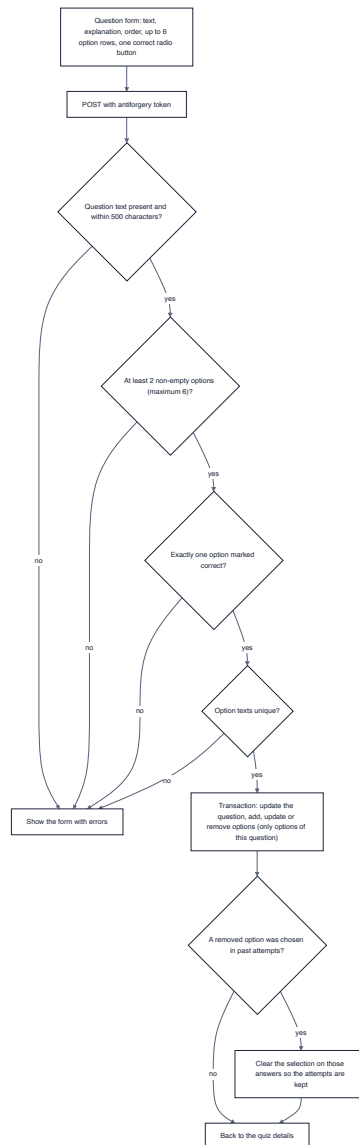
9. Admin: creating, editing and deleting a course

Areas/Admin/Controllers/CoursesController.cs and Services/CourseManagementService.cs .



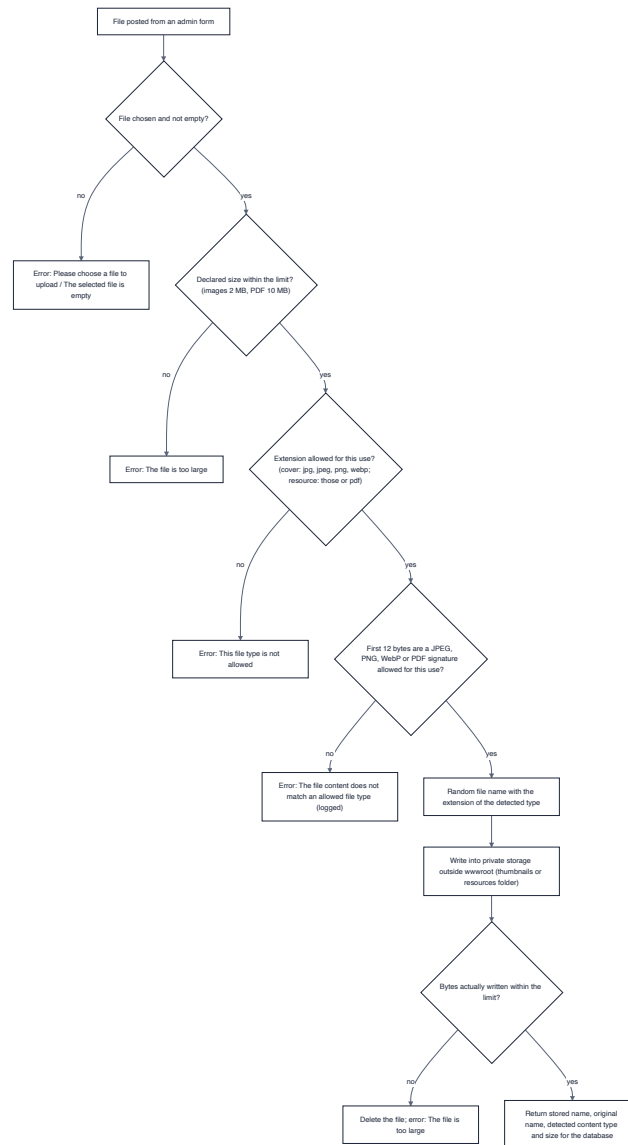
10. Admin: saving a question

`Areas/Admin/Controllers/QuestionsController.cs`, `ViewModels/Admin/AdminLearningViewModels.cs` and `Services/QuestionManagementService.cs`.



11. File upload validation

`Services/Storage/FileStorageService.cs` and `Services/Storage/UploadRules.cs`. For learning resources, `Services/ResourceManagementService.cs` first checks that the file suits the resource type (a PDF for a PDF resource, an image for an image resource).



Architecture

1. Architectural style

LearnHub is a single ASP.NET Core MVC application (.NET 10 LTS) organised in clear, conventional layers. We deliberately avoided Clean Architecture / CQRS / microservices: the goal is a maintainable application whose every part can be explained.

```

Browser (HTML5, CSS3, Bootstrap 5, JavaScript)
  | HTTP(S) requests, forms with anti-forgery tokens
  ▼
Razor views ← ViewModels (strongly typed, DataAnnotations)
  | HTML only: no queries, no business rules
  ▼
Controllers (public/student) + Areas/Admin controllers
  | HTTP concerns only: binding, ModelState, authorisation, redirects
  ▼
Services (business rules, queries) ← enrolment, progress, grading, uploads, reports
  | LINQ, ActionResult
  ▼
EF Core ApplicationDbContext (+ ASP.NET Core Identity)
  | one provider, one migration set
  ▼
SQLite database file (App_Data/learnhub.db locally, /data/learnhub.db on Railway)

```

Layer	Folder	Responsibility
Models	Models/	EF Core entities and enums (the database shape)
Data	Data/	DbContext, Fluent API configurations, migrations, seeding
Services	Services/	Business logic and queries; returns ViewModels or ActionResult
ViewModels	ViewModels/	Strongly typed page and form models with DataAnnotations
Controllers	Controllers/, Areas/Admin/Controllers/	Thin HTTP layer, authorisation attributes
Views	Views/, Areas/Admin/Views/	Razor markup only – no business logic
Infrastructure	Infrastructure/	Cross-cutting: proxy headers, security headers, options, validation attributes
Static assets	wwwroot/	CSS design system, JavaScript, images, vendored libraries

Three rules keep the layering honest:

1. **Controllers stay thin.** A controller binds a ViewModel, checks ModelState, calls one service method and chooses a view or a redirect. Anything that decides an outcome lives in Services/. Admin controllers inherit AdminControllerBase, which contributes only the area, the role requirement and small helpers for status messages.
2. **Entities are never bound directly from forms** (overposting protection). Controllers receive ViewModels; services map them onto entities.
3. **Views never query.** A view renders what its ViewModel already contains.

2. Solution structure

```

LearnHub/
├── .github/workflows/      ci.yml · ef-migrations.yml
├── src/LearnHub/           ASP.NET Core MVC application
├── tests/LearnHub.Tests/   xUnit unit + integration tests (WebApplicationFactory)
├── tests/e2e/              Playwright browser and accessibility tests (mobile / tablet / desktop)
├── scripts/                smoke test for any running instance, CI helpers, set-app-url.sh
├── FirebaseLanding/        static presentation site (Firebase Hosting)
├── docs/                   assignment documentation (this folder)
├── Dockerfile              multi-stage image built by Railway
├── railway.json             Railway build and deploy (health check) settings
├── firebase.json           Firebase Hosting settings for FirebaseLanding/
├── .env.example             every environment variable, placeholders only
├── LearnHub.sln            solution: application + test projects
└── README.md

```

src/ and tests/ are separated because an SDK-style web project compiles every *.cs file below its folder; keeping tests outside the web project folder avoids accidental compilation of test code into the application.

3. HTTP request pipeline

`src/LearnHub/Program.cs` is the composition root. `builder.Services` registers the database, Identity, the application services and the web concerns in four grouped extension methods (`AddLearnHubDatabase` , `AddLearnHubIdentity` , `AddLearnHubApplicationServices` , `AddLearnHubWeb`), then the pipeline is built in this exact order. Order matters: each component sees the request after everything above it has already run.

#	Middleware	Why it is here
1	<code>UsePlatformProxyHeaders()</code>	Replaces the request scheme and the client address from <code>X-Forwarded-Proto</code> / <code>X-Forwarded-For</code> before anything inspects them. Everything below depends on it.
2	<code>UseExceptionHandler("/Error")</code> · <code>UseHsts()</code> (<i>Production only</i>)	Friendly error page instead of a stack trace, and HSTS tells browsers to use HTTPS from now on. Both are skipped in Development so the developer exception page and local HTTP keep working.
3	<code>UseStatusCodePagesWithReExecute("/Error/{0}")</code>	Turns a bare status code (for example 404) into the styled error page by re-executing the request through <code>ExceptionHandler</code> .
4	<code>UseSecurityHeaders()</code>	Adds <code>Content-Security-Policy</code> , <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> and <code>Cross-Origin-Opener-Policy</code> to every response via <code>OnStarting</code> .
5	<code>UseHttpsRedirection()</code>	Sends <code>http://</code> requests to <code>https://</code> . Meaningful only because step 1 has already established that the original request was plain HTTP.
6	<code>UseResponseCompression()</code>	Compresses static text assets (CSS, JavaScript, SVG). HTML is deliberately not compressed because pages carry anti-forgery tokens next to user-controlled text.
7	<code>UseLearnHubStaticFiles()</code>	Serves <code>wwwroot</code> (with a one-year immutable cache for <code>?v=</code> fingerprinted files) and the uploaded course thumbnails. Uploaded lesson documents are not static: <code>ResourcesController</code> streams them after an access check.
8	<code>UseRouting()</code>	Matches the request to an endpoint so the rate limiter and authentication can see endpoint metadata.
9	<code>UseRateLimiter()</code>	Applies the <code>forms</code> policy (fixed window per client IP address) to login, registration, password change and contact posts.
10	<code>UseAuthentication()</code>	Reads the <code>LearnHub.Auth</code> cookie and builds <code>HttpContext.User</code> .
11	<code>UseAuthorization()</code>	Enforces the role requirements declared on controllers.
12	<code>MapLearnHubRoutes()</code>	Maps <code>/health</code> , the <code>areas</code> route for <code>/Admin</code> , friendly top-level <code>/About</code> , <code>/Contact</code> , <code>/Privacy</code> routes and the default <code>{controller}/{action}</code> route.
13	<code>InitializeDatabaseAsync()</code>	Applies migrations, creates the roles, creates the administrator and seeds demo data. Runs once, before the first request.
14	<code>RunAsync()</code>	Starts Kestrel on <code>http://0.0.0.0:\$PORT</code> .

3.1 Why `UsePlatformProxyHeaders` must come first

Railway terminates TLS at its **edge proxy** and forwards plain HTTP into the container, describing the original request in the `X-Forwarded-Proto` and `X-Forwarded-For` headers. ASP.NET Core does **not** read those headers by itself: without the middleware the application would see every request as `http://` and would believe the client address was the proxy.

That has three visible consequences, and all of them are fixed by step 1:

- `UseHttpsRedirection` would treat every request as plain HTTP. Behind a proxy that already guarantees HTTPS, each redirect it issues is offered again as HTTP inside the container, so the browser bounces between the two schemes instead of reaching a page – the classic redirect loop.
- `UseHsts` would never send `Strict-Transport-Security` , so browsers would keep trying plain HTTP.
- In Production the identity and anti-forgery cookies are `Secure` -only, so form posts (login, registration, contact) would fail.

The switch used on Azure App Service, `ASPNETCORE_FORWARDEDHEADERS_ENABLED=true` , was a hosting-startup feature of that platform, **not** part of ASP.NET Core; it does not exist in the framework and cannot be reused here. Hence the explicit middleware. It clears `KnownProxies` and `KnownIPNetworks` because the platform proxy is the only route into the container and arrives from a private-network address, which the default loopback-only allow-list would reject.

3.2 Port binding

Railway chooses the port at run time and passes it in the `PORT` variable. `UsePlatformPort()` reads it and binds `http://0.0.0.0:$PORT` , so the container is reachable from the proxy on every interface. The `PORT=8080` in the image is only a fallback for running the container locally; when `PORT` is absent the usual ASP.NET Core URL settings apply. `/health` (a `DatabaseHealthCheck` over `ApplicationDbContext.Database.CanConnectAsync`) is the endpoint Railway polls.

4. Database design decisions

The full, code-accurate ERD is in [ERD.md](#).

4.1 One provider, one migration set

SQLite is the **only** database provider. `Data/DatabaseServiceCollectionExtensions.cs` calls `UseSqlite` unconditionally, there is a single `ApplicationDbContext` , and there is a single migrations folder:

Concern	Location
Context	<code>Data/ApplicationDbContext.cs</code> (Fluent API split into <code>Data/Configurations/</code>)
Migrations	<code>Data/Migrations/</code> – currently <code>20260914100832_InitialCreate</code> , creating 18 tables
Design-time factory	<code>Data/DesignTimeDbContextFactory.cs</code>
Registration	<code>Data/DatabaseServiceCollectionExtensions.AddLearnHubDatabase</code>

The initial migration creates the eleven application tables (`Categories`, `Courses`, `LearningResources`, `Enrollments`, `ResourceCompletions`, `Quizzes`, `Questions`, `AnswerOptions`, `QuizAttempts`, `QuizAnswers`, `ContactMessages`) together with the seven ASP.NET Core Identity tables (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, `AspNetRoleClaims`, `AspNetUserLogins`, `AspNetUserTokens`).

An earlier design carried two providers (SQLite for development, SQL Server for production) with two contexts and two migration folders. EF Core migrations are provider-specific, so that meant generating and maintaining every schema change twice. Removing it left one model, one history and one place where schema drift can hide.

4.2 Connection-string resolution

Resolution happens in `SqliteConnectionStrings` and follows one rule:

Priority	Source	Typical value
1	Environment variable <code>DATABASE_CONNECTION_STRING</code>	<code>Data Source=/data/learnhub.db</code> (Railway volume)
2	<code>ConnectionStrings:DefaultConnection</code>	<code>Data Source=App_Data/learnhub.db</code> (local default)

A relative `Data Source` path is made absolute against the **content root** – not the current working directory – and its folder is created if it does not exist. The database location therefore does not depend on how the process was started, and an empty volume works on the first start. In-memory SQLite (`:memory:`) is recognised and left untouched, which is how the integration tests isolate themselves.

`DesignTimeDbContextFactory` resolves the connection string the same way for the `dotnet ef` tools, so a migration is always generated against the schema the running application uses.

4.3 Migrations and seeding at start-up

`Data/Seed/DatabaseInitializer.cs` runs once before the first request:

- if `Database:ApplyMigrationsOnStartup` (default `true`) is set, apply every pending migration; if it is `false` and migrations are pending, log a clear warning and skip seeding instead of failing;
- create the `Admin` and `Student` roles;
- create the first administrator from `Seed:AdminEmail` / `Seed:AdminPassword`, only when no user with that email exists – roles and passwords of existing accounts are never re-applied;
- seed the demonstration catalogue when `Seed:DemoData` is true and the database is still empty.

Seeding one empty database produces 8 categories, 12 courses (11 published, 1 draft), 64 learning resources, 11 quizzes, 54 questions, 216 answer options and 7 learners. Those accounts are DEMO ONLY and exist so dashboards and progress pages look realistic.

4.4 Delete behaviour (intentional)

Relationship	Behaviour	Reason
Category → Courses	Restrict	Deleting a category must never silently delete courses. The admin must move or delete courses first.
Course → Resources, Quizzes, Enrolments	Cascade	A course owns its content; the delete page shows the full impact before confirmation.
Quiz → Questions → AnswerOptions	Cascade	Questions cannot exist without their quiz.
Quiz → QuizAttempts → QuizAnswers	Cascade	Attempts are meaningless without the quiz.
QuizAnswer → Question / AnswerOption	Restrict	Keeps the schema portable and deterministic. Removing a question or an answer option is an explicit service operation that deletes or clears dependent answers first, in one transaction (<code>Data/TransactionExtensions.cs</code>).
User → Enrolments, QuizAttempts, ResourceCompletions	Cascade	Removing an account removes that person's learning records.
LearningResource → ResourceCompletions	Cascade	Completion records belong to the resource.

4.5 Derived rather than duplicated state

- Course progress** is calculated, not stored: completed resources + passed published quizzes divided by total resources + published quizzes. Adding a new lesson therefore correctly lowers progress, and there is no stale `ProgressPercent` column.
- Quiz attempts** store a score snapshot (`CorrectCount`, `QuestionCount`, `ScorePercent`, `Passed`) because the quiz may be edited after the attempt; history must stay truthful.
- Deactivated users** use Identity's built-in lockout (`LockoutEnd = DateTimeOffset.MaxValue`) rather than a duplicate `IsActive` flag.

- **Timestamps** are stored as UTC through a value converter (`Data/UtcDateTimeConverter.cs`) and rendered in the visitor's local time by a small progressive-enhancement script.

5. Roles and authorisation

Role	How obtained	Scope
Guest	Not signed in	Public pages, published courses, preview resources
Student	Automatically on registration	Dashboard, enrolment, resources of enrolled courses, quizzes, profile
Admin	Seeded from configuration or promoted by another admin	Everything in <code>/Admin</code> plus preview of all content

Authorisation is declarative: `[Authorize(Roles = AppRoles.Student)]` on student controllers and a shared `AdminControllerBase` with `[Area("Admin")]` and `[Authorize(Roles = AppRoles.Admin)]` (the literal role `"Admin"`) that every admin controller inherits, so a new admin controller cannot be added without protection. Ownership checks (IDOR) are done in services (for example a quiz result is only returned when `attempt.UserId == currentUserId`).

6. Routes

Public and student

Route	Controller.Action	Access
<code>/</code>	Home.Index	Everyone
<code>/About</code> , <code>/Contact</code> , <code>/Privacy</code>	Home.About / Contact / Privacy	Everyone
<code>/Courses?q=&categoryId=&difficulty=&sort=&page=</code>	Courses.Index	Everyone
<code>/Courses/Details/{id}</code>	Courses.Details	Everyone (published courses)
<code>POST /Courses/Enroll/{id}</code> · <code>POST /Courses/Leave/{id}</code>	Courses.Enroll / Leave	Student
<code>/Resources/Details/{id}</code>	Resources.Details	Preview: everyone · otherwise enrolled student or admin
<code>/Resources/Open/{id}</code>	Resources.Open (file stream)	Same rule as above
<code>POST /Resources/ToggleComplete/{id}</code>	Resources.ToggleComplete	Enrolled student
<code>/Quizzes/Take/{id}</code> · <code>POST /Quizzes/Take/{id}</code>	Quizzes.Take	Enrolled student
<code>/Quizzes/Result/{attemptId}</code> · <code>/Quizzes/History</code>	Quizzes.Result / History	Owner student
<code>/Student/Dashboard</code> · <code>/Student/MyCourses</code>	Student.Dashboard / MyCourses	Student
<code>/Profile</code> · <code>/Profile/ChangePassword</code>	Profile.Index / ChangePassword	Signed-in user
<code>/Account/Register</code> · <code>/Account/Login</code> · <code>POST /Account/Logout</code> · <code>/Account/AccessDenied</code>	Account.*	Everyone
<code>/Error/{statusCode}</code> · <code>/Error</code>	Error.*	Everyone
<code>/health</code>	Health check (database connectivity)	Everyone

Admin area (`[Authorize(Roles = "Admin")]`)

Route	Purpose
<code>/Admin</code>	Dashboard
<code>/Admin/Courses</code> (+ <code>/Create</code> , <code>/Edit/{id}</code> , <code>/Details/{id}</code> , <code>/Delete/{id}</code>)	Course CRUD
<code>/Admin/Categories</code> (+ <code>Create/Edit/Delete</code>)	Category CRUD
<code>/Admin/Resources</code> (+ <code>Create/Edit/Delete</code>)	Learning resource CRUD
<code>/Admin/Quizzes</code> (+ <code>Create/Edit/Details/Delete</code>)	Quiz CRUD
<code>/Admin/Questions/Create?quizId=</code> (+ <code>Edit/Delete</code>)	Question and answer option CRUD
<code>/Admin/QuizAttempts</code> (+ <code>Details/Delete</code>)	Quiz results
<code>/Admin/Enrollments</code> (+ <code>Create/Delete</code>)	Enrolment management
<code>/Admin/Users</code> (+ <code>Details</code> , role change , deactivate , <code>Delete</code>)	User management
<code>/Admin/Messages</code> (+ <code>Details/Delete</code>)	Contact message inbox

7. Configuration and secrets

Options classes are bound from configuration sections, and every one of them can be overridden by an environment variable: ASP.NET Core maps `Seed__AdminPassword` onto `Seed:AdminPassword` using a double underscore for the section separator, so no code change is needed to move between environments.

Options class	Section	Purpose
<code>DatabaseOptions</code>	<code>Database</code>	<code>ApplyMigrationsOnStartup</code> , and the name of the connection-string variable
<code>SeedOptions</code>	<code>Seed</code>	Demo data switch, administrator and demo-student accounts
<code>StorageOptions</code>	<code>Storage</code>	Private folder for uploaded files and thumbnails
<code>RateLimitingOptions</code>	<code>RateLimiting</code>	Permit limit and window for the form rate limiter
<code>SiteOptions</code>	<code>Site</code>	Repository and presentation-site links shown in the footer and on the About page

Two values sit outside those sections: `DATABASE_CONNECTION_STRING` (see 4.2) and `DataProtection:KeysPath`, which persists the Data Protection key ring that encrypts the sign-in and anti-forgery cookies.

`.env.example` lists every variable with a placeholder value and a comment, and is the only file of its kind in Git. `.env` itself, `App_Data/`, `*.db` and local user secrets are git-ignored, and the Docker build excludes them too (`.dockerignore`), so neither the image nor the repository can contain a real credential. In Development the demo passwords are generated on first start into `App_Data/demo-credentials.json`, which is also git-ignored; in Production they must come from `Seed__AdminPassword` / `Seed__DemoStudentPassword`.

8. Security design

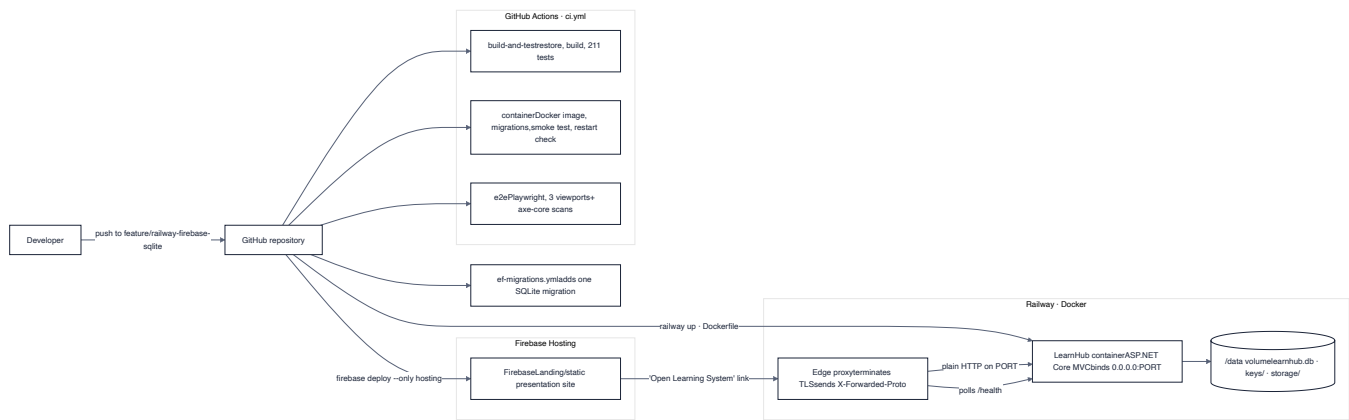
Threat	Control
SQL injection	Only EF Core LINQ (parameterised). Search uses <code>EF.Functions.Like</code> with escaped wildcards.
XSS	Razor HTML-encodes all output. Lesson text is encoded first, then a fixed allow-list of tags is applied (<code>LessonContentRenderer</code>). Strict Content-Security-Policy without inline scripts.
CSRF	Global <code>AutoValidateAntiforgeryTokenAttribute</code> ; all state changes are POST forms with tokens.
IDOR	Services check ownership/enrolment for attempts, resources and files. Resource files are streamed by a controller after an access check, never exposed as static files.
Overposting	Forms bind to ViewModels only.
File uploads	Extension allow-list (<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.webp</code> , <code>.pdf</code>), size limit (2 MB image / 10 MB document), magic-byte signature check, random file names, storage outside <code>wwwroot</code> , SVG/HTML never accepted.
Video embeds	URLs parsed with strict host/ID rules and rebuilt as <code>youtube-nocookie.com</code> / <code>player.vimeo.com</code> embed URLs; CSP <code>frame-src</code> allow-list.
Brute force	Identity lockout (5 failed attempts, 15-minute lockout) and a fixed-window rate limit (10 per minute per IP address) on login, registration, password change and contact; honeypot field on the contact form.
Session security	HttpOnly, SameSite=Lax cookies, <code>Secure</code> -only outside Development; security stamp re-validated every 5 minutes so role changes and deactivation take effect; 8-hour sliding expiry.
Secrets	Nothing sensitive is committed: the administrator password is a Railway service variable (or user-secrets locally), and <code>.env</code> , <code>App_Data/</code> and connection strings never reach Git.
Information disclosure	Developer exception page only in Development; friendly error pages elsewhere; security headers (<code>X-Content-Type-Options</code> , <code>Referrer-Policy</code> , <code>X-Frame-Options</code> , <code>Permissions-Policy</code> , HSTS in Production).

The full threat model and residual risks are in [SECURITY.md](#).

9. Front-end design

- Bootstrap 5.3 provides the grid, utilities and accessible components.
- `wwwroot/css/site.css` defines the LearnHub design system (colour tokens, typography, spacing, radii, shadows) and component styles (navbar, hero, course cards, dashboard, tables, forms, empty states). `admin.css` adds the admin shell.
- JavaScript is progressive enhancement only: validation styling and ARIA wiring, password strength meter, character counters, image preview, quiz answered-count and unanswered warning, local-time formatting, confirm dialogs. Every feature works without JavaScript because validation and processing are repeated on the server.
- Libraries are vendored in `wwwroot/lib` (no runtime CDN), which keeps the CSP strict.

10. Deployment architecture



- **Railway** builds `Dockerfile` (multi-stage: .NET 10 SDK build stage → ASP.NET Core 10 runtime stage), runs the container with the variables from `.env.example` and polls `/health`. `railway.json` holds the builder, the health-check path and timeout, and the restart policy.
- A **Railway volume mounted at `/data`** carries all persistent state: the SQLite file, the Data Protection key ring and admin uploads. Without it, a redeploy would start from an empty database and invalidate every signed-in session. This is the deliberate answer to "SQLite on an ephemeral filesystem", and the CI `container` job proves it by restarting the container and asserting that migrations are not re-applied.
- **Firebase Hosting** serves the static presentation site in `FirebaseLanding/` (a landing page, a 404 page, the design system, the Overpass webfont and eight real screenshots). Firebase cannot execute ASP.NET Core, so this part only presents the project and links to the Railway application through its "Open Learning System" call to action. It is live at <https://learnhub-wapp.web.app>.

Step-by-step commands are in [DEPLOYMENT.md](#).

10.1 Why not Azure

The assignment for this project does not permit it, and the replacement stack is a better fit anyway:

- **No vendor lock-in.** The application is an ordinary container built from a public `Dockerfile`; it can run on any container host, or locally with `docker run`. Nothing in the code names a cloud provider.
- **One database engine everywhere.** SQLite needs no server, no credentials and no network hop, so local development, CI and production run the same provider and the same migrations. The trade-off – a single writer and no horizontal scaling – is acceptable for a single-instance teaching application, and it is covered by the volume described above.
- **Static hosting stays static.** Firebase Hosting serves the presentation site from a CDN; the dynamic application lives where it can actually run.
- **Fewer moving parts to explain.** Three services, one image, one environment-variable file – which matters when the deployment has to be defended in a viva.

11. Cloud-based development workflow

Development machines with little free disk space do not need the .NET SDK locally: GitHub Actions restores, builds, runs the whole test suite, builds and exercises the container the way Railway does, runs the browser and accessibility tests, and generates EF Core migrations (`ef-migrations.yml`). With the SDK installed you can use the normal local commands documented in the README and in [DEPLOYMENT.md](#).

Database Design

LearnHub uses **Entity Framework Core 10 with SQLite**, designed code first: the C# entity classes are the source of truth, and migrations generate the schema. There is a single provider, so there is one migration set and one database file — no provider-specific code paths anywhere in the application.

For the entity relationship diagram see [ERD.md](#). For how the database file is persisted in production see [DEPLOYMENT.md](#).

1. Why SQLite

Reason	Detail
Simple to deploy	One file, no separate database service to provision, secure or pay for.
Simple to explain	A single <code>learnhub.db</code> file holds the whole system, which is easy to demonstrate and to describe in the report.
No vendor lock-in	The assignment forbids Azure, and SQLite runs anywhere — a laptop, a container or a Railway volume.
Sufficient for the scale	A single-instance academic deployment has one writer at a time; SQLite handles that comfortably.
Identical in test and production	The integration tests run the real application against SQLite (in memory), so the tested engine is the deployed engine.

SQLite has one write lock for the whole database. That is a deliberate, acceptable trade-off for this project and is recorded as a known limitation — a multi-instance deployment would need a client/server database.

2. Where the database lives

The connection string is resolved in `src/LearnHub/Data/DatabaseServiceCollectionExtensions.cs`:

1. The environment variable `DATABASE_CONNECTION_STRING` wins when it is set. Railway supplies it.
2. Otherwise `ConnectionStrings:DefaultConnection` from `appsettings.json` is used, which defaults to `Data Source=App_Data/learnhub.db`.

A relative path is resolved against the content root, and the containing folder is created automatically, so the database location never depends on the current working directory.

Environment	Connection string	Notes
Local development	<code>Data Source=App_Data/learnhub.db</code>	Created on first run. Delete <code>App_Data/</code> to start again.
Integration tests	<code>Data Source=.;Mode=Memory;Cache=Shared</code>	One private in-memory database per test factory, so tests never touch a developer's data.
Railway (production)	<code>Data Source=/data/learnhub.db</code>	<code>/data</code> is the mounted volume, so the file survives a redeploy.

`Database:ApplyMigrationsOnStartup` (default `true`) applies pending migrations during start-up. On the single-instance deployment used here that is simpler and more reliable than a separate migration step. If it is set to `false` and migrations are pending, the application logs a warning and skips seeding rather than running against an out-of-date schema.

3. Schema at a glance

One migration — `src/LearnHub/Data/Migrations/20260914100832_InitialCreate.cs` — creates **18 tables**: eleven application tables and the seven tables ASP.NET Core Identity needs.

Application tables

Table	Purpose	Key columns
<code>Categories</code>	Subject areas that group courses	<code>Id</code> , <code>Name</code> (unique), <code>Description</code> , <code>IconName</code>
<code>Courses</code>	The catalogue	<code>Id</code> , <code>CategoryId</code> → <code>Categories</code> , <code>Title</code> , <code>ShortDescription</code> , <code>Description</code> , <code>LearningOutcomes</code> , <code>InstructorName</code> , <code>Difficulty</code> , <code>DurationMinutes</code> , <code>ThumbnailPath</code> , <code>IsPublished</code>
<code>LearningResources</code>	The lessons inside a course	<code>Id</code> , <code>CourseId</code> → <code>Courses</code> , <code>Title</code> , <code>Summary</code> , <code>Type</code> , <code>Body</code> , <code>Solution</code> , <code>ExternalUrl</code> , <code>FilePath</code> / <code>FileName</code> / <code>FileContentType</code> / <code>FileSizeBytes</code> , <code>EstimatedMinutes</code> , <code>SortOrder</code> , <code>IsPreview</code> , <code>IsPublished</code>
<code>Enrollments</code>	A student's membership of a course	<code>Id</code> , <code>UserId</code> → <code>AspNetUsers</code> , <code>CourseId</code> → <code>Courses</code> , <code>EnrolledAt</code> , <code>LastAccessedAt</code> , <code>CompletionPercentage</code> , <code>CompletedAt</code>
<code>ResourceCompletions</code>	Which lessons a student has finished	<code>Id</code> , <code>UserId</code> → <code>AspNetUsers</code> , <code>LearningResourceId</code> → <code>LearningResources</code> , <code>CompletedAt</code>

Table	Purpose	Key columns
Quizzes	A quiz attached to a course	Id, CourseId → Courses, Title, Description, PassMarkPercent, IsPublished
Questions	A quiz question	Id, QuizId → Quizzes, Text, Explanation, Points, SortOrder
AnswerOptions	The selectable answers to a question	Id, QuestionId → Questions, Text, IsCorrect, SortOrder
QuizAttempts	One submitted attempt	Id, QuizId → Quizzes, UserId →AspNetUsers, StartedAt, CompletedAt, CorrectCount, QuestionCount, Score, MaxScore, ScorePercent, Passed
QuizAnswers	The answer chosen for one question in one attempt	Id, QuizAttemptId → QuizAttempts, QuestionId → Questions, SelectedOptionId → AnswerOptions (nullable), IsCorrect
ContactMessages	Enquiries from the public contact form	Id, Name, Email, Subject, Message, IsRead, CreatedAt

Identity tables

AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims, AspNetRoleClaims, AspNetUserLogins, AspNetUserTokens.

ApplicationUser extends IdentityUser with FullName, Bio and CreatedAt. Passwords are hashed by ASP.NET Core Identity and are never stored in plain text.

Quiz attempt history is a snapshot. QuizAttempt stores CorrectCount, QuestionCount, Score, MaxScore and ScorePercent as they were at submission time, so editing a quiz later does not rewrite a student's past results.

4. Relationships

Relationship	Cardinality	Delete rule
Category → Course	one-to-many	Restrict — a category holding courses cannot be deleted
Course → LearningResource	one-to-many	Cascade
Course → Quiz	one-to-many	Cascade
Course → Enrollment	one-to-many	Cascade
ApplicationUser → Enrollment	one-to-many	Cascade
ApplicationUser → ResourceCompletion	one-to-many	Cascade
LearningResource → ResourceCompletion	one-to-many	Cascade
Quiz → Question	one-to-many	Cascade
Question → AnswerOption	one-to-many	Cascade
Quiz → QuizAttempt	one-to-many	Cascade
ApplicationUser → QuizAttempt	one-to-many	Cascade
QuizAttempt → QuizAnswer	one-to-many	Cascade
Question → QuizAnswer	one-to-many	Restrict
AnswerOption → QuizAnswer	one-to-many	Restrict

The two **Restrict** rules on QuizAnswer are deliberate: they stop an administrator editing a quiz from silently destroying a student's answer history. The management services clear or delete dependent answers explicitly before removing a question or an option, so the administrator still gets a clear message instead of a database error.

5. Constraints that keep the data honest

Unique indexes

Index	Table	Prevents
Name	Categories	Two categories with the same name
(UserId, CourseId)	Enrollments	Duplicate enrolment — a student cannot enrol twice in one course
(UserId, LearningResourceId)	ResourceCompletions	The same lesson being marked complete twice
(QuizAttemptId, QuestionId)	QuizAnswers	Two answers to the same question in one attempt

The application checks for duplicates first and shows a friendly message; the unique index is the guarantee that the rule holds even if two requests race.

Check constraints

Constraint	Table	Rule
CK_Courses_DurationMinutes	Courses	DurationMinutes > 0
CK_LearningResources_SortOrder	LearningResources	SortOrder >= 0
CK_Enrollments_CompletionPercentage	Enrollments	CompletionPercentage BETWEEN 0 AND 100
CK_Quizzes_PassMarkPercent	Quizzes	PassMarkPercent BETWEEN 0 AND 100
CK_Questions_Points	Questions	Points BETWEEN 1 AND 100
CK_QuizAttempts_ScorePercent	QuizAttempts	ScorePercent BETWEEN 0 AND 100
CK_QuizAttempts_CorrectCount	QuizAttempts	CorrectCount BETWEEN 0 AND QuestionCount
CK_QuizAttempts_Score	QuizAttempts	Score BETWEEN 0 AND MaxScore

Required columns are enforced with `IsRequired()`, string lengths come from `src/LearnHub/Models/FieldLengths.cs`, and genuinely optional values (`Bio`, `ThumbnailPath`, `CompletedAt`, `SelectedOptionId`) are nullable.

Enumerations

`Difficulty` and `ResourceType` are stored as **strings** (`HasConversion<string>()`), so the database is readable and a reordering of the C# enum cannot silently change the meaning of existing rows.

Indexes for the pages that are actually used

`Courses(Title)`, `Courses(IsPublished, CategoryId)`, `LearningResources(CourseId, SortOrder)`, `LearningResources(CourseId, IsPublished)`, `Enrollments(CourseId, CompletionPercentage)`, `Questions(QuizId, SortOrder)`, `AnswerOptions(QuestionId, SortOrder)`, `QuizAttempts(UserId, QuizId)`, `QuizAttempts(CompletedAt)`.

Each one backs a real query — catalogue search and filtering, the lesson list, progress calculation, the quiz player and the admin dashboards.

6. Seeding

`src/LearnHub/Data/Seed/DatabaseInitializer.cs` runs once at start-up, after migrations:

1. Creates the `Admin` and `Student` roles.
2. Creates the administrator from `Seed:AdminEmail` / `Seed:AdminPassword` if that account does not exist. An existing account is never modified — an administrator may have changed their own password deliberately.
3. Seeds the demonstration catalogue via `DemoDataSeeder` when `Seed:DemoData` is true.

The seeded catalogue is defined in `Data/Seed/DemoCatalog*.cs` and contains **8 categories, 12 courses (11 published and 1 draft), 64 learning resources, 11 quizzes, 54 questions, 216 answer options and 7 learners**. Extra learners and staggered enrolment dates make the dashboards and progress pages show realistic figures rather than empty states.

Demonstration accounts — DEMO ONLY

Role	Email
Administrator	<code>admin@learnhub.local</code>
Student	<code>demo.student@example.com</code>

No password is ever committed. In Development, `DevelopmentSeedPasswords` generates strong random passwords on the first run and writes them to `App_Data/demo-credentials.json`, which is git-ignored. In Production the passwords must come from the `Seed__AdminPassword` and `Seed__DemoStudentPassword` environment variables; with no password set the account still exists (so seeded data stays realistic) but cannot be signed into, and the log explains what to configure. These accounts exist for demonstration and must not be used for anything real.

7. Working with the database

```
# Restore the EF Core tools pinned in .config/dotnet-tools.json
dotnet tool restore

# Create a migration after changing a model
dotnet ef migrations add <Name> --project src/LearnHub --output-dir Data/Migrations

# Apply migrations by hand (the application also does this at start-up)
dotnet ef database update --project src/LearnHub

# Inspect the generated SQL
dotnet ef migrations script --project src/LearnHub --output schema.sql
```

The `EF Core migration (cloud)` GitHub workflow (`.github/workflows/ef-migrations.yml`) can add a migration from the browser, without the SDK installed.

`src/LearnHub/Data/DesignTimeDbContextFactory.cs` supplies the context to the `dotnet ef` tools so migrations can be created without starting the web application.

Backups

The whole database is one file. Copy `/data/learnhub.db` from the Railway volume to back it up, and copy it back to restore. Because migrations are applied at start-up, restoring an older file and restarting the service brings the schema up to date automatically.

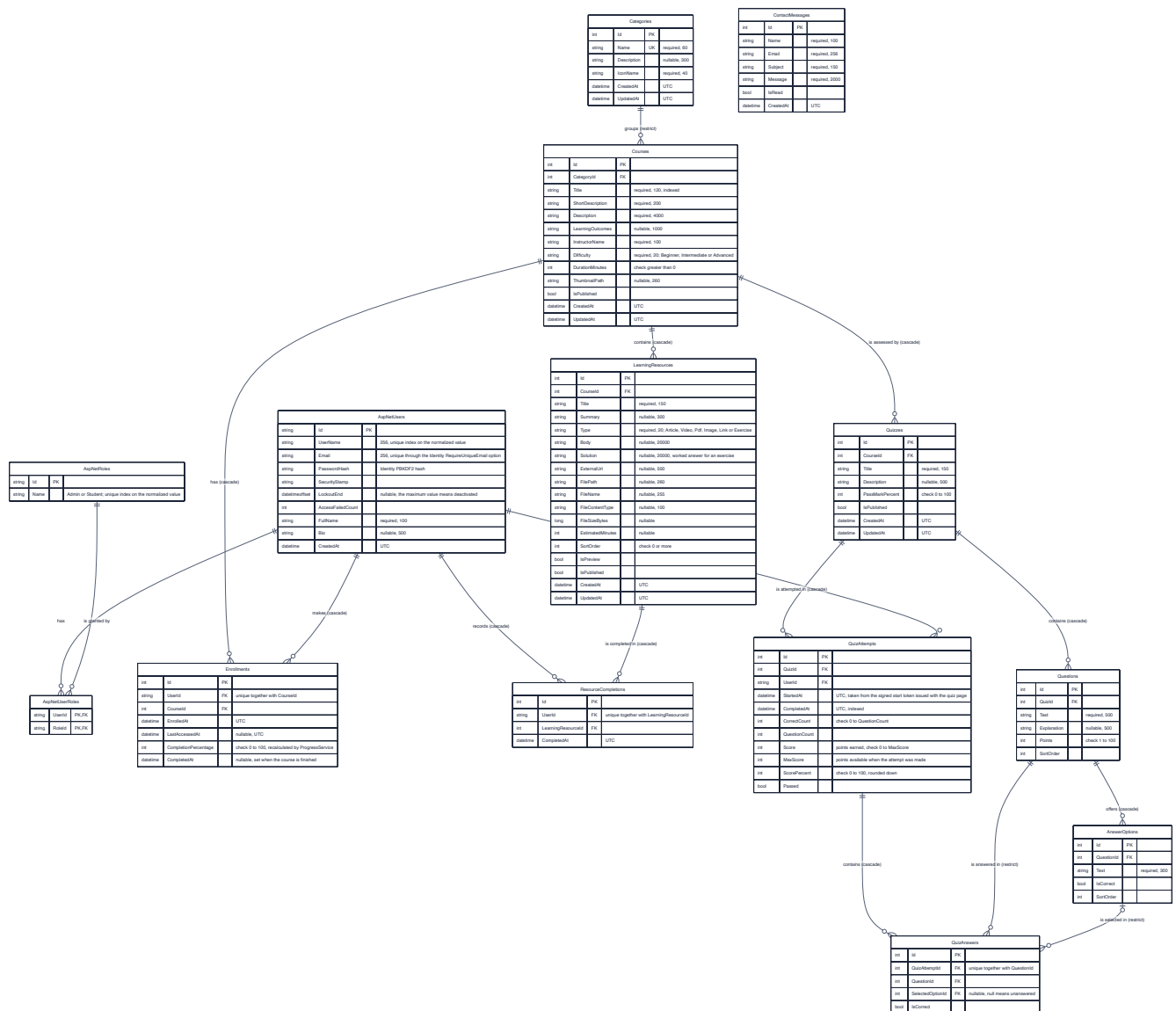
8. Verifying the schema

- `tests/LearnHub.Tests/Data/DatabaseModelTests.cs` asserts the constraints, indexes and delete rules described above, so the model cannot drift from this document without a test failing.
- `tests/LearnHub.Tests/Infrastructure/TestDatabase.cs` creates a fresh in-memory SQLite database per factory and applies the real migrations, so the migrations themselves are exercised by every integration test.

Entity Relationship Diagram

This diagram is generated from the EF Core model in `src/LearnHub/Models` and the Fluent API configuration in `src/LearnHub/Data/Configurations`. Table names are EF Core defaults, which match the `DbSet` names in `src/LearnHub/Data/ApplicationDbContext.cs`. There is one provider (SQLite), one context and one migration: `Data/Migrations/20260914100832_InitialCreate.cs` creates all **18 tables** — the eleven application tables below plus the seven ASP.NET Core Identity tables.

Notation: **PK** primary key, **FK** foreign key, **UK** unique. Column sizes are the maximum lengths from `src/LearnHub/Models/FieldLengths.cs`. Enum columns are stored as strings.



`ContactMessages` has no relationships: messages are stored as sent from the public contact form. ASP.NET Core Identity also creates `AspNetUserClaims`, `AspNetUserLogins`, `AspNetUserTokens` and `AspNetRoleClaims`; they exist in the schema but LearnHub does not use them, so they are left out of the diagram. The same applies to Identity's own plumbing columns, which the migration does create: `NormalizedUserName`, `NormalizedEmail`, `EmailConfirmed`, `ConcurrencyStamp`, `PhoneNumber`, `PhoneNumberConfirmed`, `TwoFactorEnabled` and `LockoutEnabled` on `AspNetUsers`, and `NormalizedName` and `ConcurrencyStamp` on `AspNetRoles`.

Relationships and delete behaviour

Parent → child	Cardinality	On delete	Why
Categories → Courses	1 : many	Restrict	Deleting a category must never delete courses silently. The admin delete page explains that the category still has courses.
Courses → LearningResources	1 : many	Cascade	A course owns its lessons.
Courses → Quizzes	1 : many	Cascade	A course owns its quizzes.

Parent → child	Cardinality	On delete	Why
Courses → Enrollments	1 : many	Cascade	Enrolments are meaningless without the course.
AspNetUsers → Enrollments, ResourceCompletions, QuizAttempts	1 : many	Cascade	Deleting an account removes that person's learning records.
LearningResources → ResourceCompletions	1 : many	Cascade	Completion records belong to the lesson.
Quizzes → Questions → AnswerOptions	1 : many	Cascade	Questions and options cannot exist without their quiz.
Quizzes → QuizAttempts → QuizAnswers	1 : many	Cascade	Attempts belong to the quiz.
Questions → QuizAnswers	1 : many	Restrict	A second cascade path (Quiz → Question → QuizAnswer beside Quiz → QuizAttempt → QuizAnswer) would let a quiz edit delete a student's answer history silently, so the link restricts instead.
AnswerOptions → QuizAnswers	0..1 : many	Restrict	Same reason; an answer may also be empty (unanswered question).

Because of the two Restrict rules, `CourseManagementService`, `QuizManagementService` and `QuestionManagementService` delete the affected `QuizAnswers` first with `ExecuteDeleteAsync`, then the parent, inside one transaction created through the provider's execution strategy (`Data/TransactionExtensions.cs`).

Keys, indexes and constraints

Table	Unique	Other indexes	Check constraints
AspNetUsers	NormalizedUserName (<code>UserNameIndex</code>); email uniqueness enforced by Identity (<code>RequireUniqueEmail</code>)	NormalizedEmail (<code>EmailIndex</code>)	–
AspNetRoles	NormalizedName (<code>RoleNameIndex</code>)	–	–
Categories	Name	–	–
Courses	–	CategoryId; Title; (IsPublished, CategoryId)	<code>CK_Courses_DurationMinutes</code> : DurationMinutes > 0
LearningResources	–	(CourseId, SortOrder); (CourseId, IsPublished)	<code>CK_LearningResources_SortOrder</code> : SortOrder >= 0
Enrollments	(UserId, CourseId)	(CourseId, CompletionPercentage)	<code>CK_Enrollments_CompletionPercentage</code> : 0–100
ResourceCompletions	(UserId, LearningResourceId)	LearningResourceId	–
Quizzes	–	CourseId	<code>CK_Quizzes_PassMarkPercent</code> : 0–100
Questions	–	(QuizId, SortOrder)	<code>CK_Questions_Points</code> : 1–100
AnswerOptions	–	(QuestionId, SortOrder)	–
QuizAttempts	–	(UserId, QuizId); CompletedAt; QuizId	<code>CK_QuizAttempts_ScorePercent</code> : 0–100; <code>CK_QuizAttempts_CorrectCount</code> : 0 ≤ CorrectCount ≤ QuestionCount; <code>CK_QuizAttempts_Score</code> : 0 ≤ Score ≤ MaxScore
QuizAnswers	(QuizAttemptId, QuestionId)	QuestionId; SelectedOptionId	–
ContactMessages	–	(IsRead, CreatedAt)	–

The unique indexes are the last line of defence against double-submitted forms: for example, if two requests enrol the same student at the same moment, the second insert fails on `(UserId, CourseId)` and `EnrollmentService` turns the `DbUpdateException` into the message "You are already enrolled in this course."

Design notes

- **Timestamps.** Entities implementing `IHasTimestamps` get `CreatedAt` and `UpdatedAt` set in `ApplicationDbContext.SaveChanges`. A `UtcDateTimeConverter` value converter stores every `DateTime` as UTC, so the stored values do not depend on the machine's time zone; the browser converts them for display.
- **Progress is stored, then kept honest.** `ProgressService` recalculates `Enrollments.CompletionPercentage` and `Enrollments.CompletedAt` as (published lessons completed + passed published quizzes that have questions) ÷ (published lessons + published quizzes that have questions). Caching it keeps the dashboard, the student pages and the admin lists to one query, and the recalculation runs whenever a lesson is completed, a quiz is passed, an enrolment changes or course content is added or removed — so adding a lesson correctly lowers everyone's percentage.
- **Score snapshots.** `QuizAttempts` stores `Score`, `MaxScore`, `CorrectCount`, `QuestionCount`, `ScorePercent` and `Passed`, and each `QuizAnswers` row stores `IsCorrect`, so results stay truthful even if the quiz, its points or its options are edited later. `Questions.Points` weights the questions, and the percentage is rounded down.
- **Deactivation without a flag.** A deactivated user has `LockoutEnd` set to the maximum date. Identity already refuses sign-in for locked-out users, so no extra `IsActive` column can drift out of sync.
- **Leaving a course.** Removing an enrolment keeps completion records and quiz attempts, so progress returns if the student enrolls again.

- **Files.** Uploaded files are stored outside `wwwroot`; the database keeps only the generated file name, the original file name, the detected content type and the size.

Security Review

Scope	The LearnHub application (<code>src/LearnHub</code>), its configuration, the container (<code>Dockerfile</code> , <code>railway.json</code>), the CI workflows in <code>.github/workflows</code> and the repository contents
Method	Manual code review against the OWASP Top 10:2025 and the assignment's security requirements (SEC-01 to SEC-11), backed by automated tests that attack the running application
Date	14 September 2026
Result	No open high or medium findings. Residual risks and hardening ideas are listed in section 4.

1. Summary of controls

Area	Control	Where
Authentication	ASP.NET Core Identity with PBKDF2 password hashing; 8–100 characters with upper-case, lower-case and digit; unique email; logout after 5 failures for 15 minutes; generic error for wrong passwords; one message for logout and deactivation	<code>Infrastructure/ServiceCollectionExtensions.cs</code> , <code>Controllers/AccountController.cs</code> , <code>ViewModels/Account/AccountViewModels.cs</code>
Sessions	Cookies <code>HttpOnly</code> , <code>SameSite=Lax</code> , <code>Secure</code> outside Development; 8-hour sliding expiry; security stamp re-checked every 5 minutes so role changes and deactivation apply quickly	<code>Infrastructure/ServiceCollectionExtensions.cs</code>
Authorisation	<code>[Authorize(Roles = "Student")]</code> on student controllers; every admin controller inherits <code>AdminControllerBase</code> (<code>[Area("Admin")]</code>) , <code>[Authorize(Roles = "Admin")]</code> , so a new admin page cannot be added unprotected	<code>Controllers/</code> , <code>Areas/Admin/Controllers/AdminControllerBase.cs</code>
Access to data	Services check enrolment or ownership before returning lessons, files, quizzes and results (IDOR)	<code>Services/LearningResourceService.cs</code> , <code>Services/QuizService.cs</code> , <code>Controllers/ResourcesController.cs</code>
CSRF	Global <code>AutoValidateAntiforgeryTokenAttribute</code> ; every state change is a POST form with a token	<code>Infrastructure/ServiceCollectionExtensions.cs</code>
Injection	Only EF Core LINQ (parameterised SQL); search terms go through <code>EF.Functions.Like</code> with <code>%</code> , <code>_</code> and <code>[</code> escaped	<code>Services/SearchPattern.cs</code> , <code>Services/CourseCatalogService.cs</code>
XSS	Razor encodes all output; lesson text is encoded first, then a fixed allow-list of formatting is applied; strict Content Security Policy without inline scripts or styles	<code>Services/Content/LessonContentRenderer.cs</code> , <code>Infrastructure/SecurityHeadersMiddleware.cs</code>
Embedded video	URLs parsed with strict host and id rules and rebuilt as <code>youtube-nocookie.com</code> or <code>player.vimeo.com</code> embed URLs; CSP <code>frame-src</code> allows only those players	<code>Services/Content/VideoEmbedParser.cs</code>
Overposting	Forms bind to view models; services copy allowed fields onto entities	<code>ViewModels/</code> , <code>Services/*ManagementService.cs</code>
File uploads	Extension allow-list (JPG, PNG, WebP up to 2 MB; PDF up to 10 MB), magic-byte signature check, size checked on the stream, random file names, storage outside <code>wwwroot</code> , files streamed only after an access check, SVG and HTML never accepted	<code>Services/Storage/UploadRules.cs</code> , <code>Services/Storage/FileStorageService.cs</code> , <code>Infrastructure/FileValidationAttributes.cs</code>
Abuse	Fixed-window rate limit (10 requests per 60 seconds per IP) on login, registration, password change and contact; honeypot field on the contact form	<code>Infrastructure/ServiceCollectionExtensions.cs</code> , <code>Services/ContactService.cs</code>
Transport and headers	HTTPS redirection and HSTS in production; <code>Content-Security-Policy</code> , <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: SAMEORIGIN</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> , <code>Cross-Origin-Opener-Policy</code> ; compression only for static assets (BREACH)	<code>Program.cs</code> , <code>Infrastructure/SecurityHeadersMiddleware.cs</code>
Trusted proxy	<code>UsePlatformProxyHeaders()</code> honours the platform proxy's <code>X-Forwarded-Proto</code> and <code>X-Forwarded-For</code> , so <code>Request.IsHttps</code> and the client address are correct behind the TLS-terminating edge. The container must only be reachable through that proxy (the trade-off is stated below and in section 4)	<code>Infrastructure/ApplicationBuilderExtensions.cs</code> , <code>Program.cs</code>
Errors	Developer exception page only in Development; friendly 404 and 500 pages; no stack traces; structured logging	<code>Program.cs</code> , <code>Controllers/ErrorController.cs</code>
Secrets	No passwords or keys in Git; demo passwords generated into a git-ignored file in Development; <code>Seed__AdminPassword</code> and <code>Seed__DemoStudentPassword</code> from environment variables in production; <code>.env</code> git-ignored while <code>.env.example</code> holds placeholders only	<code>appsettings.json</code> (empty seed passwords), <code>Data/Seed/DevelopmentSeedPasswords.cs</code> , <code>.env.example</code> , <code>.gitignore</code>

Content Security Policy sent with every HTML response:

```
default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; font-src 'self'; connect-src 'self';
media-src 'self'; frame-src 'self' https://www.youtube-nocookie.com https://player.vimeo.com; object-src 'none';
base-uri 'self'; form-action 'self'; frame-ancestors 'self'
```

Trusting the platform proxy's forwarded headers

Railway terminates TLS at its edge proxy and forwards plain HTTP, describing the original request in `X-Forwarded-Proto` and `X-Forwarded-For`. ASP.NET Core does not read those headers on its own, and its default allow-list trusts them only from loopback, which is not where the platform proxy lives.

`UsePlatformProxyHeaders()` therefore clears `KnownIPNetworks` and `KnownProxies`, so both headers are honoured **from any source**.

That is a deliberate trade-off, and the condition it depends on is: the application must only be reachable through that proxy. On Railway it is — the container publishes one HTTP port to the platform edge and nothing else routes to it, and the image is never given a directly reachable public address. The residual risk is that anything able to reach the container directly could forge the headers. Two consequences matter:

- `X-Forwarded-Proto` decides `Request.IsHttps`, so a forged value influences HSTS, the HTTPS redirection decision and the `Secure`-only cookie policy.
- The forms rate limiter partitions on `context.Connection.RemoteIpAddress`, so a forged `X-Forwarded-For` would let a caller present itself as a new address, start a fresh 10-requests-per-minute bucket and weaken the limit on login, registration, password change and contact.

The risk is accepted for this single-service deployment because the proxy is the only ingress. Exposing the container on a public address, or adding a second ingress, would require restoring a known-proxy allow-list (the platform's egress range) instead of clearing it.

2. OWASP Top 10:2025

Risk	How LearnHub addresses it	Evidence (automated tests)	Status
A01 Broken Access Control	Role attributes, admin base class, ownership and enrolment checks in services, files streamed through a controller, local-only return URLs	<code>AuthorizationTests</code> (access matrix, <code>Students_cannot_post_to_admin_actions_even_with_a_valid_token</code> , <code>Access_denied_page_returns_403</code>); <code>QuizAndCourseManagementTests.Students_cannot_open_other_students_results_but_admins_can</code> ; <code>Students_who_are_not_enrolled_cannot_take_or_submit_the_quiz</code> ; <code>StudentJourneyTests.Leaving_a_course_removes_access_to_its_lessons</code> ; <code>PublicSiteTests.Private_files_cannot_be_downloaded_by_guests</code> ; <code>AuthenticationTests.Login_redirects_back_to_a_local_return_url_but_never_to_another_site</code>	Address
A02 Security Misconfiguration	Production defaults: HTTPS, HSTS, secure cookies, CSP and headers, no developer pages; the image runs as Production and binds one HTTP port behind the platform's TLS-terminating proxy	<code>PublicSiteTests.Html_responses_carry_security_headers</code> ; smoke test header checks in the container job	Address
A03 Software Supply Chain Failures	Central package versions (<code>Directory.Packages.props</code>), pinned GitHub Actions major versions, vendored front-end libraries with recorded versions, lock file for browser tests	Build in CI from a clean runner on every push	Partly address: (see 4)
A04 Cryptographic Failures	Identity password hashing; TLS enforced; ASP.NET Core Data Protection for cookies and tokens; no custom cryptography	<code>AuthenticationTests.Registration_creates_a_student_signs_them_in_and_hashes_the_password</code>	Address
A05 Injection	EF Core parameterised queries, escaped LIKE patterns, encoded output, CSP	<code>ContentSafetyTests</code> (<code>Html_in_lesson_text_is_encoded_and_never_executed</code> , <code>Attack_attempts_through_formatting_markers_stay_as_text</code>); search tests	Address
A06 Insecure Design	Server-side grading from stored answers, correct answers never sent to the browser, score snapshots, unique indexes against double submissions, last-administrator protection	<code>StudentJourneyTests.Tampered_quiz_answers_are_graded_on_the_server</code> ; <code>QuizAndCourseManagementTests.The_quiz_page_never_contains_the_correct_answers</code> ; <code>Option_ids_that_belong_to_another_question_are_not_modified</code> ; <code>AdminManagementTests.User_roles_deactivation_and_self_protection</code>	Address
A07 Authentication Failures	Password policy on client and server, logout, generic messages, rate limiting, security stamp validation, deactivation through logout	<code>AuthenticationTests.Account_is_locked_after_five_failed_attempts</code> ; <code>Wrong_password_shows_a_generic_error</code> ; <code>LogicTests.Register_requires_matching_strong_passwords_and_accepted_terms</code>	Address
A08 Software or Data Integrity Failures	Antiforgery tokens on every POST; uploads verified by content signature; the image is built from this repository by Railway and the presentation site is deployed from the same repository, so no artefact is copied by hand	<code>AuthenticationTests.Posts_without_an_anti_forgery_token_are_rejected</code> ; <code>FileStorageServiceTests.Rejects_a_file_whose_content_does_not_match_its_extension</code> ; <code>Rejects_files_over_the_size_limit_even_if_the_declared_length_is_small</code>	Address

Risk	How LearnHub addresses it	Evidence (automated tests)	Status
A09 Security Logging and Alerting Failures	Structured logs for sign-in failures, lockouts, seeding, administrator creation, enrolments, deletions and discarded spam; the container log, which the <code>container</code> job keeps as an artifact	Both the <code>container</code> and <code>e2e</code> jobs fail if the application logs a <code>fail:</code> or <code>crit:</code> line	Partly address: (see 4)
A10 Mishandling of Exceptional Conditions	Global exception handler with friendly pages; <code>DbUpdateException</code> from unique indexes turned into clear messages; transactions with the execution strategy; database health check	<code>NavigationCrawlTests</code> (no 500 responses for guest, student or admin); <code>PublicSiteTests.Unknown_pages_and_invalid_ids_return_a_friendly_404</code> ; <code>Health_endpoint_reports_the_database_as_healthy</code>	Address

3. Assignment security requirements

SEC-04 IDOR (insecure direct object references)

Every identifier in a URL is treated as untrusted:

- `/Quizzes/Result/{id}` returns the attempt only when it belongs to the signed-in student. Another student's id gives 404 rather than 403, so attempt ids cannot be probed. Administrators review attempts in **Admin** → **Quiz results**.
- `/Resources/Details/{id}` and `/Resources/Open/{id}` allow preview lessons for everyone; other lessons require enrolment in that course (or the Admin role). Files are never served from `wwwroot`.
- `/Quizzes/Take/{id}` requires enrolment and a published quiz, and a submitted option is accepted only if it belongs to that question.
- Admin actions are available only to administrators, even with a valid antiforgery token.

SEC-02 XSS

- Razor encodes every value by default; the code base does not render user input with `Html.Raw`. Lesson bodies go through `LessonContentRenderer`, which encodes the whole text before applying paragraph, list, heading, code and emphasis formatting from a fixed allow-list.
- The CSP blocks inline scripts, event-handler attributes and scripts from other origins, so even an unexpected injection could not run code.
- Category icons come from a fixed allow-list, and video URLs are rebuilt rather than echoed.

SEC-03 CSRF

- `AutoValidateAntiforgeryTokenAttribute` is registered globally, so every POST, PUT and DELETE needs a valid token. The form tag helper adds the token automatically.
- State changes are never done with GET: enrol, leave, complete, submit, publish, delete and log out are all POST forms.
- `SameSite=Lax` cookies add a second layer.

SEC-06 Uploads

The upload pipeline in `FileStorageService` rejects a file unless every check passes:

- A file was chosen and its extension is on the allow-list for that use (course cover: image; resource: image or PDF).
- The size is within the limit, measured while copying rather than trusting the declared length.
- The first bytes match a JPEG, PNG, WebP or PDF signature, and the detected type fits the extension.
- The file is saved under a new random name with the detected extension, in `Storage:RootPath` outside `wwwroot`.

Uploaded files are served by `ResourcesController.Open` after the access check, with the stored content type and `X-Content-Type-Options: nosniff`. Course covers are the only uploads shown as public images.

SEC-08 Secrets

- `appsettings.json` contains no passwords (`Seed:AdminPassword` and `Seed:DemoStudentPassword` are empty) and no connection string with credentials: SQLite is a file, so `ConnectionStrings:DefaultConnection` is just a path.
- Development.** `DevelopmentSeedPasswords` generates strong random passwords for the demo accounts and writes them to `App_Data/demo-credentials.json`, which is git-ignored (`App_Data/` is excluded wholesale). They are generated once and reused on later starts, so signing in as a demo account keeps working without a committed password.
- Production.** The administrator and demo-student passwords must come from the environment variables `Seed__AdminPassword` and `Seed__DemoStudentPassword` (Railway service variables, or `.env` locally). With no password set the account is still created but cannot sign in, and the application logs a warning rather than inventing one. A password can also be set with `dotnet user-secrets` for local use.
- `.env` is git-ignored; `.env.example` documents every variable with placeholders only, so the repository can be cloned and configured without ever holding a real credential.
- CI generates throw-away passwords for the container and browser-test accounts at run time and masks them in the logs. The only fixed passwords in the repository are synthetic ones in `tests/LearnHub.Tests/Infrastructure/LearnHubWebApplicationFactory.cs` for temporary in-memory test databases.
- A repository scan for assignments of password, secret, key, token or connection string values found nothing except those synthetic test values.

SEC-05 Overposting

Controllers accept view models that contain only the fields a form may change. For example, a student profile update cannot set roles, and a course form cannot set `CreatedAt`. Services load the entity and copy the allowed values.

4. Residual risks and recommendations

Risk	Current state	Recommendation
Forwarded headers are trusted from any source	<code>KnownIPNetworks</code> and <code>KnownProxies</code> are cleared so the platform proxy's headers are honoured (see section 1). Safe only while the proxy is the sole ingress	If the container ever becomes directly reachable or gains a second ingress, restore a known-proxy allow-list; otherwise a forged <code>X-Forwarded-For</code> can influence the per-IP rate limiter and a forged <code>X-Forwarded-Proto</code> the secure-cookie and HSTS decisions
No email verification or password reset by email	Registration signs in immediately; forgotten passwords need an administrator	Add an email provider, <code>RequireConfirmedAccount</code> and the Identity password-reset flow
No multi-factor authentication	Password only, with lockout and rate limiting	Enable Identity's authenticator-app MFA, at least for administrators
Dependency updates	Versions are pinned, but updates are manual	Enable Dependabot for NuGet, npm and GitHub Actions, and add <code>dotnet list package --vulnerable</code> to CI
Monitoring and alerting	Logs are written but no alerts are configured	Add an uptime/error monitor for <code>/health</code> and a log drain with alerts on 5xx responses and lockouts
Rate limiting per instance	In-memory limiter; resets on restart and is per instance	Acceptable for the single-instance deployment; use a distributed limiter (Redis or the platform's edge rules) if the service is scaled out
SQLite on a single volume	One file on the Railway volume, reachable only from the container, so there is no database network surface — but no automatic point-in-time recovery either	Keep the volume, take a copy before a schema change, and move to a managed database if the deployment grows
Uploaded PDFs	Signature-checked but not malware-scanned	Scan uploads in the storage layer if they ever move to object storage
Third-party video	Privacy-enhanced YouTube and Vimeo embeds	Keep the frame allow-list minimal; provide text summaries for accessibility

Testing

1. Strategy

LearnHub is tested at several levels, from fast checks of single classes up to the published application running in a real browser. Every level runs automatically in GitHub Actions on every push that touches the application.

Level	What it proves	Tools	Where
Unit	Grading, progress, validation rules, content safety, file signatures and names	xUnit v3	<code>tests/LearnHub.Tests/Unit</code>
Data model	The EF Core model creates a valid SQLite schema, a create script that enforces the integrity rules, and only the delete rules the design intends	xUnit, EF Core	<code>tests/LearnHub.Tests/Data</code>
Service	Business rules against a real (in-memory SQLite) database	xUnit, EF Core	<code>tests/LearnHub.Tests/Services</code>
Integration	The whole application in memory: routing, authentication, authorisation, antiforgery, forms, views, database	<code>WebApplicationFactory</code>	<code>tests/LearnHub.Tests/Integration</code>
Link crawl	Every link reachable by a guest, a student and an administrator returns a valid page	<code>WebApplicationFactory</code>	<code>NavigationCrawlTests</code>
Container	The Docker image Railway builds, started the way Railway starts it — Production settings, <code>PORT=8080</code> , a volume at <code>/data</code> : migrations apply once, demo data is seeded, the administrator is created, the smoke test passes, and after a restart migrations are not re-applied, which proves the volume persists the SQLite file	Docker, <code>scripts/smoke-test.sh</code>	<code>ci.yml</code> job <code>container</code>
Browser	Guest, student and admin journeys on desktop, tablet and phone sizes; console errors, failed requests, broken images, sideways scrolling	Playwright	<code>tests/e2e/specs</code>
Accessibility	Automated WCAG 2.2 A and AA rules on public, student and admin pages	axe-core with Playwright	<code>tests/e2e/specs/accessibility.spec.js</code>
Smoke	A running instance (the container in CI, and the deployed application once Railway is deployed) serves pages, links, protected areas and security headers	<code>scripts/smoke-test.sh</code>	<code>ci.yml</code> job <code>container</code>
Manual	Things automation does not judge well: readability, keyboard flow, visual quality	Checklist in section 7	This document

`tests/LearnHub.Tests/Infrastructure` holds no tests of its own: it is the shared test host — the in-memory application factory, the isolated SQLite database helper, the test environment and the browser client extensions.

Test data is isolated: each integration test class gets its own in-memory SQLite database with migrations and demo data applied, so tests can run in parallel and never touch a developer's database. Test accounts use synthetic passwords defined in the test factory; CI generates random passwords for the browser tests.

The three CI jobs are `build-and-test` (the whole .NET suite), `container` (the Docker image started the way Railway starts it, plus the smoke test) and `e2e` (Playwright against a Production instance over HTTPS).

2. How to run the tests

See [README – Running the tests](#). In short:

```
dotnet test --solution LearnHub.sln
```

3. Latest results

Evidence: measured on 14 September 2026 on the working branch `feature/railway-firebase-sqlite` with `dotnet test LearnHub.sln -c Release`. Every job uploads its evidence as an artifact — the TRX result files, the Playwright report and screenshots, and the container log — so the figures below can be reproduced from the [Actions page](#) or locally.

Job	Result
<code>build-and-test</code>	Release build with warnings treated as errors: 0 warnings, 0 errors; 211 of 211 tests passed , 0 failed, 0 skipped (116 methods; theories run once per data row)
<code>container</code>	The image built and started with Production settings, <code>PORT=8080</code> and a volume at <code>/data</code> ; the health check passed; <code>scripts/smoke-test.sh</code> passed; the start-up log showed the migrations applied, the demo data seeded and the administrator account created, with no <code>fail:</code> or <code>crit:</code> lines; after a restart the log showed no migration applied again, so the SQLite file and the Data Protection key ring really live on the volume
<code>e2e</code>	42 of 42 browser checks passed (14 Playwright tests × 3 viewports: desktop 1440×900, tablet 768×1024, mobile Pixel 7), including 9 axe-core accessibility scans covering 26 page views per viewport, with no WCAG A or AA violations; no errors in the application log

4. Automated test inventory

116 test methods produce 211 test cases, because theories run once per data row (`Unit` 36 methods, `Services` 23, `Data` 3, `Integration` 54).

Class	Methods	Covers
<code>Unit/LogicTests</code> (grader, display format, progress, view model validation)	16	Scoring and the pass mark (rounding down, unanswered questions, option ids from other questions, questions weighted by their points), progress width classes and completion derived from counts, initials, human-readable durations and file sizes, category line colours, registration rules (strong matching passwords, accepted terms), question rules (two options, one correct, no duplicates), category icon allow-list
<code>Unit/ContentSafetyTests</code> (video parser, lesson renderer, search pattern)	10	YouTube and Vimeo URLs accepted only in known formats and rebuilt as privacy-enhanced embeds, HTML in lesson text is encoded, only allow-listed formatting is produced, code blocks stay encoded, attacks through formatting markers stay text, LIKE wildcards escaped, long search terms truncated
<code>Unit/FileStorageServiceTests</code>	7	Valid image saved with a random name and detected type, mismatched content rejected, oversize files rejected even with a small declared length, PDFs accepted for documents, paths outside the storage root never resolved, only uploaded thumbnails can be deleted
<code>Unit/DevelopmentSeedPasswordsTests</code>	3	Development generates strong demo passwords once and reuses them from the git-ignored credentials file; configured passwords are never replaced; other environments never generate passwords
<code>Data/DatabaseModelTests</code>	3	SQLite schema creation from the model, the create script enforcing the integrity rules, intentional delete behaviour of key relationships (Restrict and Cascade)
<code>Services/CatalogueServiceTests</code>	12	Duplicate category names (case-insensitive), category deletion rules, search and filters return published courses only (including matches in lesson titles and descriptions), unpublished details hidden except for admins, double enrolment and unpublished enrolment rejected, stored progress recalculated when content changes, draft lessons hidden and excluded from progress, leaving a course keeps completions
<code>Services/QuizAndCourseManagementTests</code>	11	Graded attempts store every answer, other students' results are hidden, non-enrolled students cannot take quizzes, quiz pages never contain correct answers, the start time comes from the signed token issued with the quiz page, question points are saved and used for later attempts, foreign option ids are not modified, removing a selected answer keeps attempts, course deletion order
<code>Integration/PublicSiteTests</code>	19	Public pages load, only published courses listed, server-side search over titles, descriptions and lesson titles, category filter and category page, paging beyond the first page, empty search state, course details, friendly 404s, drafts hidden, preview lessons open while others need login, private files protected, security headers, health endpoint, static assets and seeded covers, home map lines, isolated test database
<code>Integration/AuthenticationTests</code>	9	Registration (role, sign-in, password hashing), server-side validation, duplicate email, students and admins sent to their own dashboards after login, generic wrong-password error, local-only return URLs, logout, lockout after five failures, antiforgery rejection
<code>Integration/AuthorizationTests</code>	8	Access matrix for guest, student and admin across protected pages (47 cases over the page lists), 403 access denied page, students cannot post to admin actions even with a valid token
<code>Integration/StudentJourneyTests</code>	5	Register, enrol, study, take the quiz and see the result; tampered answers graded on the server; leaving removes lesson access; profile validation and saving; changing the email address needs the current password
<code>Integration/AdminManagementTests</code>	10	Category create, edit, delete and validation; category with courses protected; course create with cover, edit, publish, delete; resources created, validated, streamed and deleted; quiz publishing needs questions; user roles, deactivation and self-protection; enrolment management; contact inbox; spam honeypot
<code>Integration/NavigationCrawlTests</code>	3	Crawls every internal link as a guest, a student and an administrator: no 500 errors and no broken links

Browser scenarios (14 tests; each runs on desktop 1440×900, tablet 768×1024 and Pixel 7, so 42 checks)

Spec	Scenario
<code>public.spec.js</code>	Home page presents the catalogue (and the map draws one line per category)
<code>public.spec.js</code>	Navigation menu works on every screen size
<code>public.spec.js</code>	Server-side search and category filter
<code>public.spec.js</code>	Course details and a free preview lesson
<code>public.spec.js</code>	Contact form validates in the browser before submitting
<code>public.spec.js</code>	Registration form gives live password feedback
<code>public.spec.js</code>	Unknown pages show the friendly 404 page
<code>public.spec.js</code>	About page
<code>student.spec.js</code>	Dashboard, course progress, lesson and quiz
<code>admin.spec.js</code>	Dashboard and management pages
<code>admin.spec.js</code>	Category create and delete through the forms
<code>accessibility.spec.js</code>	Public pages: home, catalogue, course details, preview lesson, About, Contact, Privacy, login, register, 404
<code>accessibility.spec.js</code>	Student pages: dashboard, My Courses, lesson, quiz history, profile, change password
<code>accessibility.spec.js</code>	Administration pages: dashboard, courses, new course, categories, resources, quizzes, quiz results, enrolments, users, messages

Every browser scenario fails on console errors, failed requests to LearnHub, broken images or horizontal scrolling, and saves full-page screenshots. The app runs from the published build in Production mode over HTTPS, so secure cookies, CSP and error pages behave as in production.

5. Representative test cases

ID	Test case	Steps	Expected	Actual	Result
TC-01	Register with valid data	Fill in the form with a strong password, accept terms, submit	Account created with the Student role, signed in, dashboard shown, password stored as a hash	As expected	Pass
TC-02	Register with a weak password	Enter "short"	Message "Use at least 8 characters with an upper-case letter, a lower-case letter and a number."	As expected; the hint is hidden while the same rule is shown as the error	Pass
TC-03	Duplicate email	Register twice with the same email	One clear error, no second account	As expected	Pass
TC-04	Lockout	Five wrong passwords	Account locked for 15 minutes with the lockout message	As expected	Pass
TC-05	Open redirect	Log in with <code>returnUrl=https://example.org</code>	Redirect to the user's own dashboard	As expected	Pass
TC-06	Search	Search "sql"	Published courses that match the title, the description or a lesson title are listed with the number found, and drafts never appear	As expected	Pass
TC-07	Preview lesson	Guest opens a free preview, then a locked lesson	Preview shown; locked lesson redirects to login	As expected	Pass
TC-08	Enrol twice	Post the enrol form twice	Second attempt shows "You are already enrolled in this course."	As expected	Pass
TC-09	Quiz tampering	Submit an option id from another question	Recorded as unanswered and counted as incorrect	As expected	Pass
TC-10	Result privacy	Student B opens student A's result URL	404	As expected	Pass
TC-11	Admin access	Student opens <code>/Admin/Courses</code> and posts to an admin action	Access denied (403) and the POST is refused	As expected	Pass
TC-12	Missing antiforgery token	POST to <code>/Account/Login</code> without the token	400 Bad Request	As expected	Pass
TC-13	Disguised upload	Upload an HTML file containing a script, renamed <code>cheat-sheet.pdf</code>	Rejected with "The file content does not match an allowed file type."; nothing stored	As expected	Pass
TC-14	Category with courses	Delete a category that has courses	Deletion refused with an explanation	As expected	Pass
TC-15	Publish empty quiz	Publish a quiz without questions	"Add at least one question before publishing this quiz."	As expected	Pass
TC-16	Last administrator	The only admin tries to deactivate or demote themselves	Refused with a clear message	As expected	Pass
TC-17	Course deletion	Delete a course with attempts	Answers, attempts, quizzes, lessons and enrolments removed; no database error	As expected; the deletion order is deliberate (see ERD)	Pass
TC-18	Broken links	Crawl all pages as each role	No 404 or 500 responses from internal links	As expected	Pass
TC-19	Responsive layout	Browser tests at 1440, 768 and Pixel 7 widths	No horizontal scrolling, menus usable	As expected (after the fixes in section 6)	Pass
TC-20	Accessibility	axe-core WCAG 2.2 A/AA scan on 26 page views per viewport	No violations	As expected (after the fixes in section 6)	Pass

6. Defects found and fixed

#	Found by	Symptom	Cause	Fix	Commit
1	CI build	xUnit v3 tests would not run on the .NET 10 SDK	VSTest mode no longer supported for this setup	Opted into Microsoft Testing Platform in <code>global.json</code>	<code>32625d6</code>
2	CI build	Razor compile errors in the pagination partial	A variable named <code>page</code> clashed with the reserved <code>@page</code> directive	Renamed the variables	<code>fc83d0a</code>
3	Link crawl test	<code>/Admin/Courses/Details/{id}</code> returned 500 on SQLite	The query used SQL <code>APPLY</code> , which SQLite cannot translate	Rewrote it as separate flat queries	<code>945aed8</code>

#	Found by	Symptom	Cause	Fix	Commit
4	Admin CRUD test	Deleting a course failed with a "relationship severed" error	EF Core tried to delete tracked dependants with Restrict rules	Delete dependent quiz answers first with <code>ExecuteDelete</code> in a transaction	945aed8
5	CI smoke test	Form pages returned 500 in Production over plain HTTP	TLS is terminated by the platform proxy, so the application saw an insecure request and refused its own Secure-only antiforgery cookie	Added <code>UsePlatformProxyHeaders()</code> and made the smoke test send <code>X-Forwarded-Proto</code>	3851e07
6	Browser test (tablet)	Admin pages scrolled sideways by 119 px	Visually hidden table header text escaped the scroll container	Made <code>.table-wrap</code> the containing block	acee776
7	Screenshot review	Admin sidebar invisible on desktop	Bootstrap forces <code>.offcanvas-lg</code> to a transparent background	Restored the dark rail with a more specific rule	acee776
8	Screenshot review	Admin filters always stacked	Invalid CSS grid template (<code>auto-fit</code> combined with <code>auto</code>)	Flexible wrapping row	acee776
9	Browser test (phone)	Lesson pages scrolled sideways by 198 px	Grid column sized to the longest code line	<code>minmax(0, 1fr)</code> column	3851e07
10	Browser test (phone)	About page scrolled sideways	Bootstrap <code>g-5</code> gutters wider than the container padding	<code>g-4 g-lg-5</code>	3851e07
11	Screenshot review	Home map labels overlapped lines and the hub	Label positions did not account for long names	Two-line labels placed clear of the lines; long names shortened	ab9ad5d
12	Screenshot review	Home map invisible when animations are disabled	The resting style was hidden and only the animation fill showed it	Keyframes now animate from hidden to a visible resting style	ab9ad5d
13	Accessibility scan	Code blocks and the About table could not be scrolled with the keyboard	Scrollable regions without focusable content	Focusable regions with labels	5de245b
14	Accessibility scan	Two admin top bar links had no name on phones	Their text is hidden below the breakpoint	<code>aria-label</code> on both links	5de245b
15	Build log review	91 build warnings (xUnit1051) in test code	Async calls did not pass the test cancellation token	Token passed everywhere; CI now treats warnings as errors	273ccc6

7. Manual testing checklist

Recorded on 14 September 2026 against the container build of the `feature/railway-firebase-sqlite` working tree, using the Playwright screenshots of every scenario on desktop, tablet and phone plus the automated reports. Items that need a person with a live deployment are marked **To do**; the team completes them after the first Railway deploy.

Area	Check	Result	Evidence or note
Build	Solution builds in Release with no errors and no warnings	Pass	CI build step with <code>-p:TreatWarningsAsErrors=true</code>
Pages	Every public, student and admin page loads without server errors	Pass	Link crawl tests as all three roles
Roles	Guest, student and admin see only their own navigation and pages	Pass	Authorisation tests; screenshots of each role
CRUD	Create, read, update and delete work for courses, categories, resources, quizzes, questions, enrolments, users and messages	Pass	Admin management integration tests; admin browser test
Validation	Client-side messages appear before submit; server repeats every rule	Pass	Contact and registration browser tests; server validation tests
Responsive	Layout correct at 1440, 768 and 412 px; no sideways scrolling	Pass	Browser tests and screenshot review (defects 6–10 fixed)
Console	No JavaScript errors or failed requests	Pass	Browser tests fail on any console error
Images	No broken images, covers and diagrams load	Pass	Browser tests check every image
Links	No dead internal links	Pass	Link crawl tests
Accessibility (automated)	No WCAG 2.2 A/AA violations found by axe-core	Pass	Accessibility spec in 3 viewports
Visual quality	Consistent design system, readable typography, balanced layouts	Pass	Screenshot review; issues 7, 8 and 11 fixed
Error pages	404 and access denied pages are friendly and branded	Pass	Browser test and integration tests
Security headers	CSP and other headers present	Pass	Integration test and smoke test
Keyboard	Complete journeys (register, enrol, lesson, quiz, admin form) with keyboard only; focus always visible	To do	Focus styles exist for all interactive elements; needs a person to run through

Area	Check	Result	Evidence or note
Screen reader	Headings, landmarks, form labels and quiz questions announced correctly (NVDA or VoiceOver)	To do	Automated checks passed; needs a manual pass
Real devices	iPhone and Android phone in portrait and landscape	To do	Emulated Pixel 7 passed
Production	Health, HTTPS redirect, admin login, upload and student journey on Railway	To do	The <code>container</code> job already performs these checks against the image; run the verification table in Deployment after the first Railway deploy

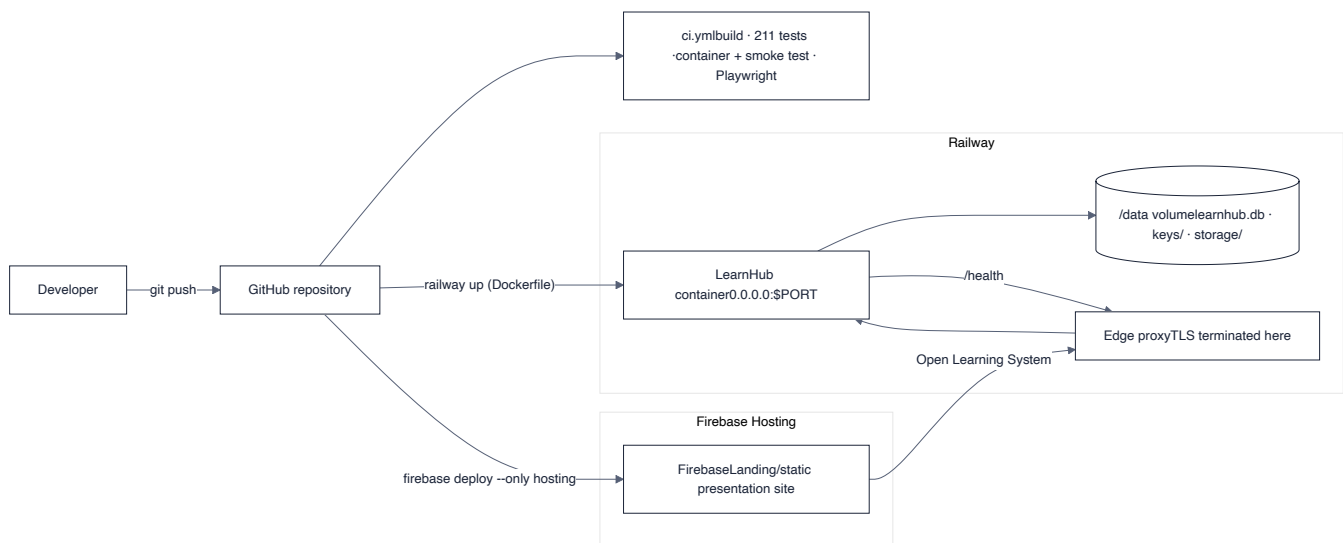
Deployment

1. Overview

LearnHub is delivered by three services, each doing one job:

Part	Service	What it does
Source and automation	GitHub (Khayit0ff/LearnHub)	Holds the code, runs <code>ci.yml</code> on every push and can add a migration with <code>ef-migrations.yml</code>
Application	Railway (Docker)	Runs the ASP.NET Core MVC application from <code>Dockerfile</code> , with a volume at <code>/data</code> holding the SQLite database, the Data Protection keys and uploads
Presentation	Firebase Hosting	Serves the static site in <code>FirebaseLanding/</code> (landing page, screenshots, design system)

Firebase Hosting cannot run ASP.NET Core. It serves static files from a CDN and has no server-side runtime, so it cannot execute a .NET application, open a database or keep a session. It therefore *presents* the project – what it is, who it is for, screenshots and links – and its "Open Learning System" call to action points at the Railway deployment, which is where the working application lives.



Current state of the deployment:

- Firebase Hosting is live:** <https://learnhub-wapp.web.app> (verified: `/` returns 200, `/assets/site.css` returns 200, `/assets/screens/home-desktop.png` returns 200, an unknown path returns 404).
- Railway has not been deployed yet.** The `Dockerfile`, `railway.json` and `.env.example` are in place and the CI `container` job builds the image and exercises it the way Railway would, but the Railway CLI on this machine is not authenticated (`railway whoami` reports `Unauthorized`). Everything in section 2 is therefore a set of steps to follow, not a description of a running service. Set `Site_PresentationUrl` and run the `set-app-url.sh` step after the first deploy, once the real domain exists.
- No credential, connection string or token is stored in the repository. Railway holds the secrets as service variables.

2. Deploy to Railway, step by step

2.1 Prerequisites

- A Railway account (<https://railway.com>) and the CLI: `npm install -g @railway/cli` (or `brew install railway`).
- The repository cloned locally, on the branch that holds the container files.
- An administrator password you generate yourself and keep out of Git (a password manager or the Railway dashboard).

2.2 Sign in and create (or link) the project

Run these from the repository root, so the CLI picks up the working directory.

```
cd LearnHub

# 1. Sign in (opens a browser; `railway login --browserless` prints a code instead)
railway login
railway whoami          # confirms the account

# 2a. First time: create the project and the service
railway init --name learnhub

# 2b. Already created the project in the dashboard: link this directory to it instead
railway link             # or: railway link --project <project-id>
```

`railway init` uploads nothing by itself; it creates the project and links the current directory to it. If you create the service in the Railway dashboard instead, run `railway service` afterwards to link the directory to that service.

2.3 Create the volume first

Create the volume **before** the first deploy, so the very first start already writes to persistent storage:

```
railway volume add --mount-path /data
railway volume list    # shows the volume, its mount path and the service it is attached to
```

2.4 Set the variables

Every variable is listed in `.env.example`. Either paste them into **Railway → your service → Variables**, or use the CLI (add `--service <name>` when the project has more than one service):

```
railway variables --set "ASPNETCORE_ENVIRONMENT=Production" \
  --set "DATABASE_CONNECTION_STRING=Data Source=/data/learnhub.db" \
  --set "DataProtection__KeysPath=/data/keys" \
  --set "Storage__RootPath=/data/storage" \
  --set "Seed__AdminEmail=admin@learnhub.local" \
  --set "Seed__AdminPassword=<a strong password you choose>" \
  --set "Seed__DemoStudentEmail=demo.student@example.com" \
  --set "Seed__DemoStudentPassword=<another strong password>" \
  --set "Seed__DemoData=true"
```

Variable	Example / placeholder	Purpose
ASPNETCORE_ENVIRONMENT	Production	Turns on the friendly error page, HSTS and the <code>Secure</code> -only cookie policy. Development is for local work only.
DATABASE_CONNECTION_STRING	Data Source=/data/learnhub.db	The SQLite file on the volume. Overrides <code>ConnectionStrings:DefaultConnection</code> from <code>appsettings.json</code> .
DataProtection__KeysPath	/data/keys	Where the Data Protection key ring lives. It encrypts the sign-in and anti-forgery cookies, so it must persist.
Storage__RootPath	/data/storage	Private folder for administrator uploads and generated thumbnails, outside <code>wwwroot</code> .
Seed__AdminEmail	admin@learnhub.local	Email of the administrator created on the first start of an empty database.
Seed__AdminPassword	<a strong password you choose>	Required for the admin area to be usable. See the warning below.
Seed__DemoStudentEmail	demo.student@example.com	Email of the demonstration student.
Seed__DemoStudentPassword	<another strong password>	Without it the demo student exists (so the dashboards look realistic) but cannot sign in.
Seed__DemoData	true	Seeds the demonstration catalogue into an empty database. Set to <code>false</code> for a clean installation.

The remaining optional variables:

Variable	Example / placeholder	Purpose
Site__RepositoryUrl	https://github.com/Khayit0ff/LearnHub	Repository link shown in the footer and on the About page.
Site__PresentationUrl	https://learnhub-wapp.web.app	Link from the application back to the presentation site.
Database__ApplyMigrationsOnStartup	true (default)	Leave <code>true</code> for the single-instance deployment used here.
RateLimiting__PermitLimit / RateLimiting__WindowSeconds	10 / 60 (defaults)	Form submissions per IP address per window.

Without `Seed__AdminPassword`, **no administrator exists**. The application creates the administrator only on the first start of an empty database, and only when that variable is set. Without it, start-up logs a warning, the `Admin` role exists but nobody holds it, and `/Admin` is unreachable – there is no default password and no back door. Set the variable (a strong one: it must satisfy the Identity rules, so at least 8 characters with upper case, lower case and a digit) and restart. If the account was already created with different credentials, the initialiser deliberately does not touch it; promote a user from an existing administrator account instead.

2.5 Deploy

```
railway up                # builds Dockerfile on Railway and deploys, streaming the build log
railway up --detach        # the same, without attaching to the log stream
railway logs               # follow the application log afterwards
```

Then generate the public domain and check the result:

```
railway domain            # creates/prints the *.up.railway.app domain
railway status            # linked project, service and environment
curl -fsS https://<service>.up.railway.app/health
```

The first start applies the migration, creates the roles, creates the administrator (when `Seed__AdminPassword` is set) and seeds the demo catalogue, so it can take a minute on a cold start. Railway waits for the health check before it routes traffic to the new deployment.

Port binding. Railway injects `PORT` at run time and the application binds `http://0.0.0.0:$PORT` (`UsePlatformPort`). Do not set `PORT` yourself on Railway. The `PORT=8080` in the image is only a fallback for running the container locally; if `PORT` is absent, the normal ASP.NET Core URL settings apply.

Health check. `railway.json` configures `/health` with a 300-second timeout and a restart-on-failure policy. `/health` runs `DatabaseHealthCheck`, which reports whether the SQLite file is reachable – so a deployment only goes live once the database really works. A push to the connected branch redeploys automatically; `railway up` deploys the working directory as it is on disk.

3. Why the volume matters

A container filesystem is **ephemeral**: it is recreated from the image on every deploy, restart or crash. Anything written inside the container is lost at that moment. A Railway volume is a disk that is mounted back into the container at the same path every time, so the three things LearnHub writes are placed there:

Path on the volume	What it holds	What is lost without the volume
<code>/data/learnhub.db</code>	The whole SQLite database	Every redeploy starts from an empty database: courses, enrolments, attempts and registered accounts disappear, and the seeder runs again from scratch
<code>/data/keys</code>	The Data Protection key ring	The keys that encrypt the authentication cookies are regenerated, so every signed-in user is signed out and any pending anti-forgery token becomes invalid
<code>/data/storage</code>	Uploaded resource files and thumbnails	Files uploaded through the admin area vanish while their database rows survive, leaving broken resources

This is also the deliberate, easy-to-explain answer to the question "**is SQLite not a bad choice on an ephemeral filesystem?**" – it would be, if the file lived inside the container. It does not: the image is disposable and the volume is the database. The CI `container` job proves the point mechanically: it runs the image with `--mount type=volume,source=learnhub-data,target=/data`, then restarts the container and asserts that the log does **not** contain `Applying migration` a second time, because the schema was already there. It then keeps the data across rollbacks too (section 8).

Deleting a volume is the destructive equivalent of dropping the database: do it only deliberately, from the Railway dashboard, and expect a fresh seed on the next start.

4. Deploy the Firebase Hosting site

The presentation site in `FirebaseLanding/` is deployed with the Firebase CLI. `firebase.json` sets `"public": "FirebaseLanding"` and `.firebaserc` sets the default project to `learnhub-wapp`.

```
# 1. Sign in and check which project is selected
firebase login
firebase projects:list

# 2a. Use the existing project (already the default in .firebaserc)
#    -> nothing to do; continue with step 3

# 2b. Or create your own project and point .firebaserc at it
firebase projects:create learnhub-wapp --display-name "LearnHub"
firebase use learnhub-wapp

# 3. Deploy only the hosting site
firebase deploy --only hosting
```

The site is already live at <https://learnhub-wapp.web.app>.

Point the call to action at the real Railway domain. The landing page ships with a conventional Railway address as a placeholder, because Railway generates the domain when the service is created. Once the application is deployed, rewrite it and redeploy:

```
scripts/set-app-url.sh https://<service>.up.railway.app
firebase deploy --only hosting
```

The script replaces every `https://<something>.up.railway.app` link on the landing page, refuses to run if it cannot find one, and prints how many links it changed. It is safe to run repeatedly. The static site contains no secrets and no server-side code, so a hosting deploy cannot break the application.

5. Local development

```
dotnet restore           # restore NuGet packages
dotnet build LearnHub.sln # build
dotnet run --project src/LearnHub # run the application
```

The default address is printed on start-up (Kestrel's `http://localhost:5xxx`). Development uses SQLite in `App_Data/learnhub.db`; no database server is needed. Migrations, roles and the administrator are applied on the first start exactly as in production, and the demo catalogue is seeded so the dashboards are not empty.

Demo credentials. In Development, when no demo passwords are configured, `DevelopmentSeedPasswords` generates strong random ones and writes them to:

```
App_Data/demo-credentials.json    (git-ignored – generated on the first Development start)
```

Read the administrator and demo-student passwords there and sign in. The file is deliberately ignored by Git, so a password is never committed; the accounts are DEMO ONLY. If you would rather choose the passwords yourself, set them with user-secrets or environment variables and the generator leaves them alone:

```
dotnet user-secrets set "Seed:AdminPassword" "<a strong password>" --project src/LearnHub
dotnet user-secrets set "Seed:DemoStudentPassword" "<another strong password>" --project src/LearnHub
```

Resetting local state. Stop the application, delete `App_Data/`, and start again: the folder holds the SQLite file, the generated credentials and the storage folder. The next start recreates the database and reseeds it, so any local accounts, enrolments or uploads are gone.

For a Railway-like run on your own machine – Production settings, the image's own variables and a volume-like mount – use the same shape the CI `container` job uses:

```
docker build --tag learnhub:local .
docker run --rm --publish 8080:8080 \
  --mount type=volume,source=learnhub-local,target=/data \
  --env Seed_AdminPassword="<a strong password>" \
  learnhub:local
# then: curl -fsS http://127.0.0.1:8080/health
```

6. Database migrations

The `dotnet-ef` tool is pinned in `.config/dotnet-tools.json`, so restore it once per clone:

```
dotnet tool restore

# Add a migration after changing an entity or a Fluent API configuration
dotnet ef migrations add <Name> --project src/LearnHub --output-dir Data/Migrations

# Apply it to the local App_Data database
dotnet ef database update --project src/LearnHub
```

Use a PascalCase name that says what changed, for example `AddCourseLevel`. There is exactly one migration folder (`src/LearnHub/Data/Migrations`) because there is exactly one provider: SQLite. `DesignTimeDbContextFactory` resolves the connection string the same way the application does, so a migration is always generated against the right schema, and `dotnet ef` does not need the web application to start.

Migrations also run automatically at start-up. `DatabaseInitializer` applies every pending migration when `Database:ApplyMigrationsOnStartup` is `true` (the default, and the right setting for the single-instance Railway deployment), then creates the roles, the administrator and the demo data. That is why the image needs no separate migration step and why a deploy can go from "image pulled" to "schema current" on its own. If you ever set the flag to `false`, start-up logs a warning and skips seeding instead of failing – run `dotnet ef database update` against the deployed database first.

Without the .NET SDK, run the **Actions → EF Core migration (cloud)** workflow (`.github/workflows/ef-migrations.yml`). Give it a PascalCase name; it restores the tools, adds the migration to `Data/Migrations`, verifies the solution still builds and commits the result to the selected branch. Pushing that commit redeploys the application, which applies the migration on start-up.

Migrations are forward-only. To undo one in production, add a new migration that reverses it rather than editing history.

7. Verify the deployment

Run these after a deploy, in order. The first two catch a broken image; the rest confirm the application really works as a signed-in user.

#	Check	How	Expected
1	Health	<code>curl -fsS https://<domain>/health</code>	HTTP 200 with <code>Healthy</code>
2	Home page	Open <code>https://<domain>/</code>	The catalogue page loads with seeded courses
3	Register and sign in	Register a new account, sign out, sign in again	Redirected to the student dashboard; the name shows in the navbar
4	Admin sign-in	Sign in with <code>Seed__AdminEmail</code> and <code>Seed__AdminPassword</code>	<code>/Admin</code> dashboard with seeded figures
5	Admin CRUD	For example Admin → Categories → create a category, edit it, delete it	The change is saved and shown; the success message appears
6	Smoke test	<code>scripts/smoke-test.sh https://<domain></code>	"All smoke checks passed for https://"
7	HTTPS and headers	Open the <code>http://</code> address, or inspect the response headers	Redirected to <code>https://</code> ; <code>Content-Security-Policy</code> , <code>X-Content-Type-Options</code> and <code>Referrer-Policy</code> present
8	Persistence	Redeploy (or <code>railway redeploy</code>) and sign in again	The account and its data are still there and the session survives

The smoke test covers the guest journey and needs no credentials:

```
SMOKE_WAIT_SECONDS=300 scripts/smoke-test.sh https://<service>.up.railway.app
```

It waits for `/health`, loads every public page, follows the course and preview-lesson links it finds, asserts that the protected areas redirect to the login page (302), and checks the security headers. `SMOKE_WAIT_SECONDS` covers a cold start.

8. Rollback

Roll back the application. Railway keeps the deployment history, so no rebuild is needed:

- **Dashboard:** open the service → **Deployments**, find the last good deployment and choose **Redeploy**.
- **CLI:** `railway redeploy` redeploys the most recent deployment of the linked service, or pick a specific one from `railway deployment list` / the dashboard.
- **From Git:** revert the faulty commit (`git revert <sha>`) and push, or run `railway up` from a checked-out good commit.

The volume preserves the data across a rollback. The volume is attached to the service, not to a deployment, so rolling the code back does not touch `/data`: the SQLite file, the Data Protection key ring and the uploads stay exactly as they are. Signed-in users therefore stay signed in.

One caveat is worth stating plainly: rolling back **code** does not roll back the **schema**. Migrations are forward-only, so if the bad deployment added a migration, the old code now runs against a newer schema. SQLite tolerates an extra table or an added nullable column, but a destructive migration (a dropped or renamed column) needs a new forward migration that restores it, or a restore of the database file from a backup you took before deploying.

9. Troubleshooting

Symptom	Likely cause and fix
Redirect loop, or HSTS never sent, or login and other form posts fail over HTTPS	The application is not reading the proxy headers, so it thinks every request is plain HTTP. <code>UsePlatformProxyHeaders()</code> must be the first middleware in <code>Program.cs</code> ; <code>ASPNETCORE_FORWARDEDHEADERS_ENABLED</code> is an Azure App Service feature and does nothing here. Check that <code>ASPNETCORE_ENVIRONMENT=Production</code> and that the container is reached through Railway's proxy, not port-forwarded directly.
Everyone is signed out after a redeploy	The Data Protection key ring is not on the volume. Set <code>DataProtection__KeysPath=/data/keys</code> , confirm the volume is mounted at <code>/data</code> , and redeploy. Sign-ins made before the fix cannot be recovered – the old keys are gone with the container.
<code>SqliteException: no such table: Courses</code> (or any other table)	The migrations did not run, or the application is looking at the wrong file. Check <code>Database__ApplyMigrationsOnStartup=true</code> , look for the <code>Applying database migrations</code> line in the log, and confirm <code>DATABASE_CONNECTION_STRING=Data Source=/data/learnhub.db</code> matches the volume mount path. A relative path resolves against the content root, which is <code>/app</code> in the image – not the volume.
No administrator account; <code>/Admin</code> always redirects to access denied	<code>Seed__AdminPassword</code> was missing on the first start of an empty database, so no administrator was created. Set it (and <code>Seed__AdminEmail</code>), restart, and read the <code>Administrator account ... created</code> line in the log. The account is only created when it does not already exist.
The build fails, or the image cannot be built on Railway	The .NET SDK version. <code>global.json</code> pins SDK <code>10.0.100</code> with <code>rollForward: latestFeature</code> and <code>Directory.Build.props</code> targets <code>net10.0</code> ; an older SDK cannot build the project, and the Docker build stage uses <code>mcr.microsoft.com/dotnet/sdk:10.0</code> . Run <code>dotnet --version</code> locally and install .NET 10 if it reports an older one.
<code>/health</code> never returns 200 and Railway keeps restarting the deployment	The database file is unreachable: the volume is missing or not mounted at <code>/data</code> . Run <code>railway volume list</code> , re-attach the volume, and check <code>railway logs</code> for the migration error.
HTTP 429 on login, registration or contact	The form rate limiter: 10 submissions per minute per IP address by default. Wait for the window to pass, or raise <code>RateLimiting__PermitLimit</code> for a demonstration.
The first request times out	A cold start: the image is starting, the migration is running and the demo catalogue is being seeded. Railway waits for the health check (300 s in <code>railway.json</code>); retry, and use <code>SMOKE_WAIT_SECONDS=300</code> with the smoke test.
The presentation site still opens the placeholder Railway URL	Run <code>scripts/set-app-url.sh https://<service>.up.railway.app</code> and then <code>firebase deploy --only hosting</code> .
Need the log	<code>railway logs</code> (or the Railway dashboard → service → Logs). Start-up prints migrations, seed counts and the administrator creation result.

10. Environment variables

Every variable is documented with placeholder values in `.env.example`, which is the only such file in Git. Copy it for local use, or paste the values into the Railway dashboard. **Never commit a filled-in `.env`, and never put a real password in this document, in `.env.example` or anywhere else in the repository:** the `.gitignore` excludes `.env`, `.env.*` (keeping only `.env.example`), `App_Data/`, `*.db` and `appsettings.*.local.json`, and `.dockerignore` keeps them out of the image as well.

Variable	Required	Example / placeholder	Purpose
<code>ASPNETCORE_ENVIRONMENT</code>	Yes	<code>Production</code>	Error handling, HSTS and cookie policy
<code>DATABASE_CONNECTION_STRING</code>	Yes	<code>Data Source=/data/learnhub.db</code>	SQLite file on the volume; wins over <code>ConnectionStrings:DefaultConnection</code>
<code>DataProtection__KeysPath</code>	Yes	<code>/data/keys</code>	Persistent key ring for the sign-in and anti-forgery cookies
<code>Storage__RootPath</code>	Yes	<code>/data/storage</code>	Private folder for uploads and thumbnails
<code>Seed__AdminEmail</code>	Yes	<code>admin@learnhub.local</code>	Administrator to create on the first start
<code>Seed__AdminPassword</code>	Yes	<code><a strong password you choose></code>	Without it there is no administrator and the admin area is unreachable
<code>Seed__DemoStudentEmail</code>	Recommended	<code>demo.student@example.com</code>	Demonstration student
<code>Seed__DemoStudentPassword</code>	Recommended	<code><another strong password></code>	Without it the demo student cannot sign in
<code>Seed__DemoData</code>	Recommended	<code>true</code>	Seed the demonstration catalogue into an empty database
<code>Database__ApplyMigrationsOnStartup</code>	Optional	<code>true</code> (default)	Apply pending migrations at start-up
<code>RateLimiting__PermitLimit</code> / <code>RateLimiting__WindowSeconds</code>	Optional	<code>10</code> / <code>60</code> (defaults)	Form submissions per IP address per window
<code>Site__RepositoryUrl</code>	Optional	<code>https://github.com/Khayit0ff/LearnHub</code>	Footer and About-page link
<code>Site__PresentationUrl</code>	Optional	<code>https://learnhub-wapp.web.app</code>	Link back to the presentation site
<code>PORT</code>	No – set by Railway	<code>8080</code> (image fallback only)	The port the application binds on <code>0.0.0.0</code>

A variable name uses a double underscore for each `:` in `appsettings.json`: `Seed__AdminPassword` is `Seed:AdminPassword`, and `DataProtection__KeysPath` is `DataProtection:KeysPath`. Both `appsettings.json` and `appsettings.Development.json` are committed and contain no secrets.

User Guide

A practical walkthrough of every part of the system, for demonstrating it and for writing the "User Guidance" section of the report.

Real screenshots of the running application are in [FirebaseLanding/assets/screens/](#) and on the project site at <https://learnhub-wapp.web.app>. Where a screenshot is named below, that file shows the page being described.

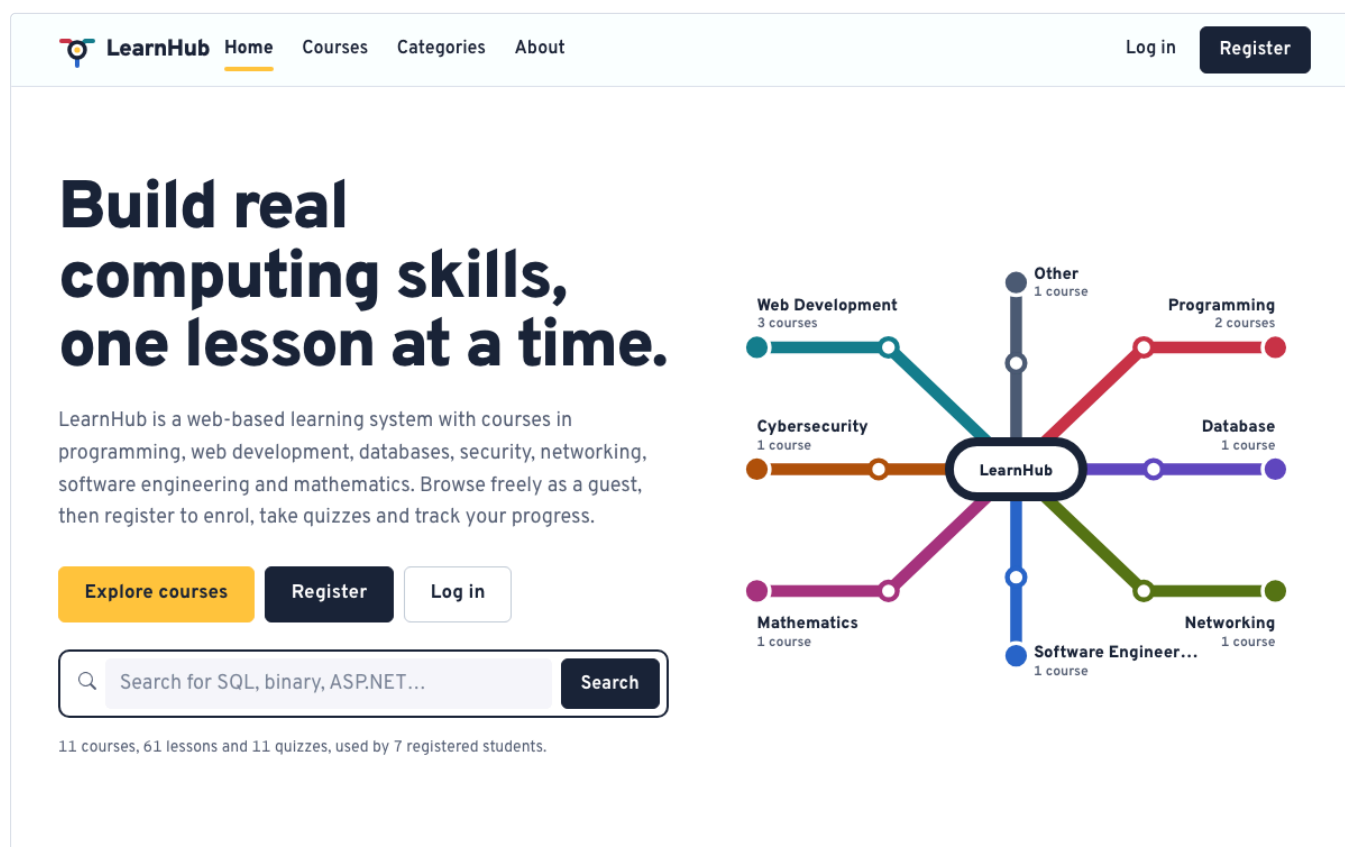
DEMO ONLY accounts. The administrator is `admin@learnhub.local` and the demonstration student is `demo.student@example.com`. Passwords are never committed: locally they are generated into `src/LearnHub/App_Data/demo-credentials.json` on the first run, and in production they come from the `Seed_AdminPassword` and `Seed_DemoStudentPassword` environment variables.

There are three kinds of user:

Role	Can do
Guest — not signed in	Browse and search the catalogue, read course details and free preview lessons, read the About, Contact and Privacy pages, register and log in.
Student — signed in	Everything a guest can, plus enrol, work through lessons, mark them complete, take quizzes, see results and progress, and manage their profile.
Administrator	Everything above, plus the whole administration area: users, courses, categories, resources, quizzes, questions, answers, enrolments, quiz results and the contact inbox.

1. Opening the website

Go to the application address. Locally that is `http://localhost:5000`; on Railway it is the service's public domain.



LearnHub Home Courses Categories About Log in Register

Build real computing skills, one lesson at a time.

LearnHub is a web-based learning system with courses in programming, web development, databases, security, networking, software engineering and mathematics. Browse freely as a guest, then register to enrol, take quizzes and track your progress.

Explore courses Register Log in

Search for SQL, binary, ASP.NET... Search

11 courses, 61 lessons and 11 quizzes, used by 7 registered students.

- Web Development 3 courses
- Cybersecurity 1 course
- Mathematics 1 course
- Software Engineer... 1 course
- Networking 1 course
- Database 1 course
- Programming 2 courses
- Other 1 course

The home page is the start of everything:

- **Hero** — the title, a short description of the platform, and the main call to action.
- **Search box** — searches course titles, descriptions, categories and lesson titles from any page.
- **Featured courses** — the courses students enrol in most often.
- **Browse by category** — every category with the number of courses in it, drawn as lines into the LearnHub hub.
- **Why learn with LearnHub** — what the platform offers.
- **Register call to action** — for visitors who are not signed in yet.

The navigation bar changes with who you are: a guest sees Home, Courses, Categories, About, Contact, Log in and Register; a signed-in student also sees My learning, Quizzes, Progress and Profile; an administrator sees a link to the administration area. Items a user may not use are not shown, and the pages behind

them are protected on the server as well.

2. Registering

Select **Register** in the navigation bar, or the register button on the home page.

Screenshot placeholder — registration page. Run the application and capture `Account/Register` to complete the report; the automated browser tests already exercise this page on three screen sizes.

1. Enter your full name, email address and a password.
2. Confirm the password.
3. Accept the terms, then select **Create account**.

The form validates as you type (client-side) **and** again on the server, which is what actually protects the database:

Field	Rule
Full name	Required, 2–80 characters
Email	Required, must be a valid address, and must not already be registered
Password	Required, at least 8 characters with an upper-case letter, a lower-case letter, a digit and a non-alphanumeric character
Confirm password	Must match the password exactly
Terms	Must be accepted

Mistakes are shown next to the field that caused them. A duplicate email is reported clearly rather than creating a second account. New accounts receive the **Student** role; a visitor can never register themselves as an administrator. Registration is rate-limited, so the form cannot be used to create accounts in bulk.

3. Logging in

Select **Log in** and enter your email address and password.

Screenshot placeholder — login page. Run the application and capture `Account/Login` to complete the report.

- On success a student lands on their dashboard and an administrator lands on the administration dashboard.
- On failure the page reports invalid credentials without revealing whether the email address exists.
- After several failed attempts the account is locked temporarily, which slows down password guessing.
- **Log out** in the navigation bar ends the session.

If you try to open a page you are not allowed to see while signed out, you are sent to the login page and returned to the page you asked for afterwards. If you are signed in but lack the role — a student opening an administration page, for example — you see a friendly "access denied" page instead.

4. Browsing courses

Select **Courses** in the navigation bar.


[Home](#)
[Courses](#)
[Categories](#)
[About](#)

[Log in](#)
[Register](#)

[Home](#) / [Courses](#) / [Web Development](#) / Modern HTML and CSS


[Web Development](#)

Modern HTML and CSS

Build accessible, responsive web pages with semantic HTML5 and modern CSS layout techniques.

Beginner
9 h
6 lessons
1 quiz
1 learner

Taught by **Ms. Sofia Reyes**



About this course

Every website starts with HTML for structure and CSS for presentation. This course teaches both the way professional front-end developers use them today.

You will write semantic HTML5 that works for screen readers and search engines, style pages with the box model, and build responsive layouts with flexbox and CSS grid. Along the way you will test pages on mobile, tablet and desktop screen sizes.

Start learning

Create a free student account to enrol, track your progress and take quizzes.

[Create a free account](#)

[Log in to enrol](#)

Every published course is listed as a card showing its cover, category, difficulty, duration, instructor and short description. Draft courses are never shown to guests or students.

Search — type into the search box and submit. The search runs on the server against course titles, descriptions, categories and lesson titles. When a match comes from a lesson title, those lessons are listed under the result, so it is never a mystery why a course appeared. Search terms are escaped before they reach the database.

Filter and sort — narrow by category and difficulty, and sort by newest, title, shortest or most popular. The filters combine with the search term and survive paging. The result count is always visible, and when nothing matches you get an empty state that explains what to try instead of a blank page.

Paging — the catalogue shows nine courses per page with pager links.

5. Course details and course route

Open any course to see its details: full description, what you will learn, instructor, difficulty, duration and the complete route of lessons and quizzes in order.

Each lesson shows its type (article, video, PDF, image, exercise, external link) and how long it takes. Lessons marked **Preview** are free for guests; the rest require enrolment. Selecting a locked lesson invites you to enrol or log in.

6. Enrolling

Enrol from the course details page with the **Enrol** button.

- You must be signed in; if you are not, you are sent to log in and returned to the course afterwards.
- Enrolling twice is impossible: the application checks first and the database enforces a unique index on `(UserId, CourseId)` as the final guarantee.
- A confirmation message appears and the course joins your learning list.
- Leave course** removes the enrolment after a confirmation step. Progress and quiz history for that course are removed with it.

7. Working through learning resources

Open **My learning** to see your enrolled courses, then open a lesson from the course route.


[Home](#)
[Courses](#)
[Categories](#)
[About](#)

[Log in](#)
[Register](#)

[Courses](#) / [Modern HTML and CSS](#) / How a web page is built

Course route

- How a web page is built
- HTML full tutorial for beginners
- The CSS box model
- CSS from zero to hero
- Responsive layouts with flexbox and grid
- MDN: Learn web development

Course overview

Article 10 min [Free preview](#)

How a web page is built

HTML gives meaning, CSS gives presentation and JavaScript adds behaviour.

Three languages, three jobs

- HTML** describes what the content *is*: headings, paragraphs, lists, forms, images.
- CSS** describes how it *looks*: colours, spacing, typography and layout.
- JavaScript** describes how it *behaves* when people interact with it.

Keeping the three separate makes pages easier to maintain and more accessible.

Semantic HTML5

```
<header>
  <nav aria-label="Main">...</nav>
</header>
<main>
  <article>
```

Learning resources come in several formats, and each is handled properly:

Type	How it is presented
Article	Formatted lesson text with headings, lists, code samples and tables.
Video	Embedded from a public provider, with a privacy-friendly embed and a fallback link.
PDF	Served through the application after an access check, with a download link.
Image	Displayed with alternative text.
Exercise	A task with a solution that can be revealed once you have attempted it.
External link	Opens the linked documentation in a new tab, marked as an external site.

A course outline sits beside the lesson so you always know where you are and what comes next. Select **Mark as complete** when you finish a lesson; the button then shows the completed state and the course progress bar updates immediately. Marking the same lesson complete twice is prevented by a unique index.

Uploaded files are **not** served as ordinary static files: they are streamed by the application after checking that you are enrolled (or that the lesson is a free preview). That is what stops a locked lesson being fetched by guessing its address.

8. Taking a quiz

Open **Quizzes** to see the quizzes available for the courses you are enrolled in, or start one from the course route.

- Select **Start quiz**. The attempt start time is recorded in a signed token posted with your answers, so it cannot be tampered with.
- Answer each question by choosing one option. A counter shows how many you have answered.
- Select **Submit answers**. The form is protected against cross-site request forgery, and the server re-checks every answer.
- Your attempt is marked immediately.

 LearnHub

[Home](#)

[Courses](#)

[My Learning](#)

[Quizzes](#)

[Progress](#)

AR

Profile

Log out

Quiz results

Every quiz you have submitted, newest first.

2

Attempts

2

Passed attempts

90%

Average score

Quiz	Course	Submitted	Score
MVC architecture quiz	ASP.NET Core MVC in Practice	Jul 19, 2026, 02:14 AM	80% Review
HTML and CSS essentials	Modern HTML and CSS	Jul 13, 2026, 10:14 AM	100% Review

 LearnHub
Structured courses, clear lessons and honest progress tracking for

Explore

All courses

Categories

Account

Profile

Privacy

Project

CT050-3-2-WAPP Web Applications group assignment Asia Pacific

The result page shows your score, the pass or fail decision against the quiz pass mark, and a review of every question: the option you chose, the correct option and an explanation. Your whole result history is on the same page, so you can see improvement over time.

Scores are stored on the attempt as a snapshot — the points earned, the points available and the percentage — so a later edit to the quiz never rewrites a result you already received. Passing a quiz counts towards the course completion percentage.

9. Checking your progress

Open **Progress** for the full picture, or **My learning** for the per-course list.


[Home](#)
[Courses](#)
[My Learning](#)
[Quizzes](#)
[Progress](#)

 Profile
 [Log out](#)

Welcome back, Aisyah

You have completed 60% of your enrolled courses. Keep going.

[Browse courses](#)

3
Enrolled courses

1
Completed courses

12
Lessons completed

2
Quizzes passed

8
Courses you can still join

Continue learning

Your next lesson in each course you are taking.

[All my courses](#)



Relational Database Design with SQL

12% complete

1 of 8 done

Resume

Next: SQL full database course



ASP.NET Core MVC in Practice

75% complete

6 of 8 done

Resume

Next: HTTP status codes reference

Recent activity

- 
Completed "Tables, keys and relationships"
 Jul 20, 2026, 08:14 AM
- 
Enrolled in Relational Database Design with SQL
 Jul 20, 2026, 12:14 AM
- 
Scored 80% in MVC architecture quiz
 Jul 19, 2026, 02:14 AM

The dashboard summarises your learning: courses in progress, lessons completed, quizzes taken and recent activity. Each course shows a progress bar calculated from the lessons you have marked complete and the quizzes you have passed, and the progress page breaks that down course by course with the counts behind the percentage.

A course reaches 100% when every published lesson is complete and every published quiz has been passed. Filters on **My learning** let you switch between all, in-progress and completed courses.

10. Updating your profile

Open **Profile** to view and change your details:

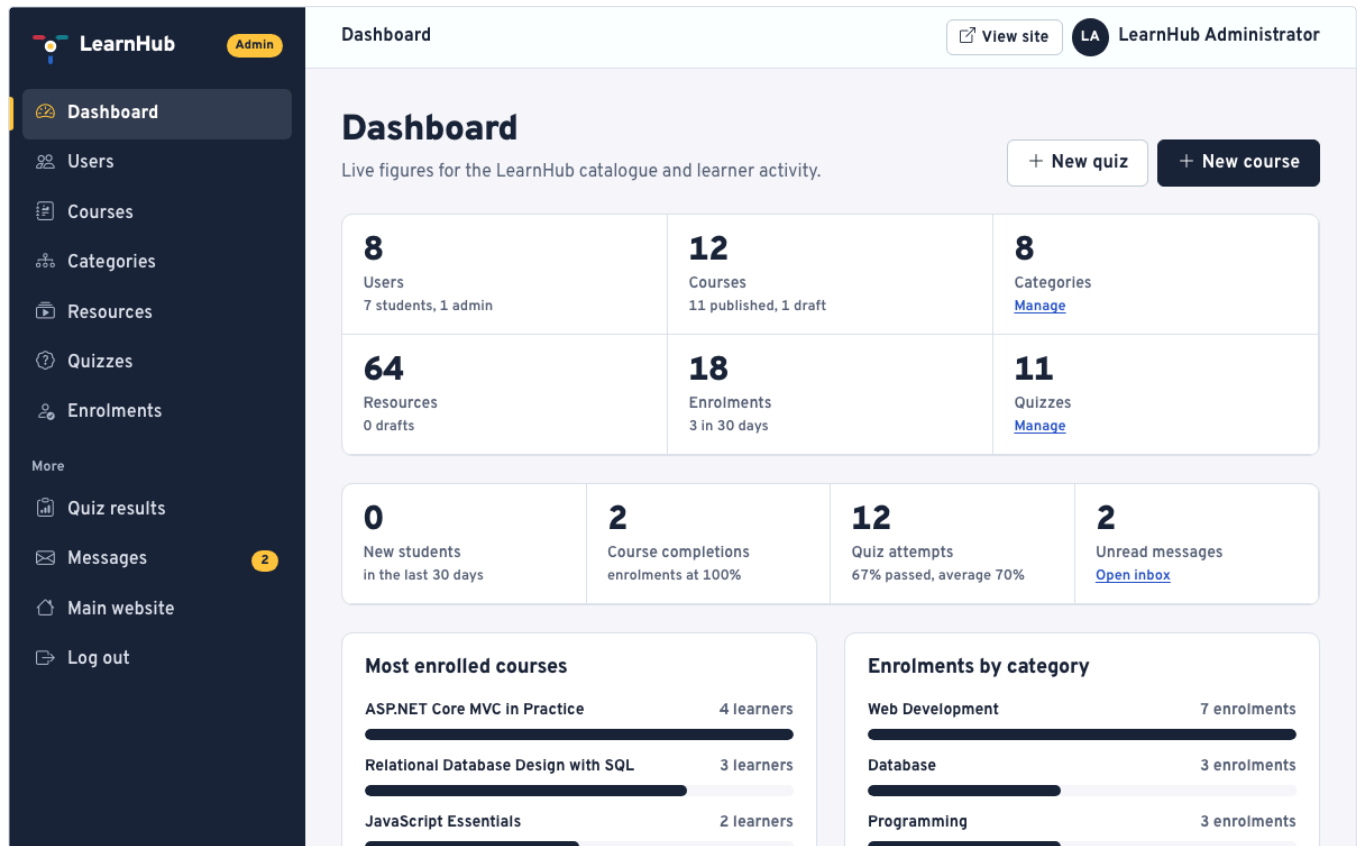
- Full name, email address and short biography.
- **Change password** asks for your current password and the new one twice, and applies the same strength rules as registration.

Your email address must remain unique, and the server re-validates everything it receives.

You **cannot** change your own role. Roles are managed by an administrator, so a student cannot promote themselves.

11. Administrator: signing in and the dashboard

Log in with an administrator account. The navigation bar shows a link to the administration area, and signing in takes you straight to the dashboard.



The dashboard reports live figures straight from the database — total users, courses, categories, resources, enrolments and quizzes — along with the most-enrolled courses, enrolments by category, recent activity and warnings about content that needs attention (a published course with no lessons, for example). Nothing on it is hard-coded.

Every administration controller carries `[Authorize(Roles = "Admin")]`. Hiding the menu is a convenience; the server is what actually refuses access, so a student who types an admin address is turned away.

12. Administrator: managing courses and categories

Open **Courses** in the administration area.

- **Create** a course with its title, short and full description, learning outcomes, instructor, category, difficulty, duration, cover image and published state.
- **Edit** any of those values at any time.
- **Publish or unpublish** — a draft is invisible to guests and students but stays in the administration area.
- **Delete** a course after a confirmation page that names what will be removed with it (lessons, quizzes, enrolments and attempts).

Categories works the same way: create, edit and delete, each with a name, description and an icon chosen from a validated list. A category that still has courses cannot be deleted — the database restricts it and the page explains why rather than failing with an error.

Validation is enforced on the server as well as in the browser: required fields, length limits, a duration greater than zero, a valid category, and an uploaded cover image checked for type, size and a genuine image signature.

13. Administrator: managing learning resources

Open **Resources** to see every lesson, filterable by course and type.

For each resource you can set the title, summary, type, estimated minutes, order in the course, whether it is a free preview and whether it is published, and then supply the body appropriate to the type:

- **Article** — the lesson text.
- **Video** — the address of a public video, which is parsed and validated before being embedded.
- **PDF or image** — an uploaded file, validated for extension, content type, size and file signature.
- **External link** — a URL that must be a valid absolute http or https address.
- **Exercise** — the task and an optional solution.

Uploaded files are stored outside `wwwroot` and streamed only to users entitled to see them.

14. Administrator: managing quizzes, questions and answers

Open **Quizzes** to see the quizzes for each course.

- **Create a quiz** with a title, description, pass mark between 0 and 100, and a published state.
- **Add questions** with the question text, an optional explanation, points and their order.
- **Add answer options** to each question — between two and six, with exactly one marked correct. The form refuses to save a question that breaks either rule.
- **Edit or delete** any of them. Deleting a question that students have already answered is handled explicitly: the application tells you what will happen to that answer history instead of destroying it silently.

15. Administrator: users, enrolments, results and messages

Users lists every account with its roles. You can:

- View a user's details, enrolments and quiz results.
- Change a user's role between Student and Administrator.
- Deactivate an account so it can no longer sign in, or reactivate it.
- Delete an account, with safeguards: you cannot delete your own account or remove the last administrator, so the system can never be left without somebody able to administer it.

Enrolments lists who is enrolled in what, with progress, and allows an enrolment to be created or removed on a student's behalf.

Quiz results lists every submitted attempt with its score, pass or fail state and the answers given, and allows an individual attempt to be removed.

Messages is the inbox for the public contact form, with read and unread states.

16. Quick demonstration route

A short sequence that shows every requirement in a few minutes:

#	Do this	Shows
1	Open the home page and search for <code>sql</code> .	Public landing page, server-side search
2	Filter the catalogue by category and open a course.	Categories, filtering, course details
3	Open a free preview lesson as a guest.	Multimedia resources, guest access
4	Register a new account with a weak password first.	Client and server validation, duplicate prevention
5	Register properly, then enrol in the course.	Registration, authentication, enrolment
6	Open a lesson, mark it complete, watch the progress bar move.	Resources, progress tracking
7	Take a quiz, submit, and read the explained result.	Quiz, grading, results
8	Open Progress and Profile, change the profile, change the password.	Member area, profile, validation
9	Try to open <code>/Admin/Dashboard</code> as the student.	Authorisation is enforced on the server
10	Log in as the administrator and open the dashboard.	Role-based access, live statistics
11	Create a category, a course, a lesson and a quiz question.	Full CRUD, validation, database writes
12	Edit and delete what you created, and manage a user's role.	Update, delete, user management

17. Error pages

The application never shows a stack trace, a connection string or a file path to a visitor:

Situation	What the user sees
A page does not exist	A friendly "not found" page, still returning HTTP 404 so search engines and monitoring are not misled.
Signed in without the right role	An "access denied" page explaining that the account lacks permission.
A form fails validation	The same page with the values preserved and a clear message beside each problem field.
An unexpected server error (production)	A generic error page with a reference; the detail goes to the server log only.
The database is unreachable	<code>/health</code> reports unhealthy so the platform can react, and users get the error page rather than a hang.

18. Accessibility and smaller screens

- Every page is keyboard navigable, with a **skip to content** link and a visible focus outline.
- Form controls have real labels, images carry alternative text, and headings follow a sensible order.
- Text and interface colours meet WCAG 2.2 AA contrast, and animation is disabled for visitors who ask for reduced motion.
- The layout is responsive across desktop, tablet and phone. On a narrow screen the navigation collapses into a menu and the category map is replaced by a list of category links.

 LearnHub

Build real computing skills, one lesson at a time.

LearnHub is a web-based learning system with courses in programming, web development, databases, security, networking, software engineering and mathematics. Browse freely as a guest, then register to enrol, take quizzes and track your progress.

[Explore courses](#)[Register](#)

[Log in](#)

[Search](#)

11 courses, 61 lessons and 11 quizzes, used by 7 registered students.

Featured courses

The Playwright browser tests check these journeys at 1440×900, 768×1024 and a phone viewport, and run axe-core accessibility scans on every page.

Report Notes

Content for the CT050-3-2-WAPP group report. Paste each section into the university template; diagrams are Mermaid (render them on GitHub or export them as images) and the screenshots used below are the ones the presentation site serves from `FirestoreLanding/assets/screens/`.

Title page

Title	LearnHub – A Web-Based Learning Management System
Module	CT050-3-2-WAPP Web Applications
Assessment	Group assignment
Institution	Asia Pacific University of Technology & Innovation (APU)
Team members	Khaytboy Khayrullaev (TP072305) — carried out individually
Lecturer	<i>Lecturer's name</i>
Intake	<i>Intake code</i>
Submission date	<i>Date</i>
Repository	https://github.com/KhayitOf/LearnHub
Presentation site	https://learnhub-wapp.web.app (Firebase Hosting, live)

Abstract

LearnHub is a web-based learning management system for computing students, built with ASP.NET Core MVC on .NET 10, C# and Razor views, Bootstrap 5.3, Entity Framework Core 10 and ASP.NET Core Identity. Guests explore a catalogue of courses and free preview lessons; registered students enrol, study articles, videos, PDF documents and images, mark lessons complete, take quizzes with explained answers and follow their progress on a data-driven dashboard. Administrators manage courses, categories, learning resources, quizzes, enrolments, users and contact messages in a protected admin area. One SQLite database with a single code-first migration set serves every environment, and on the hosting platform its file lives on a mounted volume so it survives each redeploy. Security follows the OWASP Top 10:2025: role-based authorisation, antiforgery tokens, encoded output with a strict Content Security Policy, ownership checks, validated uploads, rate limiting and no committed credentials. Quality is assured by 211 automated .NET tests, 42 browser and accessibility checks on three screen sizes, and continuous integration that also builds the Docker image and runs it the way the host will. The static presentation site is live on Firebase Hosting and links to the application, because Firebase cannot execute ASP.NET Core.

1. Introduction

1.1 Background

Students learning programming, web development, databases, security, networking and cloud computing usually rely on scattered resources: slides, videos, articles and cheat sheets. The material rarely has a clear order, gives no feedback on understanding and offers no view of progress. Lecturers who want to share structured material need a tool that is simple to maintain and safe for students to use.

1.2 Problem statement

1. Learners do not know what to study next or how far they are from finishing a topic.
2. Reading and watching do not show whether a topic was understood.
3. Content managers lack one secure place to publish, update and retire courses and to see learner activity.

1.3 Proposed solution

LearnHub organises learning into courses made of ordered lessons and quizzes. Guests can explore freely; students enrol, study and receive immediate, explained quiz feedback; progress is calculated from real activity and shown on a dashboard. Administrators use a separate, protected area to manage everything. The whole interface follows one metaphor: LearnHub is an **interchange** where each category is a coloured line, each course is a route and each lesson is a stop.

1.4 Report structure

Section 2 covers planning. Section 3 presents the design (architecture, use cases, flowcharts, wireframes, navigation and database). Section 4 explains the implementation. Sections 5 and 6 describe testing and deployment, section 7 walks through screenshots, and sections 8 to 10 evaluate the work, propose enhancements and conclude.

2. Project planning

2.1 Mission statement

LearnHub gives every computing student a clear route through each subject, with honest feedback at every stop, and gives administrators one simple place to

keep that route up to date.

2.2 Objectives

#	Objective	Achieved	Evidence
O1	Public catalogue with search, filters, details and preview lessons	Yes	Public site tests; screenshots 7.1–7.3
O2	Continuous student journey: register, enrol, study, quiz, result	Yes	<code>StudentJourneyTests</code> ; Playwright student scenario
O3	Data-driven student dashboard	Yes	Dashboard figures, progress, results, activity, recommendations (7.5)
O4	Full admin CRUD for content, enrolments and users	Yes	<code>AdminManagementTests</code> ; admin browser tests
O5	Protection against OWASP Top 10 risks	Yes	Security review; attack-style automated tests
O6	Responsive and accessible on phone, tablet and desktop	Yes	42 browser checks at three sizes; no WCAG 2.2 A/AA violations from axe-core
O7	Automated testing and container deployment with a presentation site	Partly – Firebase Hosting is live; the Railway service is configured but not yet deployed	Three CI jobs; https://learnhub-wapp.web.app

2.3 Audience modelling

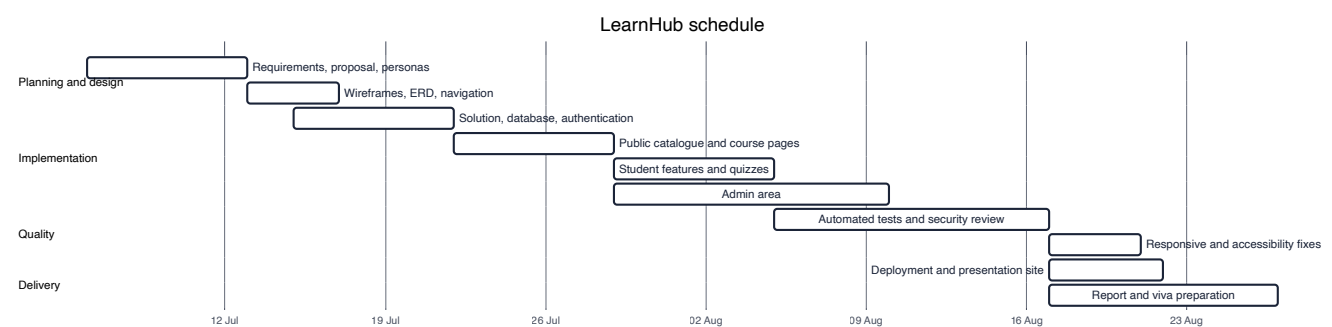
Persona	Profile	Needs	How LearnHub responds
Nurul, first-year student	19, Diploma in IT, studies on a mid-range Android phone	A clear order of topics; checking understanding before tests	Ordered course routes, a Resume button, quizzes with explanations, mobile-first layout
Daniel, career switcher	31, full-time job, studies in the evenings on a laptop	Practical skills and visible progress in limited time	Difficulty and duration on every course, progress bars, "Continue learning", PDF cheat sheets
Ms. Priya, lecturer and administrator	42, manages networking and cloud modules on a desktop	Quick publishing, safe editing, overview of learner activity	Admin dashboard, publish/unpublish, impact-aware delete confirmations, validated uploads, quiz result reports

2.4 Scope

In scope: public catalogue and pages, registration and login, student dashboard, enrolment, lessons of five types, completion tracking, quizzes and results, profile management, a complete admin area, security controls, responsive and accessible design, automated tests, CI/CD, a Docker image that runs on Railway and a Firebase Hosting presentation site.

Out of scope for version 1.0: payments, certificates, chat, forums, live classes, an AI tutor, an instructor self-service role, email delivery (confirmation and password reset), native apps and multiple languages.

2.5 Schedule



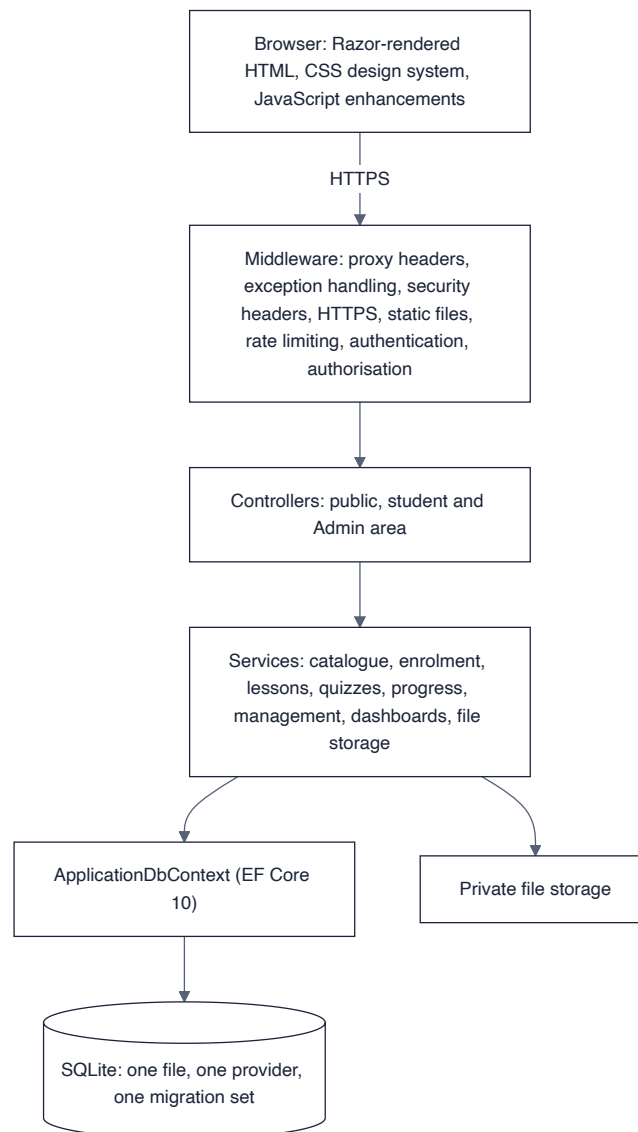
2.6 Team roles

This submission was completed by one member working alone, so a single person covered every role: architecture, database design, authentication, the public site, the student area, the administration area, the security review, the automated tests, the documentation and the deployment. The areas covered and the way the work was organised are set out in [TEAM.md](#); the full proposal is in [PROPOSAL.md](#).

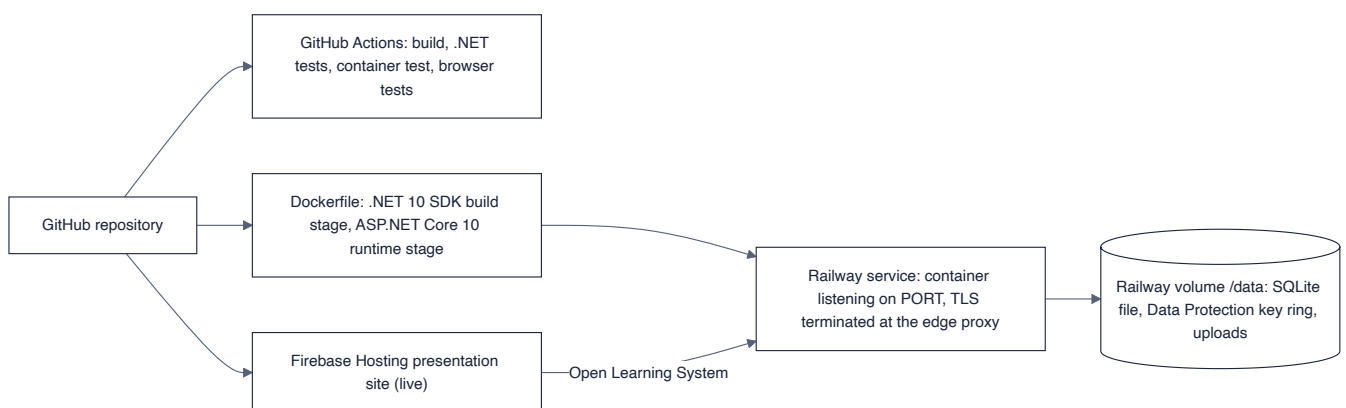
3. System design

3.1 Architecture

LearnHub is a single ASP.NET Core MVC application with clear layers. Controllers handle HTTP concerns, services contain business rules and queries, and Entity Framework Core accesses the database. Views receive view models only, and forms bind to view models, never to entities.



Deployment view:

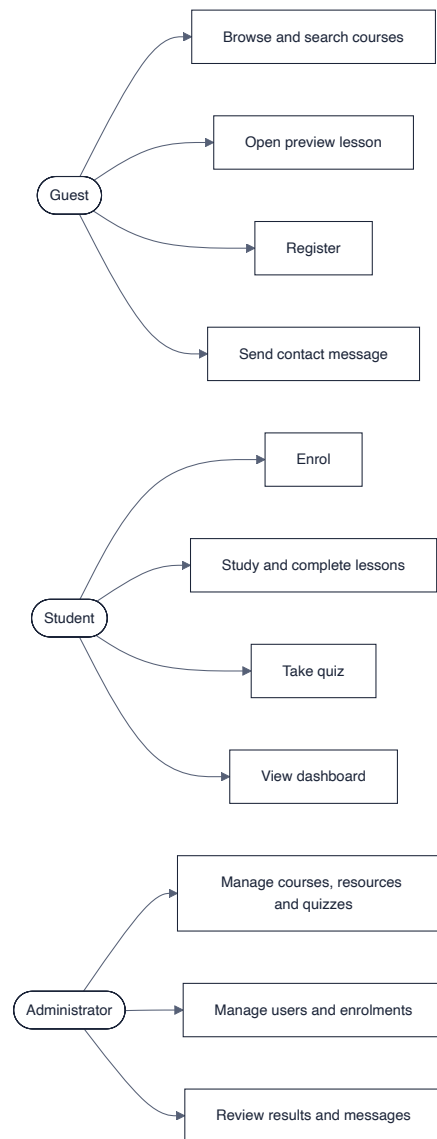


The detailed design, including every route, is in [ARCHITECTURE.md](#).

3.2 Use cases

ID	Use case	Actor
UC-01–02	Register, log in and log out	Guest, student, administrator
UC-03–05	Browse, search and filter courses; view details; open a preview lesson	Guest
UC-06	Send a contact message	Guest
UC-07–09	Enrol, leave, study lessons and mark them complete	Student

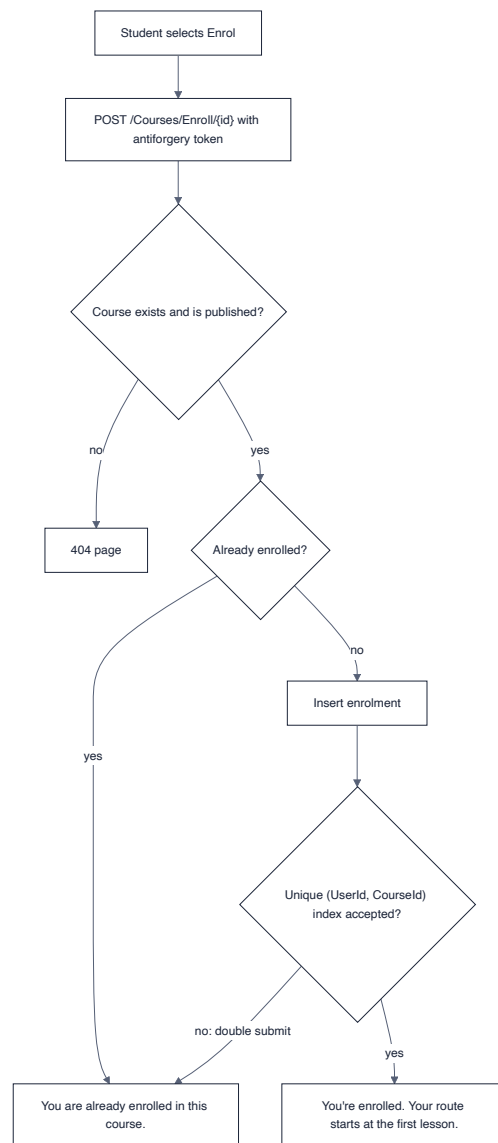
ID	Use case	Actor
UC-10–13	Take quizzes, review results, use the dashboard, manage profile	Student
UC-14–22	Admin dashboard; manage courses, categories, resources, quizzes and questions, enrolments, users, attempts and messages	Administrator



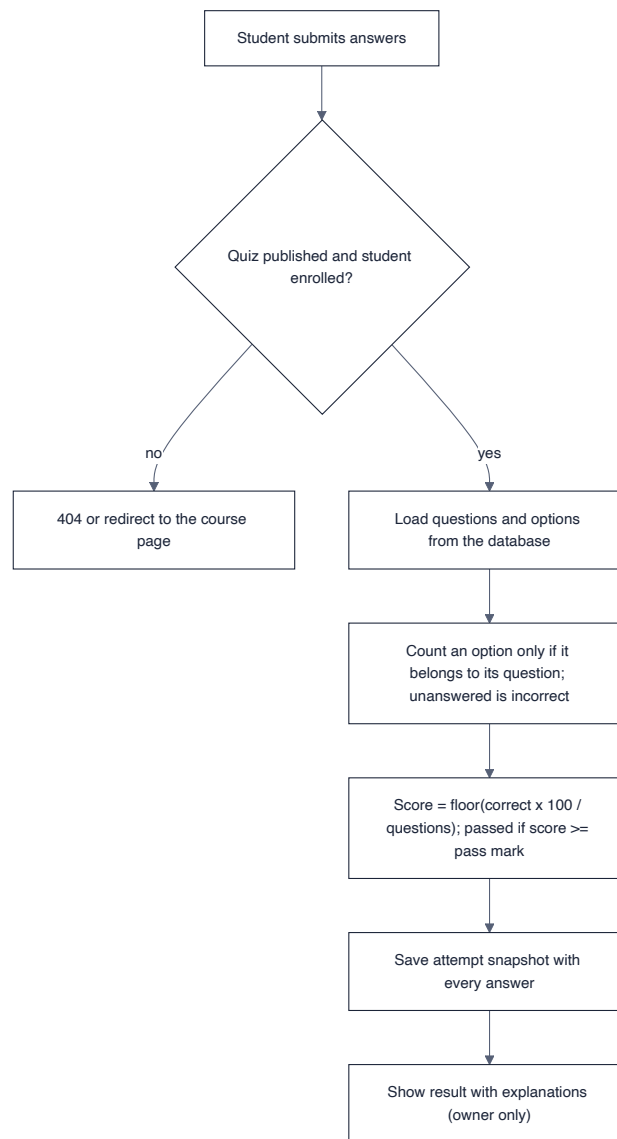
Every use case has preconditions, main and alternative flows and its implementing controller in [USE_CASES.md](#).

3.3 Flowcharts

Enrolment (from `CoursesController.Enroll` and `EnrollmentService`):



Quiz submission and grading (from `QuizzesController`, `QuizService` and `QuizGrader`):

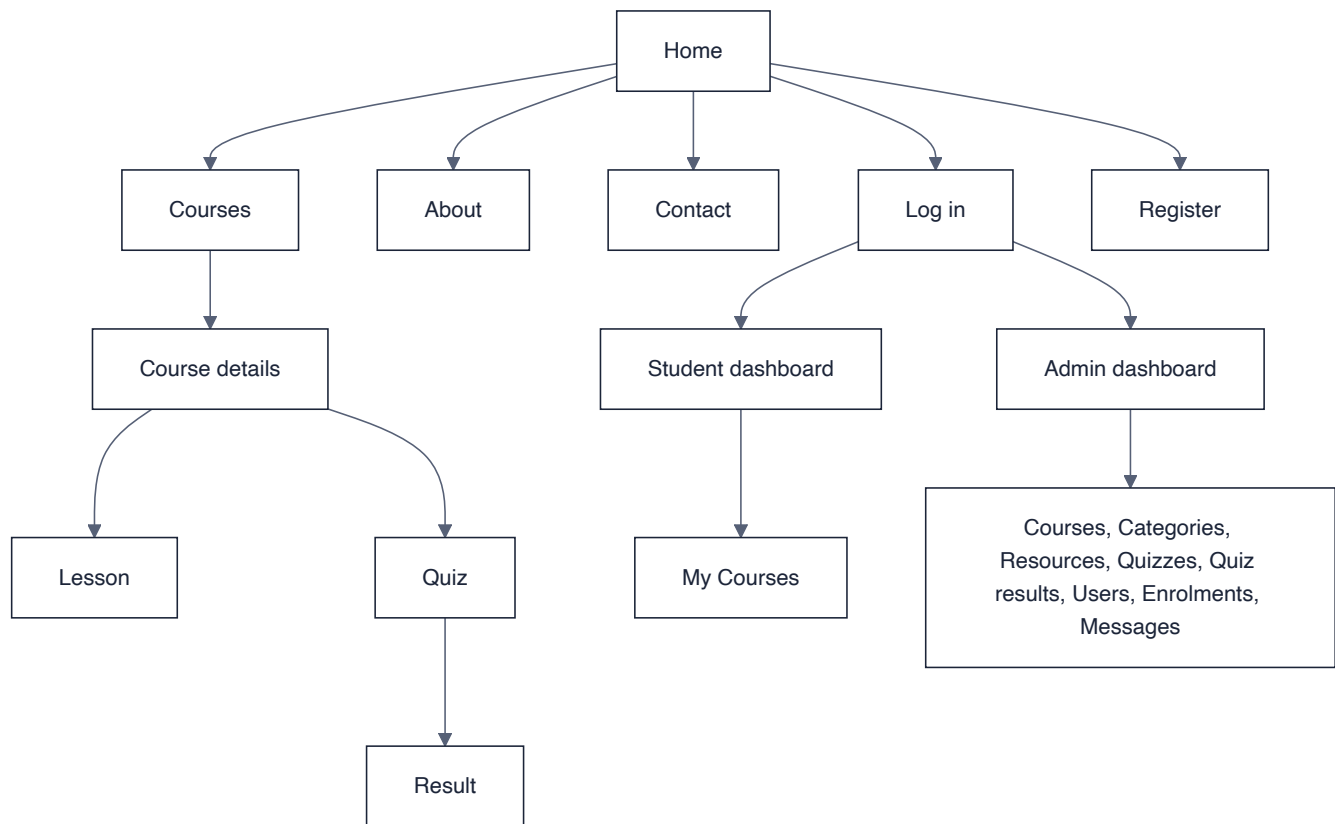


Eleven flowcharts, including the request pipeline, login, lesson access, course deletion and upload validation, are in [FLOWCHARTS.md](#).

3.4 Wireframes

Low-fidelity wireframes were produced for the home page, catalogue, course details, lesson player, quiz and result, student dashboard, authentication forms, admin dashboard, admin list pages and admin forms, each in desktop and phone variants. They fixed the key layout decisions early: a two-column hero with the category map, a sidebar of coloured category filters in the catalogue, a sticky enrolment panel beside the course route, a sticky lesson outline, and an admin shell with a dark sidebar that becomes an off-canvas menu on small screens. See [WIREFRAMES.md](#).

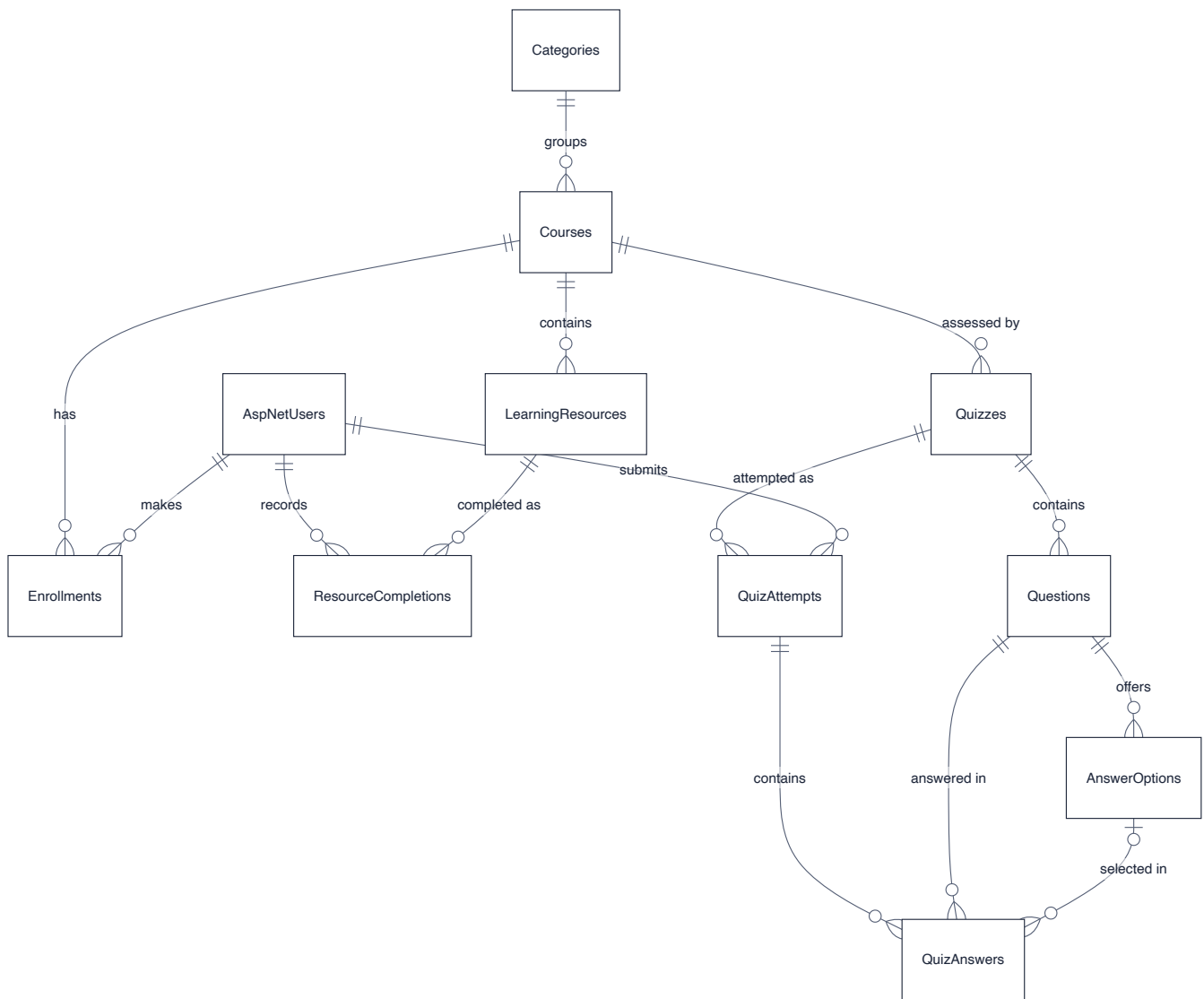
3.5 Navigation structure



Guests see Home, Courses, About, Contact, Log in and Register; students also see My Courses, Dashboard and a user menu; administrators see an Admin link and use the admin sidebar. Breadcrumbs appear on nested pages. The complete site map, route table and redirect behaviour are in [NAVIGATION.md](#).

3.6 Database design

The database has eleven application tables — `Categories`, `Courses`, `LearningResources`, `Enrollments`, `ResourceCompletions`, `Quizzes`, `Questions`, `AnswerOptions`, `QuizAttempts`, `QuizAnswers` and `ContactMessages` — plus the seven ASP.NET Core Identity tables. One code-first migration creates all eighteen tables, and SQLite is the only provider, so there is one context and one migration set to keep in step rather than one schema per dialect.



Key design decisions:

- **Integrity:** primary and foreign keys, required columns with lengths from one `FieldLengths` class, unique indexes (category name; one enrolment per student and course; one completion per student and resource; one answer per question per attempt) and check constraints (positive course duration, completion percentage, pass mark and score inside their ranges, question points between one and one hundred, correct count never above the question count).
- **Delete behaviour:** deleting a category is restricted while courses exist; a course cascades to its resources, quizzes, attempts and enrolments; and quiz answers restrict deletion of the questions and options they refer to, so editing a quiz can never silently erase a student's answer history — the services delete the dependent answers first, inside one transaction.
- **Truthful history:** an attempt stores a snapshot of `CorrectCount`, `QuestionCount`, `Score`, `MaxScore`, `ScorePercent` and `Passed`; progress is calculated rather than stored; deactivation uses Identity lockout instead of a separate flag.
- **Location:** the connection string is read from `DATABASE_CONNECTION_STRING`, falling back to `ConnectionStrings:DefaultConnection`; a relative path is resolved against the content root and its folder is created automatically, and on the hosting platform the path points at the persistent volume (`Data Source=/data/learnhub.db`).
- **Time:** all dates are stored in UTC with automatic `CreatedAt` and `UpdatedAt`.

The full diagram with every column, index and constraint is in [ERD.md](#), and the reasoning behind each decision is in [DATABASE.md](#).

4. Implementation

4.1 HTML5 and Razor

Pages use semantic HTML5 landmarks (`header`, `nav`, `main`, `section`, `aside`, `footer`), headings in order, and labelled forms. Razor layouts (`Views/Shared/_Layout.cshtml`, `Areas/Admin/Views/Shared/_AdminLayout.cshtml`) provide the page frame; partials such as `_CourseCard`, `_Pagination` and `_QuizReview` avoid repetition. Built-in tag helpers generate URLs, validation attributes and antiforgery tokens, and two custom tag helpers render navigation state and course covers.

```
<div class="mb-3">
  <label asp-for="Password" class="form-label"></label>
  <input asp-for="Password" class="form-control" autocomplete="new-password" data-password-meter="password-meter" aria-describedby="password-help" />
  <div class="password-meter" id="password-meter" data-score="0" aria-hidden="true"><span></span><span></span><span></span><span></span></div>
  <div class="form-text" aria-live="polite"></div>
  <div id="password-help" class="form-text field-hint">@PasswordRules.Description</div>
  <span asp-validation-for="Password"></span>
</div>
```

Figure: password field in `Views/Account/Register.cshtml` – the label, input type, maximum length and validation attributes all come from the view model through tag helpers.

4.2 CSS3 and the design system

`wwwroot/css/site.css` defines the design system as CSS custom properties and builds every component from them; the admin shell in `admin.css` reuses the same tokens.

```
--lh-ink: #1b2438;
--lh-paper: #f5f7fa;
--lh-surface: #ffffff;
--lh-rail: #dde3eb;
--lh-muted: #566076;
--lh-signal: #ffc53d;
--lh-line-0: #4b5a73;
--lh-line-1: #c9344a;
--lh-line-2: #157f8e;
--lh-font: "Overpass", "Segoe UI", system-ui, -apple-system, Roboto, "Helvetica Neue", Arial, sans-serif;
```

Figure: an excerpt of the design tokens in `wwwroot/css/site.css`.

Each category receives one of eight line colours from its id, used on course covers, category filters, course routes and the home page map. Layouts are mobile-first with CSS Grid and Flexbox, and key responsive decisions are documented in the wireframes. Accessibility is built in: a visible yellow focus halo on every interactive element, skip links, contrast checked against WCAG AA, reduced-motion support, and focusable scroll regions for wide code and tables.

4.3 JavaScript

JavaScript is used only for progressive enhancement, so every feature still works when it is disabled. Examples in `wwwroot/js/site.js`: local date formatting, character counters, a password strength meter, image previews before upload, type-specific fields on the resource form, the answer option editor, confirmation prompts and the quiz helper below.

```
const answeredCount = () => questions.filter((q) => q.querySelector("input:checked")).length;
const update = () => {
  if (counter) {
    counter.textContent = `${answeredCount()} of ${questions.length} answered`;
  }
};

form.addEventListener("change", update);
form.addEventListener("submit", (event) => {
  const unanswered = questions.length - answeredCount();
  if (unanswered > 0) {
    const noun = unanswered === 1 ? "question is" : "questions are";
    if (!window.confirm(`${unanswered} ${noun} unanswered and will be marked incorrect. Submit anyway?`)) {
      event.preventDefault();
    }
  }
});
```

Figure: the quiz answered counter and unanswered warning in `wwwroot/js/site.js`.

4.4 Validation

Validation is defined once on view models and enforced twice: in the browser through jQuery Validation Unobtrusive and on the server through model binding and `ModelState`.

```
[Required(ErrorMessage = "Please choose a password.")]
[StringLength(PasswordRules.MaximumLength, MinimumLength = PasswordRules.MinimumLength, ErrorMessage = "Password must be between {2} and {1}
characters.")]
[RegularExpression(PasswordRules.Pattern, ErrorMessage = PasswordRules.Description)]
[DataType(DataType.Password)]
public string Password { get; set; } = string.Empty;

[Required(ErrorMessage = "Please confirm your password.")]
[Compare(nameof(Password), ErrorMessage = "The passwords do not match.")]
[DataType(DataType.Password)]
```

Figure: password rules in `ViewModels/Account/AccountViewModels.cs`.

Custom attributes extend this: `MustBeTrueAttribute` requires the terms checkbox, and `MaxFileSizeAttribute` and `AllowedExtensionsAttribute` check uploads; all three also emit client-side rules. Rules that need the database are checked in services: unique category names, two to six answer options with exactly one correct, a question before publishing a quiz, enrolment only in published courses and protection of the last administrator. Upload validation on the server goes further than the browser can: it checks the file signature and the real number of bytes written.

4.5 Database and Entity Framework Core

`ApplicationDbContext` holds the whole model, and one provider is registered for every environment: SQLite, in `Data/DatabaseServiceCollectionExtensions.cs`. There is a single migrations folder, and `Data/DesignTimeDbContextFactory.cs` is the only design-time factory `dotnet ef` needs. Relationships and constraints use the Fluent API:

```
public void Configure(EntityTypeBuilder<Enrollment> builder)
{
    // A student can enrol in a course only once.
    builder.HasIndex(e => new { e.UserId, e.CourseId }).IsUnique();

    builder.HasOne(e => e.User)
        .WithMany(u => u.Enrollments)
        .HasForeignKey(e => e.UserId)
        .OnDelete(DeleteBehavior.Cascade);

    builder.HasOne(e => e.Course)
        .WithMany(c => c.Enrollments)
        .HasForeignKey(e => e.CourseId)
        .OnDelete(DeleteBehavior.Cascade);
}
```

Figure: enrolment configuration in `Data/Configurations/UserAndCatalogConfigurations.cs`.

Queries use LINQ with projections to view models and `AsNoTracking` for reading. Multi-step changes run inside the provider's execution strategy, so the unit stays correct if a retrying strategy is ever configured:

```
public static Task InTransactionAsync(this ApplicationDbContext db, Func<Task> operation, CancellationToken cancellationToken = default)
{
    var strategy = db.Database.CreateExecutionStrategy();
    return strategy.ExecuteAsync(async () =>
    {
        await using var transaction = await db.Database.BeginTransactionAsync(cancellationToken);
        await operation();
        await transaction.CommitAsync(cancellationToken);
    });
}
```

Figure: `Data/TransactionExtensions.cs`.

At start-up `DatabaseInitializer` applies pending migrations, creates the `Admin` and `Student` roles, creates the administrator from configuration and, for an empty database, seeds the demonstration catalogue: 8 categories, 12 courses (11 published and 1 draft), 64 learning resources (including original PDF cheat sheets and diagrams), 11 quizzes, 54 questions and 216 answer options, together with 7 learners whose enrolments, completions and attempts make the dashboards and progress pages realistic. All four data operations are demonstrated: insert (enrolments, attempts, admin create forms), display (catalogue, dashboards), update (edit forms, completion toggles) and delete (admin delete pages, leaving a course).

4.6 Authentication and authorisation

ASP.NET Core Identity provides password hashing, lockout (five failures, fifteen minutes), roles and cookie authentication. The cookie is `HttpOnly`, `SameSite=Lax` and `Secure` in production, and the security stamp is re-validated every five minutes so role changes and deactivations take effect quickly. A claims factory adds the user's full name for the interface.

```
[Area("Admin")]
[Authorize(Roles = AppRoles.Admin)]
public abstract class AdminControllerBase : Controller
{
    protected string CurrentUserId => User.GetRequiredUserId();

    protected void Success(string message) => TempData.SetStatus(message);

    protected void Failure(string message) => TempData.SetStatus(message, StatusKind.Danger);
}
```

Figure: every admin controller inherits this base class (`Areas/Admin/Controllers/AdminControllerBase.cs`).

Student controllers carry `[Authorize(Roles = AppRoles.Student)]`. Login redirects only to local return URLs. Administrators cannot change, deactivate or delete their own account, and the last active administrator is protected. No password is stored in the repository: in Development a strong random password is generated into the git-ignored `App_Data/demo-credentials.json`, locally it can be overridden with user secrets, and in production it must come from the `Seed__AdminPassword` variable.

4.7 Security

Threat	Control in LearnHub
Broken access control and IDOR	Role attributes, admin base class, ownership and enrolment checks in services, files streamed after access checks, 404 for other users' results
Cross-site request forgery	Global antiforgery validation on every unsafe request; state changes only through POST forms
Cross-site scripting	Razor encoding, encode-first lesson renderer, strict Content Security Policy without inline scripts
SQL injection	EF Core parameterised queries; escaped LIKE patterns
Overposting	View models only
Malicious uploads	Extension allow-list, size limits, file signature checks, random names, storage outside <code>wwwroot</code>
Brute force and spam	Lockout, generic login errors, rate limiting, contact form honeypot
Misconfiguration	HTTPS and HSTS, security headers, friendly error pages, secure cookies
Secrets exposure	User secrets locally; placeholder-only <code>.env.example</code> with a git-ignored <code>.env</code> ; hosting variables in production; no demo password in Git

The review against the OWASP Top 10:2025 with test evidence is in [SECURITY.md](#).

4.8 Multimedia

- **Images:** ten original SVG course covers built from each category's line colour and an icon, PNG teaching diagrams (for example the MVC request flow, CSS box model and OSI model), and administrator-uploaded course covers.
- **Video:** lessons embed YouTube or Vimeo videos through a strict URL parser and privacy-enhanced players, allowed by the CSP `frame-src` directive.
- **Documents:** original PDF cheat sheets (C#, HTTP status codes, SQL joins, OWASP Top 10:2025, subnetting) open inline for enrolled students through a protected download action.
- **Interactive graphics:** the home page category map is an accessible SVG whose lines are links.

4.9 Error handling and logging

Production uses the exception handler and status code pages, so users see branded 404, 403 and 500 pages without technical details, while developers get the detailed exception page locally. Services turn expected problems (not found, validation, unique-index races) into `OperationResult` values and friendly messages. Structured logging records important events such as registrations, sign-ins and blocked sign-ins, administrator creation, enrolments, quiz submissions, deletions and rejected uploads. A `/health` endpoint checks the database connection for monitoring and smoke tests.

5. Testing

LearnHub combines several test levels, all run by GitHub Actions on every push:

Level	Scope	Result on 14 September 2026
Unit, service and integration (xUnit v3, <code>WebApplicationFactory</code>)	Grading, validation, content safety, uploads, catalogue and quiz services, public site, authentication, authorisation, student journey, admin CRUD, link crawl	211 of 211 passed on SQLite
Container (Docker image, production settings)	The image the host will run, started with <code>PORT=8080</code> and a volume mounted at <code>/data</code> , then smoke tested, checked for clean logs and restarted	Passed; migrations were applied once and were not re-applied after the restart
Browser (Playwright)	14 scenarios — public pages, the student journey, the admin journey and accessibility scans — on desktop, tablet and phone	42 of 42 passed (14 scenarios × 3 viewports)
Accessibility (axe-core)	WCAG 2.2 A and AA rules on 26 page views per screen size, inside the browser suite	9 of 9 passed, no violations

Representative test cases:

Test case	Expected	Result
Register with a weak password	Field-level error with the password rule	Pass
Five wrong passwords	Account locked for 15 minutes	Pass
Login with an external return URL	Redirect to the user's own dashboard	Pass
Guest opens a locked lesson	Redirect to login	Pass
Enrol twice (including simultaneous requests)	"You are already enrolled in this course."	Pass
Submit a quiz option from another question	Counted as incorrect	Pass
Student opens another student's result	404	Pass
Student posts to an admin action with a valid token	Refused	Pass
Upload HTML disguised as a PDF	Rejected; nothing stored	Pass
Delete a category that still has courses	Refused with an explanation	Pass
Delete a course with quiz attempts	Everything removed in order, with the dependent quiz answers deleted first	Pass
Crawl every page as guest, student and admin	No broken links or server errors	Pass

Testing found and fixed defects that no single level would have caught, including a query that SQLite could not translate, a relationship error when deleting courses, form pages that failed behind a TLS-terminating proxy until forwarded headers were configured explicitly, a migration that was never regenerated after a model change and broke 126 tests until it was recreated, pages that scrolled sideways on phones and tablets, an invisible admin sidebar, overlapping map labels, two accessibility problems and 91 build warnings in the test code. The strategy, full inventory, defect log and manual checklist are in [TESTING.md](#).

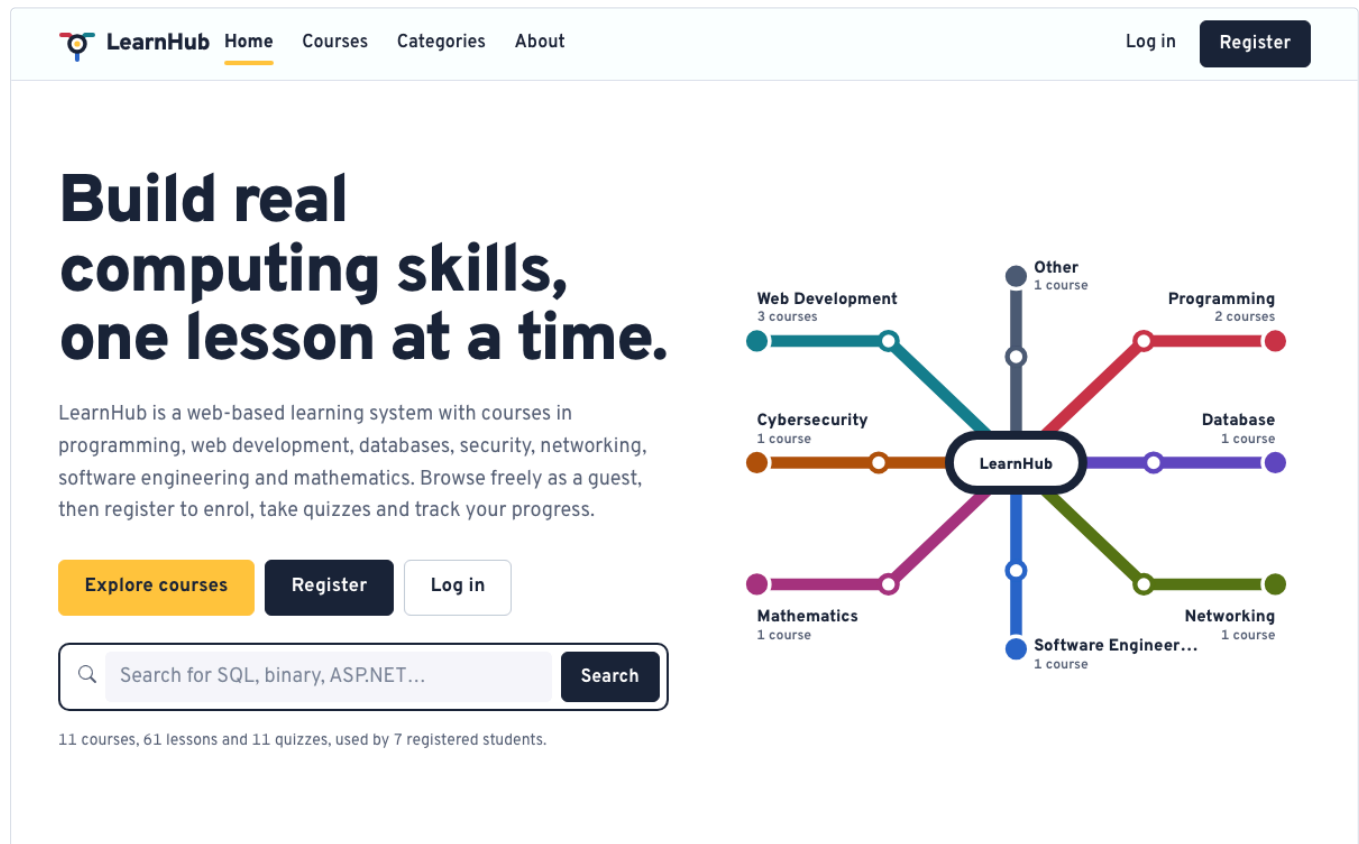
6. Deployment

- **Source and delivery:** the code lives on GitHub, which is both the source of truth and the CI runner. `ci.yml` runs three jobs on every push: build and the whole .NET test suite; the container job that builds the image and runs it the way the host will; and the Playwright browser suite against a production instance over HTTPS.
- **Application host:** Railway builds the multi-stage `Dockerfile` — a .NET 10 SDK stage that publishes the application and an ASP.NET Core 10 runtime stage that runs it — and `railway.json` selects the Dockerfile builder, checks `/health` with a 300-second timeout and restarts the service on failure. Railway supplies `PORT`, and the application reads it so that it listens on `0.0.0.0:<PORT>`; the image sets `PORT=8080` only as a fallback for local runs.
- **Database:** SQLite, with the file on a Railway volume mounted at `/data` and its path supplied as `DATABASE_CONNECTION_STRING=Data Source=/data/learnhub.db`. Migrations, roles, the administrator and the demonstration catalogue are applied at start-up, so the image needs no separate migration step.
- **Persistent state:** the same volume holds the Data Protection key ring under `DataProtection__KeysPath=/data/keys` and administrator uploads under `Storage__RootPath=/data/storage`. Without a volume every redeploy would start from an empty database, sign every user out and lose the uploaded files.
- **Proxy and TLS:** Railway terminates TLS at its edge proxy and forwards plain HTTP with `X-Forwarded-Proto` and `X-Forwarded-For`. `UsePlatformProxyHeaders()` turns those headers into `Request.IsHttps` and the real client address, because the framework does not read them on its own; without it HSTS would never be sent and secure cookies would be refused.
- **Presentation site:** <https://learnhub-wapp.web.app> is live on Firebase Hosting, served from `FirebaseLanding/` with `firebase.json` and `.firebaserc` at the repository root. Firebase cannot execute ASP.NET Core, so this is a static presentation and documentation page whose "Open Learning System" buttons link to the Railway application; after the first Railway deploy, `scripts/set-app-url.sh <railway-url>` rewrites that link and `firebase deploy --only hosting` republishes the site.
- **Status:** the Railway service has **not** been deployed yet — the CLI is installed but not authenticated. Every file and command is ready, and the container job in CI already proves the image, the migrations and the volume behaviour.

The step-by-step guide, configuration reference, verification checklist and rollback procedure are in [DEPLOYMENT.md](#).

7. Screenshots

7.1 Home page



The screenshot shows the LearnHub Home page. The header includes the LearnHub logo, navigation links (Home, Courses, Categories, About), and user actions (Log in, Register). The main content area features a large heading 'Build real computing skills, one lesson at a time.' followed by a description of the platform. Below this is a row of buttons: 'Explore courses', 'Register', and 'Log in'. A search bar with the placeholder text 'Search for SQL, binary, ASP.NET...' and a 'Search' button is also present. At the bottom of the main content, it states '11 courses, 61 lessons and 11 quizzes, used by 7 registered students.' To the right of the text is a radial diagram with 'LearnHub' at the center, connected to various course categories: Web Development (3 courses), Cybersecurity (1 course), Mathematics (1 course), Other (1 course), Programming (2 courses), Database (1 course), Networking (1 course), and Software Engineering (1 course).

Build real computing skills, one lesson at a time.

LearnHub is a web-based learning system with courses in programming, web development, databases, security, networking, software engineering and mathematics. Browse freely as a guest, then register to enrol, take quizzes and track your progress.

[Explore courses](#) [Register](#) [Log in](#)

Search for SQL, binary, ASP.NET... [Search](#)

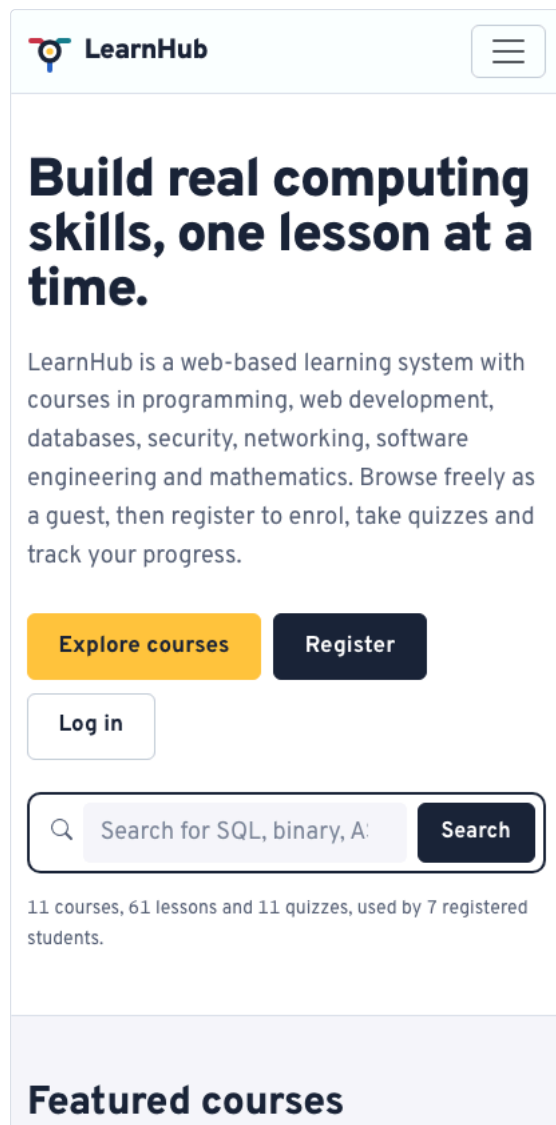
11 courses, 61 lessons and 11 quizzes, used by 7 registered students.

LearnHub categories:

- Web Development: 3 courses
- Cybersecurity: 1 course
- Mathematics: 1 course
- Other: 1 course
- Programming: 2 courses
- Database: 1 course
- Networking: 1 course
- Software Engineering: 1 course

The home page introduces LearnHub, offers a server-side course search and draws each category as a line into the LearnHub hub; every line links to that category's courses. Below are the most popular courses, how learning works and a call to action. Requirements: GUEST-01, UI-01, UI-02.

7.2 Home page on a phone



On small screens the navigation collapses behind a menu button and the map is replaced by category links, so the page stays fast and readable. Requirement: UI-05.

7.3 Course details


[Home](#)
[Courses](#)
[Categories](#)
[About](#)

[Log in](#)
[Register](#)

[Home](#) / [Courses](#) / [Web Development](#) / Modern HTML and CSS


[Web Development](#)
Beginner
9 h
6 lessons
1 quiz
1 learner

About this course

Every website starts with HTML for structure and CSS for presentation. This course teaches both the way professional front-end developers use them today.

You will write semantic HTML5 that works for screen readers and search engines, style pages with the box model, and build responsive layouts with flexbox and CSS grid. Along the way you will test pages on mobile, tablet and desktop screen sizes.

Start learning

Create a free student account to enrol, track your progress and take quizzes.

[Create a free account](#)

[Log in to enrol](#)

Course details show difficulty, duration, lessons, quizzes, learners and instructor, the description and learning outcomes, and the course route. Guests can open the free preview lesson while the other stops are locked; the panel invites them to register or log in to enrol. Requirements: GUEST-06, GUEST-07.

7.4 Lesson player


[Home](#)
[Courses](#)
[Categories](#)
[About](#)

[Log in](#)
[Register](#)

[Courses](#) / [Modern HTML and CSS](#) / How a web page is built

Course route

- How a web page is built
- HTML full tutorial for beginners
- The CSS box model
- CSS from zero to hero
- Responsive layouts with flexbox and grid
- MDN: Learn web development

[Course overview](#)

Article 10 min [Free preview](#)

How a web page is built

HTML gives meaning, CSS gives presentation and JavaScript adds behaviour.

Three languages, three jobs

- HTML** describes what the content *is*: headings, paragraphs, lists, forms, images.
- CSS** describes how it *looks*: colours, spacing, typography and layout.
- JavaScript** describes how it *behaves* when people interact with it.

Keeping the three separate makes pages easier to maintain and more accessible.

Semantic HTML5

```
<header>
  <nav aria-label="Main">...</nav>
</header>
<main>
  <article>
```

An enrolled student reads a lesson with code examples beside the course outline, which shows completed and current stops. The course progress bar sits above the lesson, and the student can mark the lesson complete and move to the next one. Requirements: STU-08, STU-09, TECH-09.

7.5 Student dashboard


[Home](#)
[Courses](#)
[My Learning](#)
[Quizzes](#)
[Progress](#)

 Profile
 [Log out](#)

Welcome back, Aisyah

You have completed 60% of your enrolled courses. Keep going.

[Browse courses](#)

3
Enrolled courses

1
Completed courses

12
Lessons completed

2
Quizzes passed

8
Courses you can still join

Continue learning

Your next lesson in each course you are taking.

[All my courses](#)



Relational Database Design with SQL

12% complete

1 of 8 done

Resume

Next: SQL full database course



ASP.NET Core MVC in Practice

75% complete

6 of 8 done

Resume

Next: HTTP status codes reference

Recent activity

- 
Completed "Tables, keys and relationships"
 Jul 20, 2026, 08:14 AM
- 
Enrolled in Relational Database Design with SQL
 Jul 20, 2026, 12:14 AM
- 
Scored 80% in MVC architecture quiz
 Jul 19, 2026, 02:14 AM

The dashboard shows figures from the database (enrolled and completed courses, lessons completed, quizzes passed, courses still available), courses in progress with Resume buttons, recent quiz results, recent activity and recommendations. Requirements: STU-01, STU-02, STU-03.

7.6 Quiz result


[Home](#)
[Courses](#)
[My Learning](#)
[Quizzes](#)
[Progress](#)

 Profile
 [Log out](#)

Quiz results

Every quiz you have submitted, newest first.

2
Attempts

2
Passed attempts

90%
Average score


LearnHub
 Structured courses, clear lessons and honest progress tracking for

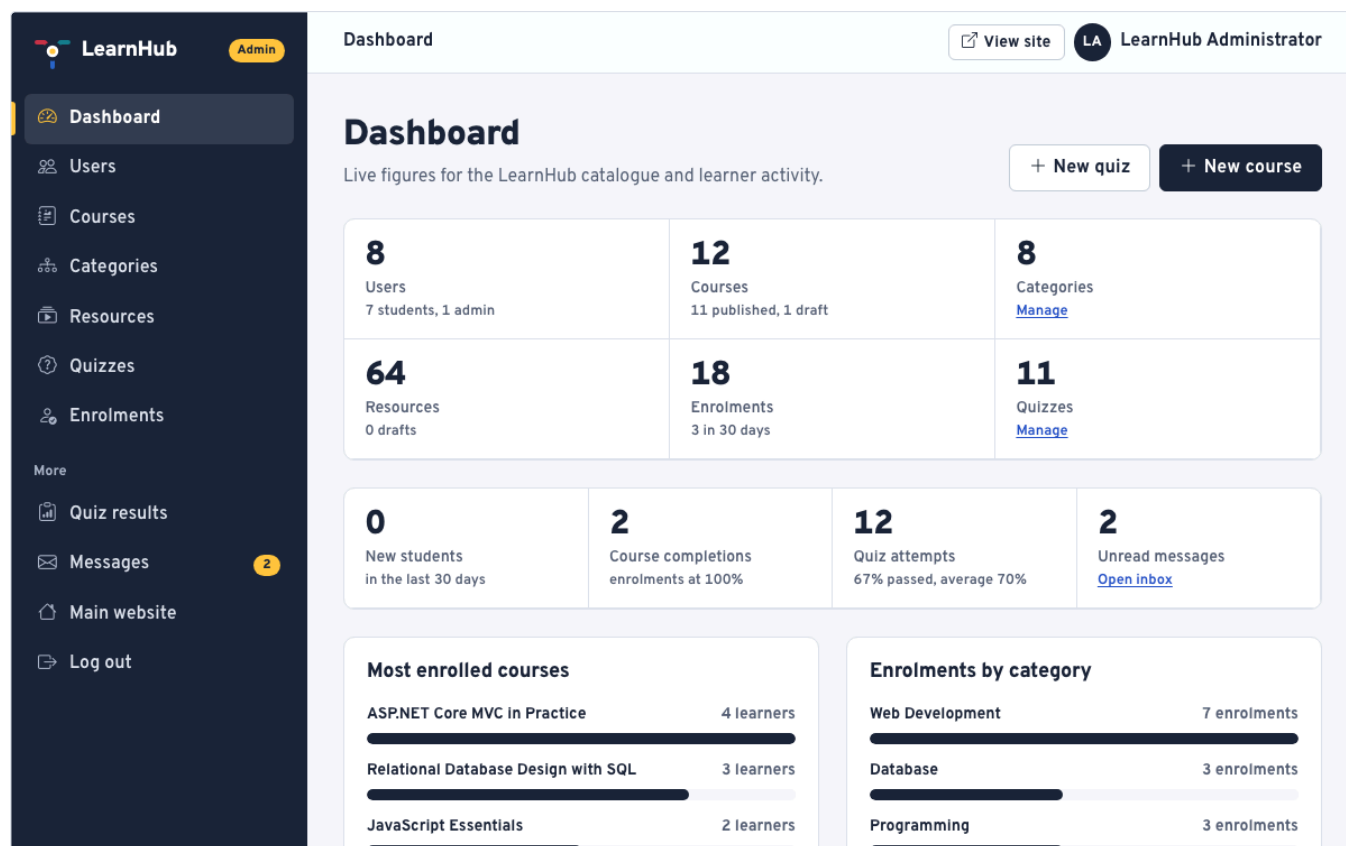
Explore
 All courses
 Categories

Account
 Profile
 Privacy

Project
 CT050-3-2-WAPP Web Applications group assignment Asia Pacific

After submitting, the student sees the score, whether the pass mark was reached, and every question with the chosen answer, the correct answer and an explanation. Requirements: STU-10, STU-11.

7.7 Admin dashboard



Dashboard

Live figures for the LearnHub catalogue and learner activity.

8 Users
7 students, 1 admin

12 Courses
11 published, 1 draft

8 Categories
[Manage](#)

64 Resources
0 drafts

18 Enrolments
3 in 30 days

11 Quizzes
[Manage](#)

0 New students
in the last 30 days

2 Course completions
enrolments at 100%

12 Quiz attempts
67% passed, average 70%

2 Unread messages
[Open inbox](#)

Most enrolled courses

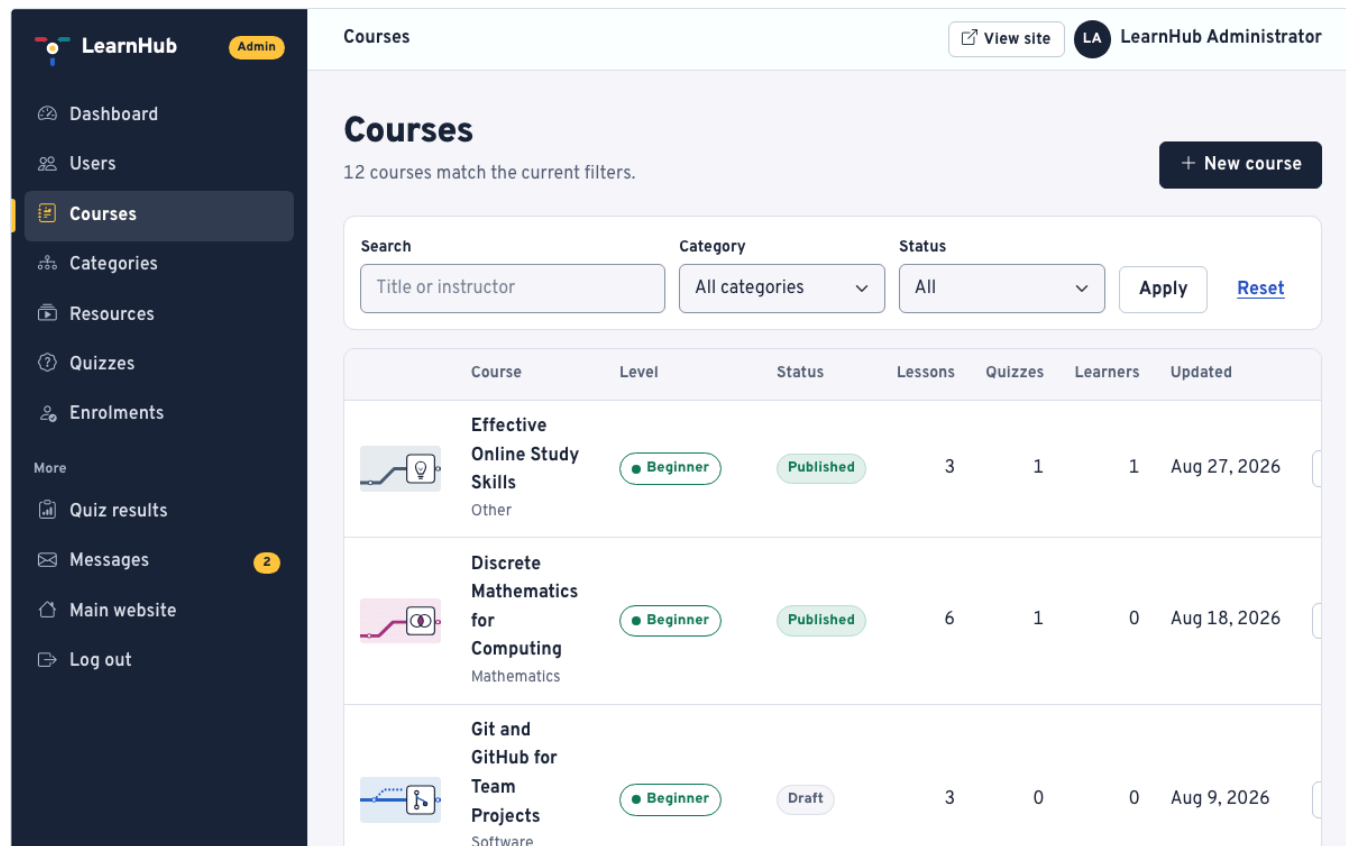
Course	Learners
ASP.NET Core MVC in Practice	4 learners
Relational Database Design with SQL	3 learners
JavaScript Essentials	2 learners

Enrolments by category

Category	Enrolments
Web Development	7 enrolments
Database	3 enrolments
Programming	3 enrolments

The administrator sees live platform figures, the most enrolled courses, enrolments by category and recent activity in the protected admin area with its own navigation. Requirements: ADM-01, ADM-02.

7.8 Course management



Courses

12 courses match the current filters.

Search **Category** **Status**

Title or instructor All categories All Apply [Reset](#)

Course	Level	Status	Lessons	Quizzes	Learners	Updated
 Effective Online Study Skills Other	Beginner	Published	3	1	1	Aug 27, 2026
 Discrete Mathematics for Computing Mathematics	Beginner	Published	6	1	0	Aug 18, 2026
 Git and GitHub for Team Projects Software	Beginner	Draft	3	0	0	Aug 9, 2026

The course list combines search and filters with cover thumbnails, level and publication status, counts of lessons, quizzes and learners, and edit and delete actions. Requirement: ADM-03.

7.9 Further screenshots to add

Screenshot to add: registration form showing validation messages (VAL-02, VAL-03). *Screenshot to add:* admin course form with cover upload (ADM-03, VAL-05). *Screenshot to add:* admin user details with role and deactivation actions (ADM-10). *Screenshot to add:* admin menu opened on a phone (UI-05).

Every GitHub Actions run also stores full-page screenshots of all browser scenarios on desktop, tablet and phone as the `e2e-screenshots` artifact.

8. Critical evaluation

8.1 Strengths

- **Complete journeys:** every role's journey works end to end, and automated tests prove it on the real database engine, inside the container the host will run and at three screen sizes.
- **Security by default:** access control, antiforgery, encoding, upload checks and secrets handling come from the framework and are tested with attack-style cases.
- **Truthful data model:** unique indexes, check constraints, deliberate delete rules, score snapshots and calculated progress.
- **Distinctive, accessible interface:** a consistent design system around one metaphor, with no automated accessibility violations.
- **Reproducible delivery:** one migration set, deterministic seed data, CI checks and a multi-stage `Dockerfile` mean anyone can rebuild the system from the repository and run the image the way the host will.

8.2 Limitations

- No email confirmation or password reset by email, and no multi-factor authentication.
- A single-instance design: migrations run at start-up, and the SQLite file, the key ring and the uploads share one mounted volume, so scaling out would need a different database and object storage.
- The Railway service is configured but not yet deployed, so the application has so far been exercised only locally, in the container job in CI and in the browser tests.
- Manual keyboard, screen-reader and real-device testing still needs to be completed by the team.

8.3 Lessons learned

- The assignment forbids Azure, so the earlier Azure deployment was replaced with Railway and Firebase. Reducing the data layer to a single SQLite provider at the same time removed a whole class of dialect differences from the test matrix and one entire schema to maintain.
- A migration that was never regenerated after a model change left the schema behind the model and broke 126 tests until it was recreated: a model change and its migration belong in the same change.
- Form pages failed on any host that terminates TLS in front of the application until forwarded headers were configured explicitly. `ASPNETCORE_FORWARDEDHEADERS_ENABLED` was an App Service hosting feature, and the framework does not read `X-Forwarded-Proto` on its own.
- Screenshots and browser tests at several sizes found layout bugs that unit and integration tests could not see.
- Designing from real content and one clear concept produced a more coherent interface than adding styles page by page.
- Framework security features, used consistently, are safer and simpler than custom code.

9. Future enhancements

1. Email confirmation, password reset and optional multi-factor authentication.
2. Certificates when a course is completed, and a public profile of achievements.
3. An instructor role that can manage only their own courses.
4. Question banks, randomised questions and timed quizzes.
5. Discussion threads per lesson, moderated by administrators.
6. A managed relational database and object storage for uploads, so the application can run more than one instance instead of sharing one mounted volume.
7. Monitoring and alerting for the deployed service, once the Railway deployment has been made.
8. Dependabot updates and vulnerability scanning in CI.

10. Conclusion

The team delivered LearnHub, a secure, responsive and tested learning management system that meets the assignment's requirements for guests, students and administrators. ASP.NET Core MVC, Entity Framework Core and Identity provided a solid, explainable foundation; a relational model with deliberate constraints keeps data correct; and a distinctive design system makes the product easy to use on any device. Automated testing — 211 .NET test cases, a container check that proves the database survives a restart, and 42 browser and accessibility checks at three screen sizes — together with continuous integration gives confidence that the system works as described and can be maintained by future teams. The presentation site is live on Firebase Hosting, and the Railway service is built, configured and waiting for its first deploy.

References

- Deque Systems. (n.d.). *axe-core* [Computer software]. GitHub. <https://github.com/dequelabs/axe-core>
- Docker. (n.d.). *Multi-stage builds*. Docker Docs. <https://docs.docker.com/build/building/multi-stage/>
- Firebase. (n.d.). *Firebase Hosting*. Firebase Documentation. <https://firebase.google.com/docs/hosting>

Microsoft. (n.d.-a). *Configure ASP.NET Core to work with proxy servers and load balancers*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/host-and-deploy/proxy-load-balancer>

Microsoft. (n.d.-b). *Integration tests in ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/test/integration-tests>

Microsoft. (n.d.-c). *Introduction to Identity on ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/security/authentication/identity>

Microsoft. (n.d.-d). *Migrations overview*. Microsoft Learn. <https://learn.microsoft.com/ef/core/managing-schemas/migrations/>

Microsoft. (n.d.-e). *Model validation in ASP.NET Core MVC and Razor Pages*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/mvc/models/validation>

Microsoft. (n.d.-f). *Overview of ASP.NET Core MVC*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/mvc/overview>

Microsoft. (n.d.-g). *Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/security/anti-request-forgery>

Microsoft. (n.d.-h). *Rate limiting middleware in ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/performance/rate-limit>

Microsoft. (n.d.-i). *Safe storage of app secrets in development*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/security/app-secrets>

Microsoft. (n.d.-j). *SQLite EF Core Database Provider*. Microsoft Learn. <https://learn.microsoft.com/ef/core/providers/sqlite/>

Microsoft. (n.d.-k). *Tag Helpers in ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/intro>

Mozilla. (n.d.). *Content Security Policy (CSP)*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/2025/>

OWASP Foundation. (n.d.-a). *Cross Site Scripting prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *File upload cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Playwright. (n.d.). *Installation*. Playwright documentation. <https://playwright.dev/docs/intro>

Railway. (n.d.). *Using volumes*. Railway Docs. <https://docs.railway.com/volumes>

The Bootstrap Authors. (n.d.). *Get started with Bootstrap (Version 5.3)*. <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>

xUnit.net. (n.d.). *xUnit.net*. <https://xunit.net/>

Appendices

Appendix A – Requirements traceability

Each requirement ID in `REQUIREMENTS_CHECKLIST.md` is mapped to its implementation, evidence and status in `REQUIREMENT_TRACEABILITY.md`.

Appendix B – Test run evidence

`.github/workflows/ci.yml` defines three jobs, all triggered by a push or a pull request:

Job	What it proves
build-and-test	The solution restores and builds with <code>-p:TreatWarningsAsErrors=true</code> , and all 211 .NET test cases pass on SQLite.
container	The Docker image builds, starts in Production with <code>PORT=8080</code> and a volume mounted at <code>/data</code> , answers <code>/health</code> , passes <code>scripts/smoke-test.sh</code> , logs that migrations were applied and the demo data seeded, and — after a restart — does not re-apply migrations, which proves the SQLite file really persisted.
e2e	14 Playwright scenarios × 3 viewports (42 checks) run against a production instance over HTTPS, including axe-core accessibility scans.

Each run stores the TRX result file, the container logs, the Playwright report, the full-page screenshots and the application log as downloadable artifacts. The figures in section 5 were measured on 14 September 2026 by running the same commands locally, and `.github/workflows/ef-migrations.yml` keeps the single SQLite migration regenerated on demand.

Appendix C – User guide

Guest: open the site, search or pick a category on the Courses page, open a course and try a lesson marked "Free preview". Select **Register** to create an account. The illustrated walkthrough is in `USER_GUIDE.md`.

Student: from the dashboard, select **Resume** on a course or open **Courses** and select **Enrol**. Work through the course route, select **Mark as complete** after each lesson, take the quiz when ready and review the explanations. Use **My Courses** for progress, **Quiz results** for history and **Profile** to change details or the password.

Administrator: log in to reach the admin dashboard. Use **Courses**, **Categories** and **Resources** to build the catalogue (drafts stay hidden until published), **Quizzes** to add questions, **Enrolments** and **Users** to manage people, **Quiz results** to review attempts and **Messages** for the contact inbox.

Appendix D – Installation summary

With the .NET 10 SDK: clone the repository and run `dotnet run --project src/LearnHub`. The SQLite database is created, migrated and seeded automatically. In Development, strong random passwords for the DEMO ONLY administrator and student accounts are generated into the git-ignored `App_Data/demo-credentials.json`; set `Seed:AdminPassword` and `Seed:DemoStudentPassword` with `dotnet user-secrets` to choose your own. To run the image the way the host will, build the `Dockerfile` and mount a volume at `/data`; every variable is documented in `.env.example`. Full instructions are in the [README](#); production deployment is in [DEPLOYMENT.md](#).

Appendix E – Individual contribution

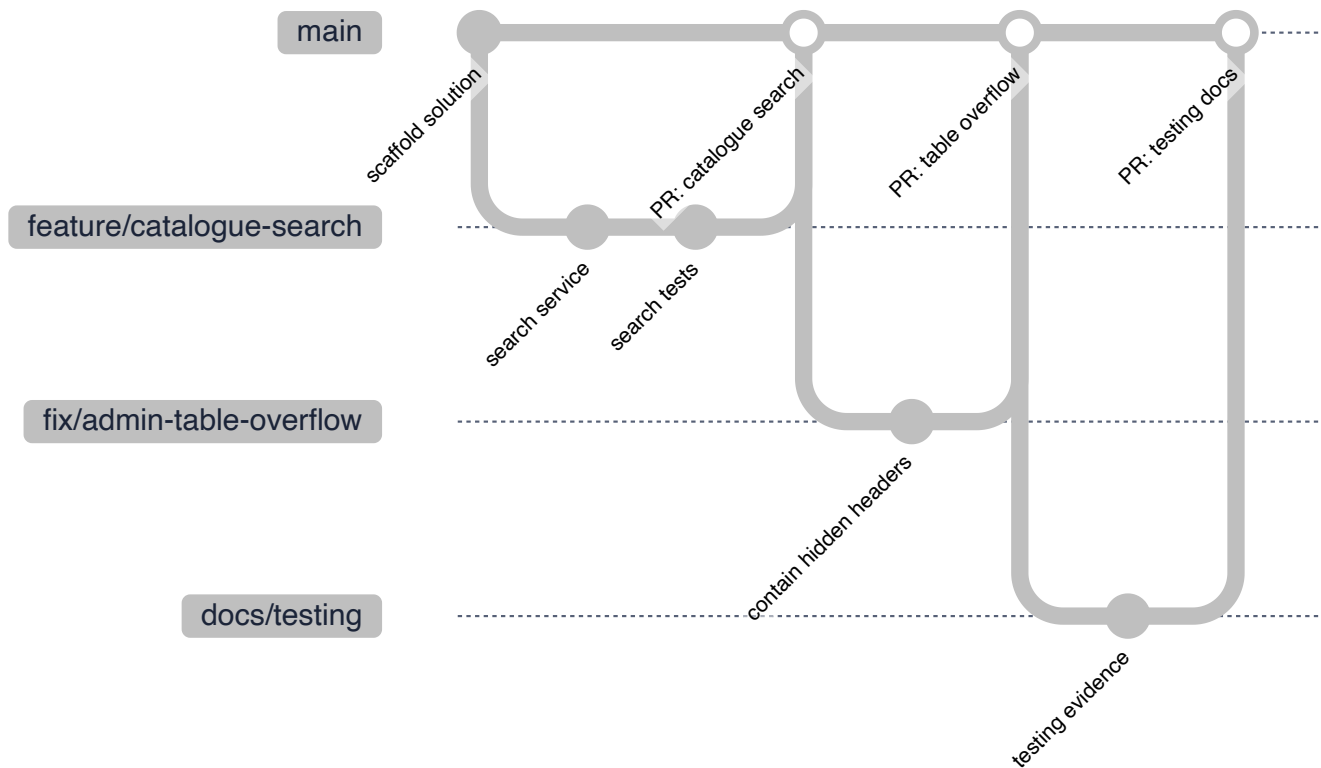
Member	Contribution summary	Signature
Khaytboy Khayrullaev (TP072305)	Sole author: requirements, architecture, database, application code, tests, documentation and deployment — the complete system	

Git Workflow

1. Branches

Branch	Purpose	Rules
<code>main</code>	Always deployable; Railway builds and redeploys the service from this branch once the project is linked	Changes arrive through pull requests with green CI. Enforce this with a branch protection rule (Settings → Branches → require a pull request and the three CI checks)
<code>feature/<area>-<short-name></code>	New functionality, e.g. <code>feature/quiz-history-filter</code>	Branch from <code>main</code> ; keep it small; delete after merge
<code>fix/<short-name></code>	Bug fixes, e.g. <code>fix/admin-table-overflow</code>	Same as feature branches
<code>docs/<short-name></code>	Documentation only, e.g. <code>docs/viva-questions</code>	CI skips documentation-only changes

The current working branch is `feature/railway-firebase-sqlite`, which moves hosting to **Railway** (Docker), storage to **SQLite** on a persistent volume and the presentation site to **Firebase Hosting**. It is still merged into `main` through a pull request like any other branch.



2. Day-to-day flow

```
git switch main && git pull
```

```
git switch -c feature/quiz-history-filter
```

Make the change with its tests, then commit and push:

```
git add -A && git commit -m "feat(quizzes): filter quiz history by course"
```

```
git push -u origin feature/quiz-history-filter
```

Open a pull request into `main`. It can be merged when:

- All three CI jobs pass: **Build and test** (`build-and-test`); **Railway image, migrations and smoke test** (`container`); **Browser tests (mobile, tablet, desktop)** (`e2e`).
- The change has been reviewed against the requirement it serves (see [REQUIREMENT_TRACEABILITY.md](#)); with a single author the CI jobs and that check stand in for a second reviewer, which is why they are treated as mandatory rather than advisory.

3. The description explains what changed and why, with screenshots for visible changes.

Use **Squash and merge** for small branches so `main` keeps one meaningful commit per change, then delete the branch.

3. Commit messages

The repository uses [Conventional Commits](#): `type(scope): summary` in the imperative mood, with a body that explains *why* when it is not obvious.

Type	Use for
<code>feat</code>	New behaviour
<code>fix</code>	Bug fixes
<code>test</code>	Tests only
<code>docs</code>	Documentation only
<code>ci</code>	Workflows
<code>chore</code>	Maintenance such as dependencies or migrations
<code>refactor</code>	Code changes that do not change behaviour

Real examples from this repository:

- `fix: avoid SQL APPLY on SQLite in admin course details, delete via ExecuteDelete, tidy upload file names`
- `fix(ally): keyboard access to scrollable regions, link names on small screens`
- `fix(tests): opt into Microsoft Testing Platform for xUnit v3 on .NET 10`

4. Pull request checklist

- ☐ The change works for every affected role (guest, student, administrator).
- ☐ Server-side validation and authorisation are in place; forms bind to view models.
- ☐ Tests were added or updated, and all CI jobs are green.
- ☐ New pages are responsive, keyboard accessible and pass the accessibility scan.
- ☐ No secrets, personal data, `.env` files, database files or build output are committed.
- ☐ Database changes add one SQLite migration (see section 5) and the ERD is updated.
- ☐ Documentation is updated (README, route tables, ERD or testing evidence) when behaviour changes.

5. Database migrations

LearnHub has one EF Core model, one provider (SQLite) and one migration folder, `src/LearnHub/Data/Migrations`, so every schema change needs **one** migration. The design-time factory that `dotnet ef` uses is `src/LearnHub/Data/DesignTimeDbContextFactory.cs`.

With the .NET SDK installed (run from the repository root):

```
dotnet tool restore
```

```
dotnet ef migrations add AddCourseLevel --project src/LearnHub --output-dir Data/Migrations
```

Without the SDK: push your model change to your branch, open **Actions** → **EF Core migration (cloud)** → **Run workflow**, choose your branch and enter the name in PascalCase (for example `AddCourseLevel`). The workflow creates one SQLite migration in `Data/Migrations`, checks that the solution still builds and commits it to your branch.

Rules:

- Never edit a migration that is already on `main`; add a new one instead.
- If two branches add migrations at the same time, do not hand-merge the model snapshots. Remove your branch's migrations, merge `main` into your branch and generate them again.
- The `container` job starts the image against a fresh volume and asserts that the migration was applied, so a broken migration fails the pull request before anything reaches Railway.

6. Keeping secrets out of Git

- In Development, demo passwords are generated into `App_Data/demo-credentials.json`, which is git-ignored; a fixed local password can be set instead with `dotnet user-secrets set "Seed:AdminPassword" "<password>" --project src/LearnHub`.
- `appsettings.json` keeps `Seed:AdminPassword` and `Seed:DemoStudentPassword` empty, and its connection string is only the path of the SQLite file, so it holds no credential.

- Production takes `Seed__AdminPassword` and `Seed__DemoStudentPassword` from environment variables (Railway service variables). `.env.example` documents every variable with placeholders only, while `.env` is git-ignored.
- `.gitignore` excludes build output, `App_Data`, SQLite databases, uploaded files, `.env` files, publish profiles and test artefacts.
- The repository holds no deployment secret: Railway builds the image from the repository and Firebase Hosting is deployed from the same checkout.
- If a secret is ever committed, treat it as leaked: rotate it immediately, then remove it from the history.

7. Releases and deployment

1. A pull request is merged into `main`.
2. `ci.yml` runs the three jobs — `build-and-test`, `container` and `e2e` — on the merge commit.
3. Railway builds the same `Dockerfile` for the pushed commit and redeployes the service once the project is linked to the repository; the volume at `/data` keeps the database and the Data Protection keys across the restart (see [DEPLOYMENT.md](#)).
4. The presentation site is republished from the same commit when it changes: `firebase deploy --only hosting` from the repository root. It is deployed to Firebase Hosting and live at <https://learnhub-wapp.web.app>.
5. For the assignment milestones, tag the release so the submitted version can always be found:

```
git tag -a v1.0.0 -m "LearnHub 1.0 - assignment submission" && git push origin v1.0.0
```

To roll back, revert the faulty commit on `main` (`git revert <sha>`) and push; see [DEPLOYMENT.md](#).

Team and Responsibilities

Name	Khaytboy Khayrullaev
TP number	TP072305
Email	TP072305@mail.apu.edu.my
Module	CT050-3-2-WAPP Web Applications
Assessment	Group assignment, carried out individually
Role	Sole developer: every area of the application, its database, tests, documentation and deployment

The brief describes a group assignment, and this submission was completed by one member working alone. Every part of the system — requirements, database design, application code, tests, documentation and deployment — was designed, built and verified by the author above. The sections that follow describe what that involved, so the marker can see the full scope of the work rather than a divided set of responsibilities.

1. Areas owned

Every area below was the responsibility of the single member.

Area	What it covers
Requirements and proposal	Interpreting the brief, audience modelling, scope, requirement list and schedule
Architecture	Solution structure, layering, configuration, dependency injection and the request pipeline
Database	Entity model, EF Core configuration, migrations, constraints, indexes and the ERD
Authentication and accounts	ASP.NET Core Identity, roles, registration, login, profile and password change
Public site	Home page, About, Contact, Privacy, the course catalogue, search, filters and paging
Student area	Dashboard, My Courses, enrolment, lessons, quizzes, results and progress tracking
Learning content	Lessons as articles, videos, PDFs, images and exercises; validated uploads and private file streaming
Quizzes	Quiz, question and answer management; points-based grading; attempt history and review
Administration	Admin dashboard plus CRUD for courses, categories, resources, quizzes, questions, answers, users and enrolments
Design system	Colour system, typography, components, responsive layout and accessibility
Security	Authorisation, antiforgery, security headers, rate limiting, upload validation and the security review
Testing	Unit, service, data and integration tests; Playwright browser tests; the smoke test
Deployment	Dockerfile, Railway hosting with a persistent volume, Firebase Hosting for the presentation site
Documentation	All eighteen documents in <code>docs/</code> , the README, the ERD, the wireframes and the PDF build
Viva preparation	Preparing explanations for every part of the system

2. Files owned

Paths are relative to the repository root. With one member the whole repository is in scope; the list below groups it by area so it is clear what was built.

Application and configuration

- `src/LearnHub/Program.cs`, `src/LearnHub/appsettings.json`, `LearnHub.csproj`, `Directory.Build.props`, `Directory.Packages.props`, `global.json`
- `src/LearnHub/Infrastructure/` — middleware, options, tag helpers, validation attributes, claims helpers and the interchange-map renderer
- `src/LearnHub/Controllers/` — Home, Courses, Categories, Account, Profile, Quizzes, Resources, Student, Error
- `src/LearnHub/Areas/Admin/` — the administration controllers and views

Data

- `src/LearnHub/Models/`, `src/LearnHub/Data/ApplicationDbContext.cs`
- `src/LearnHub/Data/Configurations/` — entity configuration, indexes, check constraints and delete rules
- `src/LearnHub/Data/Migrations/` — the single `InitialCreate` migration
- `src/LearnHub/Data/Seed/` — the demonstration catalogue, the seeder and the development password generator

Services

- `src/LearnHub/Services/` — catalogue, category, enrolment, dashboard, progress, resource, quiz, grading, management, user, contact and lookup services
- `src/LearnHub/Services/Storage/` and `src/LearnHub/Services/Content/` — upload rules, file storage, video parsing and lesson rendering

Interface

- `src/LearnHub/ViewModels/`, `src/LearnHub/Views/`, `src/LearnHub/Areas/Admin/Views/`
- `src/LearnHub/wwwroot/` — design system CSS, admin CSS, JavaScript, course artwork and vendored libraries

Tests

- `tests/LearnHub.Tests/` — unit, service, data and integration tests
- `tests/e2e/` — Playwright browser tests across desktop, tablet and mobile

Delivery and documentation

- `Dockerfile`, `.dockerignore`, `railway.json`, `firebase.json`, `.firebaseerc`, `.env.example`, `.gitignore`
- `.github/workflows/ci.yml`, `.github/workflows/ef-migrations.yml`
- `scripts/` — smoke test, application-URL helper, PDF build and CI helpers
- `FirebaseLanding/` — the static presentation site deployed to Firebase Hosting
- `docs/` — all project documentation and the generated PDFs

3. How the work was organised

Working alone, the discipline a group normally supplies had to be built into the process. The practices below were followed throughout.

Practice	How it was applied
Plan before building	Requirements were extracted from the brief into a requirement list before any code was written, and each one was later traced to its implementation in REQUIREMENT_TRACEABILITY.md
Small, single-purpose commits	Work was committed in reviewable steps with messages explaining <i>why</i> , not just what
The build must stay green	<code>dotnet build</code> runs with <code>-p:TreatWarningsAsErrors=true</code> , so no warning is carried forward
Tests alongside features	Every service and controller change came with tests; the suite stands at 211 automated tests
Automated quality gates	Three CI jobs — build and test, the container with its volume proof, and the browser tests — had to pass before work was considered finished
Review against the brief	The traceability document was re-read at the end and every claim checked against the code
No secrets in Git	Demo passwords are generated into a git-ignored file locally and supplied as environment variables in production
Definition of done	Builds, tests pass, CI green, no secrets committed, documentation updated

With no second reviewer, correctness was established by evidence rather than opinion: automated tests, the end-to-end journey checks, the container job that runs the image the way the host runs it, and a final audit in which each requirement was verified against the running system and its database.

4. Individual contribution statement

To be completed in the first person before submission — roughly 100–150 words covering what was built, the hardest problem solved, one thing that would be improved, and links to representative commits.

The hardest problem is worth naming here as a prompt, because it was a genuine one. The application passed every test locally but returned HTTP 500 on every page containing a form once it ran inside a container. The cause was that `KnownIPNetworks = { }` inside an object initialiser adds nothing rather than clearing the default list, so the loopback-only proxy allow-list survived, `X-Forwarded-Proto` was ignored, the request looked insecure and the antiforgery system refused to issue its cookie. It reproduced only from a non-loopback address, which is why local testing never showed it — the container job in CI caught it instead.

5. Marking

With a single author there is no peer assessment to record: the whole of the group mark is attributable to the member named above.

Viva Preparation

Likely questions with model answers based on this code base. File paths are relative to `src/LearnHub/` unless they start with `tests/`, `docs/`, `.github/`, `FirebaseLanding/` or `scripts/`. You should be able to answer the architecture, database, hosting and security questions, not only those in their own area.

1. Project and requirements

Q1. What problem does LearnHub solve? Computing students often study from scattered material with no order and no feedback. LearnHub gives them structured courses (lessons and quizzes in a fixed route), explanations after every quiz and a dashboard that shows progress. Administrators get one protected area to manage courses, files, quizzes, users and enrolments.

Q2. Who are the users and what can each do? Guests browse, search and filter the catalogue, open course details and preview lessons, register, log in and send contact messages. Students additionally enrol, study lessons, mark them complete, take quizzes, review results and edit their profile. Administrators manage every entity in `/Admin` and see platform statistics.

Q3. What was deliberately left out, and why? Payments, certificates, chat, forums, an instructor role, email delivery and multiple languages. They would add external services or large features without improving the core learning journey the brief assesses. They are recorded as future enhancements so the scope stayed under control.

Q4. How do you prove every requirement was met? `docs/REQUIREMENTS_CHECKLIST.md` gives each requirement an ID, and `docs/REQUIREMENT_TRACEABILITY.md` maps each ID to the implementing files and to test or screenshot evidence.

2. Architecture and MVC

Q5. Describe the architecture. One ASP.NET Core MVC application in layers: controllers handle HTTP concerns (binding, `ModelState`, redirects and authorisation attributes); services in `Services/` hold business rules and queries; EF Core's `ApplicationDbContext` talks to the database. Views only display view models. We avoided extra projects or patterns such as CQRS because they would add complexity without benefit at this size.

Q6. Walk through what happens when a student opens a course page. Routing maps `/Courses/Details/5` to `CoursesController.Details`. Middleware has already applied security headers, authentication and authorisation. The action asks `ICourseCatalogService` for a `CourseDetailsViewModel`, which runs an EF Core query (published course, route items, enrolment and progress for this user). The controller returns the view, which Razor renders with the layout.

Q7. Why do services return view models or `OperationResult` instead of entities? Views then receive exactly the data they need, queries select only those columns, and controllers can react to outcomes (success, not found, validation failure) without knowing EF Core. `Services/OperationResult.cs` makes those outcomes explicit.

Q8. How are services wired up? Dependency injection in `Infrastructure/ServiceCollectionExtensions.cs`: services are scoped (one per request, like the `DbContext`); `FileStorageService` and `TimeProvider.System` are singletons. Controllers receive interfaces through constructors, which also lets tests replace them.

Q9. Why is `TimeProvider` injected instead of calling `DateTime.UtcNow`? Services that record times (enrolments, attempts, completions) can be tested with a fixed clock, and the seeder can create realistic past activity relative to one "now".

Q10. What are Areas, why use one for the admin, and how is that area protected? An Area groups controllers and views under `/Admin` with its own layout (`Areas/Admin/Views/Shared/_AdminLayout.cshtml`). Every admin controller inherits `Areas/Admin/Controllers/AdminControllerBase.cs`, which carries `[Area("Admin")]` and `[Authorize(Roles = AppRoles.Admin)]`, so a new admin page cannot be added without protection. The protection is layered: the role comes from ASP.NET Core Identity rather than a flag in the database, the layout only renders the admin navigation for that role, services protect the last active administrator, and administrators cannot demote, deactivate or delete their own account. Authorisation tests post to admin actions as a student and as a guest and expect a refusal, so the protection is verified rather than assumed.

3. Razor views and front end

Q11. What are tag helpers? Give examples from the project. Server-side components that turn attributes into HTML. Built-in ones such as `asp-for`, `asp-action` and `asp-validation-for` generate names, URLs, antiforgery tokens and validation attributes. We wrote our own in `Infrastructure/TagHelpers.cs`: `lh-nav` highlights the current navigation link and sets `aria-current="page"`, and `<course-cover>` renders a cover image or a fallback icon.

Q12. How did you make the site look like LearnHub rather than default Bootstrap? `wwwroot/css/site.css` defines design tokens (ink, paper, signal yellow, eight category line colours, Overpass font, spacing and radii) and restyles components around the interchange idea: category lines, course routes, progress bars with a signal marker, and the home page map. Bootstrap provides the grid, utilities and accessible components.

Q13. How is the home page map built? `Infrastructure/InterchangeMap.cs` builds an SVG with `TagBuilder` (so every value is HTML-encoded): up to eight category lines with 45° bends meeting at the hub, each line a link to the filtered catalogue with an accessible name. The CSS animates the lines drawing in only when the user has not asked for reduced motion; the resting style is the finished map.

Q14. What does JavaScript do, and does the site work without it? `wwwroot/js/site.js` adds progressive enhancements: local time formatting, character counters, the password strength meter, image previews, the quiz answered counter and unanswered warning, confirmation prompts, auto-submitting filters, type-specific resource fields and the answer option editor. Every rule is enforced on the server, so forms still work and stay safe without JavaScript.

Q15. Why are Bootstrap and jQuery stored in `wwwroot/Lib` instead of loaded from a CDN? The Content Security Policy allows scripts, styles and fonts only from our own origin (`'self'`). Loading from a CDN would require weakening the policy and would make the site depend on a third party.

Q16. How did you make the interface responsive and accessible? Mobile-first CSS with Bootstrap breakpoints: the navbar collapses, the admin sidebar becomes off-canvas below 992 px, admin tables become labelled stacked records below 768 px, and course grids change column counts. Accessibility: semantic

landmarks, skip links, labels and hints linked with `aria-describedby`, visible focus styles, `fieldset` and `legend` for quiz questions, and reduced-motion support. Playwright tests run at three sizes, and axe-core reports no WCAG 2.2 A/AA violations.

4. Validation

Q17. How does validation work on the client and on the server? View models carry `DataAnnotations` such as `[Required]`, `[StringLength]`, `[EmailAddress]`, `[RegularExpression]` and `[Compare]` (see `ViewModels/Account/AccountViewModels.cs`). Tag helpers turn them into `data-val-*` attributes that jQuery Validation Unobtrusive enforces in the browser. On submit, model binding validates again and controllers check `ModelState.IsValid` before calling services, because client checks can be bypassed.

Q18. Did you write custom validation attributes? Yes. `Infrastructure/MustBeTrueAttribute.cs` requires the terms checkbox, and `Infrastructure/FileValidationAttributes.cs` checks upload sizes and extensions. Both implement `IClientModelValidator`, so the same rule also runs in the browser through `wwwroot/js/validation.js`.

Q19. Which rules cannot be expressed with attributes? Business rules that need the database or several fields: unique category names, exactly one correct answer among two to six options, a quiz needing a question before publishing, enrolment only in published courses, and protection of the last administrator. Services check them and return `OperationResult` failures that controllers add to `ModelState`.

Q20. How do you keep form limits and database column sizes consistent? `Models/FieldLengths.cs` is the single source of truth: EF Core configurations use it for `HasMaxLength` and view models use it in `[StringLength]`, so a form can never accept more than the column holds.

5. Database design

Q21. Explain the main entities and relationships. A category has many courses. A course has many learning resources, quizzes and enrolments. A quiz has questions, a question has answer options. A student has enrolments, resource completions and quiz attempts; an attempt has one quiz answer per question. Contact messages stand alone. That is eleven application tables plus the seven ASP.NET Core Identity tables. The full diagram is in `docs/ERD.md` and the decisions behind it are in `docs/DATABASE.md`.

Q22. Which constraints protect data integrity? Primary and foreign keys; required columns with maximum lengths; unique indexes on category name, `(UserId, CourseId)` for enrolments, `(UserId, LearningResourceId)` for completions and `(QuizAttemptId, QuestionId)` for answers; and check constraints such as `DurationMinutes > 0` and `ScorePercent BETWEEN 0 AND 100` (`Data/Configurations/`).

Q23. Why does deleting a category restrict, while deleting a course cascades? A category is a label shared by many courses; deleting it by mistake must not wipe courses, so the delete is blocked while courses exist. A course owns its lessons, quizzes and enrolments, which are meaningless without it, and the confirmation page shows exactly what will be removed.

Q24. Why are `QuizAnswer → Question` and `QuizAnswer → AnswerOption` set to Restrict? Because a quiz can be edited after students have taken it. A cascade from a question or an option down to the stored answers would let one edit silently erase a student's answer history, so the foreign keys restrict instead. When a quiz, question or option really has to go, the services delete the dependent answers explicitly first, inside one transaction, so the removal is deliberate and ordered.

Q25. Why do quiz attempts store the score instead of recalculating it? Administrators can edit a quiz after students took it. Storing `CorrectCount`, `QuestionCount`, `ScorePercent`, `Passed` and `IsCorrect` per answer keeps every student's history truthful.

Q26. And why is course progress not stored? Progress depends on the current number of lessons and published quizzes. Calculating it from completions and passed attempts means adding a lesson correctly lowers everyone's percentage, with no stale column to update.

Q27. How are dates handled? All `DateTime` values are stored as UTC through a value converter configured in `Data/ApplicationDbContext.cs`, and `CreatedAt` and `UpdatedAt` are set automatically in `SaveChanges`. The browser converts them to local time.

6. Entity Framework Core

Q28. Why is SQLite the only provider, and what would a second provider cost? `Data/DatabaseServiceCollectionExtensions.cs` calls `UseSqlite` unconditionally, so there is one context, one migration set and one schema to reason about in development, in tests and in production. An earlier version of the project kept SQLite for development and SQL Server for production, which meant an abstract base context, two derived contexts, two migrations folders and a test matrix that ran everything twice. Removing it deleted a whole class of dialect differences — and the bugs they cause — at the cost of accepting SQLite's single-writer, file-based model, which is a reasonable trade for a single-instance course project.

Q29. How are migrations created and applied? `dotnet ef migrations add InitialCreate --project src/LearnHub --output-dir Data/Migrations` uses `Data/DesignTimeDbContextFactory.cs`, the single design-time factory, because the application's own start-up code is not what `dotnet ef` runs. One migration exists — `20260914100832_InitialCreate` — and it creates all eighteen tables: eleven application tables plus the seven Identity tables. `.github/workflows/ef-migrations.yml` regenerates and commits a migration when the model changes, and at start-up `Data/Seed/DatabaseInitializer.cs` applies any pending migration before the application serves a request.

Q30. You found a query that SQLite could not translate. What happened? The admin course details query projected nested collections with `Take`, which translates to a correlated subquery and, on some providers, to SQL `APPLY`. SQLite has no `APPLY`, so the page returned 500 in the tests. We rewrote it as separate flat queries — the course, then its recent enrolments — which SQLite executes happily (`Services/CourseManagementService.cs`).

Q31. Why use `AsNoTracking`, `ExecuteDeleteAsync` and projections? Read-only queries use `AsNoTracking` and project to view models, so EF Core neither tracks entities nor loads unneeded columns. `ExecuteDeleteAsync` deletes rows in the database with one statement, which avoids loading whole object graphs and avoids the relationship errors we hit when deleting tracked dependants.

Q32. How do transactions work here? `Data/TransactionExtensions.cs` runs the unit of work inside `Database.CreateExecutionStrategy().ExecuteAsync`, with the transaction started and committed inside that delegate. SQLite's default strategy simply executes once, so the helper costs nothing today; the point is that the whole unit is retried together the moment a retrying strategy is configured, and the calling code does not have to change if the provider ever does. The seeding path uses the same helper, which is why a slow or interrupted first start cannot leave half a catalogue behind.

Q33. Is LINQ safe from SQL injection? Yes: EF Core sends values as parameters. The catalogue search uses `EF.Functions.Like` with a pattern built by `Services/SearchPattern.cs`, which also escapes `%`, `_` and `[` so they are matched literally.

Q34. What happens if two requests enrol the same student at the same time? Both may pass the "already enrolled" check, but the unique `(UserId, CourseId)` index rejects the second insert. `Services/EnrollmentService.cs` catches the `DbUpdateException`, clears the change tracker and shows "You are already enrolled in this course." instead of an error page.

7. Authentication and authorisation

Q35. How are passwords stored, and why are the demo passwords not in Git? ASP.NET Core Identity stores a salted PBKDF2 hash with many iterations, never the password. The policy (8–100 characters with upper-case, lower-case and a digit) is applied in Identity options and in the registration view model. No password is ever committed: in Development, `DevelopmentSeedPasswords` generates strong random passwords and writes them to `App_Data/demo-credentials.json`, which `.gitignore` excludes, so a fresh clone can sign in as the DEMO ONLY administrator and student without a secret in the repository. In production the passwords must come from the `Seed__AdminPassword` and `Seed__DemoStudentPassword` variables, and `.env.example` documents every variable with a placeholder only.

Q36. How are roles assigned? Registration adds the `Student` role. The first administrator is created at start-up from configuration (`Seed:AdminEmail`, `Seed:AdminPassword`) if no such account exists; after that, administrators change roles in the admin area.

Q37. Why is the admin password not in `appsettings.json`? Anything in Git can leak, and a repository is copied, forked and archived. Locally the password is set with `dotnet user-secrets` or generated into the git-ignored `App_Data/demo-credentials.json`; in production it is a hosting variable (`Seed__AdminPassword`) that exists only in the platform's environment. With no password set, the account is simply not created, so a misconfigured deployment cannot end up with a weak default administrator.

Q38. What protects against brute-force login attempts? Identity lockout (five failed attempts lock the account for 15 minutes), a generic error message that does not reveal whether an email exists, and a rate limit of 10 form submissions per minute per IP on login, registration, password change and contact.

Q39. How does deactivating a user work, and how fast does it take effect? `Services/UserManagementService.cs` sets `LockoutEnd` to the maximum date and updates the security stamp. Identity refuses future sign-ins, and because the cookie's security stamp is re-validated every five minutes, an existing session ends within that time. Administrators cannot deactivate, demote or delete themselves or the last active administrator.

Q40. How does the app redirect users after login without an open redirect? `AccountController.RedirectToLocal` follows `returnUrl` only when `Url.IsLocalUrl` accepts it; otherwise administrators go to `/Admin` and students to their dashboard. A test tries an external URL and expects the dashboard.

8. Security

Q41. How do you prevent CSRF? `AutoValidateAntiforgeryTokenAttribute` is registered globally, so every unsafe HTTP method needs a valid token, which the form tag helper adds automatically. All state changes are POST forms. A test posts to login without a token and expects 400.

Q42. How do you prevent XSS, especially in lesson content written by admins? Razor encodes all output and the views never use `Html.Raw` on user input. Lesson text goes through `Services/Content/LessonContentRenderer.cs`, which encodes everything first and then applies a fixed allow-list of formatting. A strict CSP blocks inline scripts as a second layer.

Q43. What is IDOR and where did you prevent it? Insecure direct object reference: changing an id in a URL to reach someone else's data. `QuizService` returns a result only for the attempt's owner (others get 404), lessons and files require enrolment unless they are previews, and admin actions require the Admin role. Tests cover each case.

Q44. Why not bind the entity directly in forms? Overposting: a user could add fields such as a role flag or `IsPublished` to the request. Forms bind to view models that contain only editable fields, and services copy those values onto entities.

Q45. How are file uploads secured? `Services/Storage/FileStorageService.cs` checks the declared size, the extension allow-list and the file signature (the first bytes must be JPEG, PNG, WebP or PDF), saves under a random name outside `wwwroot`, and enforces the size again on the bytes actually written. Resource files are streamed by `ResourcesController.Open` after the access check.

Q46. How are embedded videos kept safe? `Services/Content/VideoEmbedParser.cs` accepts only known YouTube and Vimeo URL formats, extracts the video id and builds a `youtube-nocookie.com` or `player.vimeo.com` embed URL itself. The CSP `frame-src` allows only those two players.

Q47. Which security headers are sent? `Infrastructure/SecurityHeadersMiddleware.cs` sends a Content Security Policy, `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy`, `Permissions-Policy` and `Cross-Origin-Opener-Policy`; production also uses HTTPS redirection and HSTS.

Q48. How does the application reach its database, and is there a database password to protect? There is no database server to authenticate to. SQLite is a file, so the application opens `Data_Source=/data/learnhub.db`, a path supplied by the `DATABASE_CONNECTION_STRING` variable; the only credential in the system is the SQLite file itself, protected by the container's file system and by the platform's access control to the service and its volume. That removes an entire class of risk — no connection string with a password, no firewall rule and no network database port to expose — and it is why the volume is the piece that matters: whoever has the file has the data.

Q49. How are production secrets supplied now that there is no cloud key vault? Through hosting variables. `.env.example` documents each one — `DATABASE_CONNECTION_STRING`, `Seed__AdminPassword`, `Seed__DemoStudentPassword`, `DataProtection_KeysPath`, `Storage__RootPath` — with placeholders only, while the real `.env` file is git-ignored, and GitHub Actions generates throw-away credentials per run, masks them in the log and never stores them. For a project of this size that is proportionate: the secrets live in one place, they are never in the repository, and no code change is needed when the environment changes because ASP.NET Core reads the variables directly.

9. Testing

Q50. What kinds of tests did you write? Unit tests (grading, progress, validation, content safety, file signatures), service tests against a real in-memory SQLite database, integration tests that run the whole app in memory with `WebApplicationFactory`, a crawler that follows every link as each role, Playwright browser scenarios at three screen sizes with axe-core accessibility scans, a container job that starts the Docker image the way the host will and smoke tests it, and `scripts/smoke-test.sh` for a running instance. In total 211 .NET test cases and 42 browser checks (14 browser tests x 3 viewports).

Q51. How do integration tests log in and submit forms? `tests/LearnHub.Tests/Infrastructure/BrowserClientExtensions.cs` gets the page, extracts the antiforgery token from the HTML, posts the form with cookies kept by the client and follows redirects manually, just like a browser.

Q52. How do you test the container and the volume the way the host will run them? The `container` job in `.github/workflows/ci.yml` builds the same `Dockerfile` Railway builds, then runs it with `ASPNETCORE_ENVIRONMENT=Production`, `PORT=8080` and a volume mounted at `/data`. It waits for `/health`, runs the smoke test, asserts from the log that migrations were applied, the demo data was seeded and the administrator was created with no `fail:` or `crit:` lines, and then restarts the container and asserts that migrations were **not** re-applied. That last assertion is the real test of persistence: the SQLite file and the Data Protection key ring only survive the restart because they live on the volume.

Q53. Which bugs did testing find? The SQLite `APPLY` failure, a relationship error when deleting courses, form pages failing behind a TLS-terminating proxy until forwarded headers were configured, a migration that was never regenerated after a model change and broke 126 tests, several layouts scrolling sideways on phones and tablets, a transparent admin sidebar, map labels overlapping, and accessibility issues (unfocusable scroll regions, links without names on phones). All are listed with their fixes in `docs/TESTING.md`.

10. Deployment, hosting and persistence

Q54. Why is the application on Railway while Firebase Hosting serves a separate site? Because Firebase Hosting cannot host it. Firebase Hosting serves static files only: there is no server-side runtime, so it cannot execute ASP.NET Core, and it offers no writable file system for a SQLite database or uploads. The application therefore runs as a Docker container on Railway, and the presentation site stays a static page on Firebase, which links to the application with its "Open Learning System" buttons. That split is deliberate: each host does the one thing it is good at.

Q55. What do the GitHub Actions workflows do? `ci.yml` has three jobs. `build-and-test` restores, builds with `-p:TreatWarningsAsErrors=true` and runs the whole .NET test suite. `container` builds the Docker image and runs it the way Railway does — `Production`, `PORT=8080`, a volume at `/data` — smoke tests it, checks the log, and restarts it to prove the data persisted. `e2e` runs the Playwright suite against a production instance over HTTPS. `ef-migrations.yml` regenerates and commits the SQLite migration when the model changes. The old Azure deploy, GitHub Pages and provisioning workflows were removed along with the platform they targeted.

Q56. What does `UsePlatformProxyHeaders` do, and why is it needed? Railway terminates TLS at its edge proxy and forwards plain HTTP inside the platform, describing the original request in `X-Forwarded-Proto` and `X-Forwarded-For`. ASP.NET Core does not read those headers by itself, so `ApplicationBuilderExtensions.UsePlatformProxyHeaders()` enables `ForwardedHeadersMiddleware` for exactly those two headers. It is needed because without it the application believes every request arrived over HTTP: HSTS would never be sent, secure cookies and HTTPS redirection would misbehave, and every request would look like it came from the proxy — which matters because the rate limiter partitions its fixed window by `context.Connection.RemoteIpAddress`, so all users would share one counter and one abuser could lock everybody out. The loopback-only default allow-list is cleared deliberately, because the platform proxy is the only route into the container and arrives from a private-network address; the middleware therefore runs first, before anything that inspects the scheme, the client address or a generated link. `ASPNETCORE_FORWARDEDHEADERS_ENABLED` does not do this job — it was an Azure App Service hosting feature, not part of the framework.

Q57. Why is `PORT` read from the environment? Platforms like Railway choose the port when the container starts and pass it in `PORT`; hard-coding a port would mean the router and the application disagree. `UsePlatformPort()` reads the variable at start-up and binds `http://0.0.0.0:{PORT}` so the container accepts traffic from outside its own network namespace, and falls back to the normal ASP.NET Core URL configuration when `PORT` is absent. The `Dockerfile` sets `PORT=8080` only as a fallback for running the image locally, which is also how the CI container job exercises it.

Q58. Why SQLite, and how is the database persisted on Railway? SQLite keeps the whole relational model in one file with real foreign keys and transactions, and it needs no server, no connection credential and no network hop, which suits a single-instance academic application. The catch is that a container's own file system is disposable, so the file must not live there. The connection string comes from the `DATABASE_CONNECTION_STRING` variable (`Data Source=/data/learnhub.db`), and `/data` is a Railway volume: storage that is mounted into the container and survives restarts, redeploys and image changes. `Data/DatabaseServiceCollectionExtensions.cs` resolves a relative path against the content root and creates its folder, so the same code works with the default `App_Data/learnhub.db` on a developer machine and with the volume path in production.

Q59. What lives on the `/data` volume, and what breaks without it? Three things, and each fails differently. The SQLite file (`/data/learnhub.db`) holds every course, enrolment, attempt and account, so without the volume each redeploy starts from an empty database and re-seeds the demo catalogue. The Data Protection key ring (`DataProtection__KeysPath=/data/keys`) encrypts the authentication cookies, so without it every restart invalidates them and signs every user out — re-running migrations would not fix that. Administrator uploads (`Storage__RootPath=/data/storage`) hold the course thumbnails and resource files, so without it the database rows survive while the files they point at disappear. This is exactly what the `container` job checks when it restarts the container and asserts that migrations were not re-applied.

Q60. How are migrations and seeding run at start-up, and is that safe? `Program.cs` calls `InitializeDatabaseAsync()` after the routes are mapped and before the application starts serving. `Data/Seed/DatabaseInitializer.cs` applies pending migrations when `Database:ApplyMigrationsOnStartup` is true (the default), creates the `Admin` and `Student` roles, creates the administrator from configuration if no such account exists, and then lets `DemoDataSeeder` fill an empty database with the demonstration catalogue. Seeding runs only when no categories exist, so it can never duplicate or overwrite real data. For this single-instance deployment the approach is safe — one container, and EF Core takes a migration lock so two instances could not migrate at once — and it removes the "forgot to migrate" failure mode that had already broken 126 tests once. The trade-off is that a start-up migration is part of the request path's readiness: it is why the health check is `/health` and why the Railway health check timeout is 300 seconds.

Q61. How is the image built and configured for Railway? The `Dockerfile` is multi-stage: `mcr.microsoft.com/dotnet/sdk:10.0` restores and publishes the project, and `mcr.microsoft.com/dotnet/aspnet:10.0` carries only the published output, so the SDK and the NuGet cache never reach the running container.

`railway.json` selects the Dockerfile builder, sets the health check to `/health` with a 300-second timeout, and restarts the service on failure. There is no separate migration step in the pipeline because the container migrates and seeds itself on start-up.

Q62. What is deployed today? Firebase Hosting is live at <https://learnhub-wapp.web.app>: the landing page, its design system, the Overpass webfont and the eight screenshots are served from `FirebaseLanding/`, and unknown paths return the custom 404 page. The Railway service has **not** been deployed yet — the CLI is installed but not authenticated, so we can honestly say the image, the migrations and the volume behaviour are proven in CI and locally, while the hosted deployment is the next step.

Q63. How does the Firebase site point at the application? The landing page ships with the conventional address `https://learnhub-production.up.railway.app`, because Railway generates the real domain when the service is created. After the first deploy, `scripts/set-app-url.sh <railway-url>` rewrites every call-to-action in `FirebaseLanding/index.html`, and `firebase deploy --only hosting` republishes the site. `firebase.json` sets the public folder to `FirebaseLanding` with caching headers, and `.firebaserc` sets the default project to `learnhub-wapp`.

11. Teamwork and reflection

Q64. How did the team avoid conflicts? Clear ownership per area, short-lived branches, pull requests with green CI and review, small changes to shared files such as `site.css` and the EF Core model, and regenerating migrations instead of hand-merging snapshots.

Q65. What are the main limitations? No email confirmation or password reset by email, no multi-factor authentication, and a single-instance design: the SQLite file, the Data Protection key ring and the uploads share one mounted volume, so scaling out would need a different database and object storage. The hosted Railway deployment has not been made yet, and there is no monitoring or alerting.

Q66. What would you add next? Email confirmation and password reset, MFA for administrators, certificates on course completion, an instructor role, a managed relational database with object storage so the application can run more than one instance, monitoring and alerting for the deployed service, and Dependabot for dependency updates.